

Rientr@

Semantic Microservice

Documentazione Tecnica

Moduli: `main.py`, `models.py`, `reasoner.py`, `importer.py`,
`electron/main.js`, `electron/preload.js` + Ontologia &
Configurazione

Versione	docu-	2.3
mento:		
File Python:		<code>main.py</code> , <code>models.py</code> , <code>reasoner.py</code> , <code>importer.py</code>
File Electron:		<code>electron/main.js</code> , <code>electron/preload.js</code>
Frontend React:		<code>App.tsx</code> , <code>WorkersPage</code> , <code>JobAnalysisView</code> , <code>HealthConditionWizard</code>
API layer:		<code>semanticService.ts</code>
Config:		<code>package.json</code> , <code>vite.config.ts</code> , <code>tsconfig.json</code>
Framework:		FastAPI + Uvicorn + Pydantic v2
Reasoner:		Pellet via owlready2
Standard:		OWL 2 DL + SWRL + R2RML

Documento prodotto per uso interno di progetto. Da integrare con la documentazione degli altri moduli.

Contents

Elenco delle figure	10
Elenco delle tabelle	12
Elenco dei listing	15
1 Introduzione e contesto di progetto	16
1.1 Il sistema Rientr@	16
1.2 Posizione nell'architettura generale	16
2 Dipendenze e import	17
2.1 Librerie standard Python	17
2.2 Dipendenze esterne	17
2.3 Moduli interni	18
3 Ciclo di vita dell'applicazione (Lifespan)	21
3.1 Motivazione del pattern lifespan	21
3.2 Sequenza di inizializzazione	21
4 Configurazione dell'applicazione FastAPI	23
4.1 Istanza dell'applicazione	23
4.2 Middleware CORS	23
5 Guard di disponibilità del reasoner	25
5.1 La funzione <code>_require_ready()</code>	25
6 Endpoint REST	27
6.1 Panoramica	27
6.2 Tag: Service	28
6.2.1 GET <code>/status</code>	28
6.3 Tag: Workers	28
6.3.1 GET <code>/workers</code>	28
6.3.2 GET <code>/workers/{worker_id}</code>	28
6.3.3 POST <code>/workers/select</code>	28
6.3.4 POST <code>/workers/{worker_id}/jobs/update</code>	29
6.4 Tag: Jobs	29
6.4.1 GET <code>/jobs/{job_id}/importance</code>	29
6.4.2 GET <code>/jobs/{job_id}/profile</code>	29
6.5 Tag: Matching	29
6.5.1 Query Q1+Q2 — GCS% e AISA%	30

6.5.2	Query Q3 — Dettaglio per coppia (worker, job)	30
6.6	Tag: Health Conditions	31
6.6.1	GET /health-conditions/{worker_id}	31
6.6.2	POST /health-conditions/{worker_id}/update	31
6.6.3	GET /icf-codes e GET /core-sets	31
6.7	Tag: Import	32
6.7.1	POST /import/workers	32
7	Gestione degli errori	34
7.1	Strategia generale	34
7.2	Gerarchia dei codici di errore	34
8	Entrypoint di sviluppo	35
9	Modulo <code>models.py</code> — Schemi Pydantic v2	36
9.1	Ruolo e motivazione del modulo	36
9.2	Dipendenze del modulo	37
9.3	Panoramica dei gruppi di modelli	37
9.4	Gruppo: Status	38
9.4.1	OntologyStats	38
9.4.2	StatusResponse	38
9.5	Gruppo: Workers	38
9.5.1	WorkerSummary e WorkerDetail	38
9.5.2	SelectWorkerRequest e SelectWorkerResponse	39
9.5.3	UpdateWorkerJobsRequest e UpdateWorkerJobsResponse	39
9.6	Gruppo: Jobs	40
9.6.1	JobSummary	40
9.6.2	JobSkillProfileEntry	40
9.6.3	ImportanceEntry e SkillScore	41
9.7	Gruppo: Matching	41
9.7.1	MatchResult	41
9.7.2	MatchDetailRequest	42
9.7.3	SkillDetail e SkillDetailResponse	42
9.8	Gruppo: Health Conditions	43
9.8.1	HealthCondition e HealthConditionsResponse	43
9.8.2	IcfCodeEntry	44
9.8.3	HcChangeItem, UpdateHealthConditionsRequest e UpdateHealthConditionsResponse	44
9.9	Diagramma delle relazioni tra modelli	45
9.10	Convenzioni adottate nel modulo	46

10	Modulo <code>reasoner.py</code> — Logica semantica e ragionamento	47
10.1	Ruolo e responsabilità del modulo	47
10.2	Dipendenze e silenziamento dei log	47
10.3	Costanti IRI dell'ontologia Rientr@	48
10.3.1	Soglie di idoneità (Fig. 4 del paper)	49
10.4	Classe <code>ReasonerState</code> — Stato globale del servizio	50
10.4.1	Variabili globali di modulo	51
10.5	Funzioni di utilità	51
10.5.1	<code>local_name(entity)</code>	52
10.5.2	<code>criticality_label(cs)</code>	52
10.5.3	<code>job_suitability(gcs, aisa)</code>	52
10.5.4	<code>_score_to_anchor(score)</code>	53
10.6	Sistema di cache snapshot	53
10.6.1	Strategia generale	53
10.6.2	<code>_ontology_fingerprint(path)</code>	53
10.6.3	<code>load_snapshot_cache(rdf_path, update_state)</code>	55
10.6.4	<code>_save_snapshot_cache(source_path, elapsed, stats)</code>	55
10.6.5	<code>_use_snapshot_cache()</code>	56
10.7	Fase 1 — Caricamento dell'ontologia	56
10.7.1	<code>_find_rdf_file()</code> e <code>_resolve_rdf_path()</code>	56
10.7.2	<code>_load_ontology(path)</code>	56
10.8	Fase 2 — Esecuzione di Pellet	57
10.8.1	<code>_run_pellet(onto)</code>	57
10.9	Entry point pubblico — <code>load_and_reason()</code>	58
10.10	Query SPARQL — Pattern generale	59
10.11	Query Q0 — Workers e Jobs	60
10.11.1	<code>get_workers()</code>	60
10.11.2	<code>get_jobs()</code>	60
10.12	Query — Condizioni di salute ICF	60
10.12.1	<code>get_health_conditions(worker_id)</code>	60
10.12.2	<code>get_all_icf_codes()</code>	61
10.13	Query Q4 — Importanza skill per job	61
10.13.1	<code>get_importance_summary(job_id)</code>	61
10.14	Query Q1+Q2 — GCS% e AISA%	62
10.14.1	<code>get_match_results(worker_id)</code>	62
10.14.2	Calcolo aggregato <code>_compute_metrics(agg)</code>	63
10.14.3	Fallback <code>_match_results_fallback()</code>	64
10.15	Query Q3 — Dettaglio skill per coppia (worker, job)	64
10.15.1	<code>get_skill_detail(worker_id, job_id)</code>	64

10.16	Mutazione — Selezione del worker (<code>set_selected_worker</code>) . . .	64
10.17	Mutazione — Aggiornamento condizioni ICF	65
10.17.1	Struttura del grafo HC nell'ontologia	65
10.17.2	<code>_safe_set_prop(prop, ind, value)</code>	65
10.17.3	Logica delle tre azioni (ADD / REMOVE / MODIFY) . . .	66
10.17.4	Routing BFqual / AP1qual	66
10.18	Mutazione — Aggiornamento job assegnati	67
10.18.1	<code>update_worker_jobs(worker_id, job_ids)</code>	67
10.19	<code>_refresh_reasoning_artifacts()</code> — Pipeline di re-reasoning .	68
10.20	<code>inject_imported_workers()</code> — Integrazione post-import	68
10.21	Mappa ICF → Core Set	69
10.21.1	<code>_build_icf_core_set_map()</code>	69
10.22	Concorrenza e thread safety	70
10.23	Flusso dati completo — Dalla query al JSON	70
11	Ontologia Rientr@ — Struttura e contenuto	72
11.1	Panoramica dell'ontologia	72
11.2	Statistiche degli individui	73
11.3	Classi principali	74
11.4	ICF Core Set inclusi	75
11.5	Proprietà OWL	75
11.5.1	Proprietà oggetto (Object Properties)	75
11.5.2	Proprietà dato (Datatype Properties)	76
11.6	Regole SWRL	76
11.6.1	Famiglia A — Calcolo del Criticality Score (regole 1–9) . . .	77
11.6.2	Famiglia B — Classificazione dell'importanza per job (regole 10–13)	78
11.7	Struttura del grafo di matching	78
12	Setup, deployment e configurazione	80
12.1	Requisiti di sistema	80
12.2	Dipendenze Python (<code>requirements.txt</code>)	81
12.3	Procedure di installazione	82
12.4	Modalità di avvio e readiness	82
12.5	Integrazione con Spring Boot	83
12.6	Configurazione Pyright (<code>pyrightconfig.json</code>)	84
13	Modulo <code>import_workers/importer.py</code> — Pipeline RDB2RDF	86
13.1	Ruolo, contesto e versione	86
13.2	Perché RDB2RDF: motivazione del paradigma	86

13.3	Perché Morph-KGC come engine R2RML	87
13.4	Dipendenze e importazione condizionale	88
13.4.1	Pattern <code>_DEPS_OK</code> e stub <code>_SAFE_ANY</code>	88
13.4.2	Namespace e costanti di modulo	88
13.4.3	<code>ImportResult</code> — dataclass di risposta	89
13.5	Funzione pubblica <code>ensure_mapping()</code>	90
13.6	Generazione del mapping R2RML — <code>_generate_mapping()</code>	90
13.6.1	Scoperta automatica delle proprietà FOAF:Person	90
13.6.2	Correzione colonna <code>ZIPcode</code>	91
13.6.3	Le 5 TriplesMap del mapping R2RML	92
13.6.4	Il mapping generato: <code>rientra_mapping.ttl</code>	92
13.7	Perché le VIEW SQL invece di query R2RML complesse	94
13.7.1	<code>_create_views(conn, known_icf, known_jobs)</code>	94
13.8	Perché SQLite in-memory	95
13.8.1	<code>_load_sql(sql_path)</code>	95
13.9	Struttura del dataset SQL di input	96
13.10	Il problema del boolean con Pellet	97
13.10.1	Analisi del problema	97
13.10.2	Soluzione definitiva	98
13.11	Perché l’iniezione testuale invece della re-serializzazione	98
13.12	Conversione triple → blocchi RDF/XML	99
13.12.1	<code>_triples_to_rdfxml_blocks(new_graph, persons_to_add)</code>	99
13.12.2	Uso di <code>xml_escape</code>	99
13.13	Estrazione entità dall’ontologia	99
13.13.1	<code>_extract_known_icf(rdf_text)</code>	100
13.13.2	<code>_extract_known_jobs(ontology_path)</code>	100
13.13.3	<code>_person_exists(rdf_text, person_id)</code>	101
13.13.4	<code>_sanitize_id(name)</code>	101
13.14	Separazione <code>-generate-mapping</code> dall’import	102
13.15	Struttura degli IRI generati	102
13.16	Entry point pubblico — <code>import_sql_dataset()</code>	103
13.17	Architettura della pipeline — diagramma completo	104
14	Layer applicativo Electron — Struttura del progetto grafico	105
14.1	Architettura generale dell’applicazione desktop	105
14.2	Architettura a tre processi di Electron	107
14.3	Modulo <code>electron/main.js</code> — Processo principale	107
14.3.1	Ruolo e responsabilità	107

14.3.2	Risoluzione dell'eseguibile Python	108
14.3.3	<code>startPythonService()</code> — Avvio del sottoprocesso	108
14.3.4	<code>stopPythonService()</code> — Terminazione pulita	109
14.3.5	Creazione della finestra — <code>createWindow()</code>	110
14.3.6	Dev vs Production — caricamento dell'interfaccia	110
14.3.7	Gestione del menu nativo	111
14.3.8	Handler IPC esposti al renderer	112
14.4	Modulo <code>electron/preload.js</code> — Bridge sicuro	112
14.4.1	Ruolo e principio di funzionamento	112
14.4.2	API esposte: <code>window.electronAPI</code>	113
14.4.3	Perché <code>contextBridge</code> è necessario	114
14.4.4	Flusso di import del dataset SQL: end-to-end	114
14.5	Ciclo di vita dell'applicazione Electron	114
15	Layer React — Sorgenti <code>src/</code>	117
15.1	Struttura della directory <code>src/</code>	117
15.2	<code>src/main.tsx</code> — Entry point React	118
15.2.1	Scelte implementative	118
15.3	<code>src/App.tsx</code> — Root component e routing	119
15.3.1	Pattern di routing senza React Router	120
15.3.2	Tipi esportati	120
15.3.3	Flusso di navigazione	121
15.3.4	Tre punti di ingresso alla <code>WorkersPage</code>	121
15.4	<code>src/index.css</code> — Reset e stili globali	122
15.4.1	Scelte implementative	122
15.5	<code>src/App.css</code> — Stili legacy del template Vite	123
15.6	<code>src/components/HomePage.tsx</code> e <code>HomePage.css</code>	123
15.6.1	Struttura del file e responsabilità	123
15.6.2	Il tipo <code>NavCard</code> e l'array <code>cards</code>	124
15.6.3	Icone SVG inline — componenti funzionali	125
15.6.4	Layout e struttura JSX	125
15.6.5	<code>HomePage.css</code> — sistema visivo	127
15.7	<code>src/components/WorkersPage.tsx</code> e <code>WorkersPage.css</code>	128
15.7.1	Ruolo, dimensioni e dipendenze	128
15.7.2	Icone SVG inline del componente	129
15.7.3	Parser <code>parseConditionDescription</code>	129
15.7.4	Componente <code>LoadingScreen</code> — schermata di warm-up	130
15.7.5	Inventario dello stato di <code>WorkersPage</code>	131
15.7.6	Strategia di polling adattivo	131

15.7.7	Selezione del worker — sequenza critica	133
15.7.8	Sistema di ordinamento multi-colonna	134
15.7.9	Tabella ICF — rendering a doppia riga	135
15.7.10	Il qualificatore effettivo	135
15.7.11	Gestione import — flusso completo con stati	136
15.7.12	Navigazione a tre modalità e sidebar collassabile	137
15.8	<code>src/components/JobAnalysisView.tsx</code> e <code>JobAnalysisView.css</code>	137
15.8.1	Ruolo, dimensioni e dipendenze	138
15.8.2	Costanti e mappe di critica	138
15.8.3	Componente <code>ScatterPlot</code> — SVG responsive con <code>ResizeObserver</code>	139
15.8.4	Assi dinamici: <code>axisMax</code> e <code>makeTicks</code>	139
15.8.5	Rette di soglia SVG clippate	140
15.8.6	Sidebar dei job ordinata per idoneità	140
15.8.7	Funzione <code>renderSkillsTable</code> — render helper con parametri completi	141
15.8.8	Cache di sessione anti-stale-closure	141
15.8.9	Salvataggio job e re-reasoning	143
15.8.10	Modale “Edit Jobs” — checkbox accessibili	143
15.9	<code>src/components/HealthConditionWizard.tsx</code> e <code>HealthConditionWizard.css</code>	144
15.9.1	Ruolo, dimensioni e props	144
15.9.2	La macchina a stati della wizard	144
15.9.3	Inventario completo dello stato interno	145
15.9.4	Inizializzazione dallo stato attuale del worker	146
15.9.5	Normalizzazione del catalogo con <code>useMemo</code>	146
15.9.6	Filtro multi-criterio con <code>useMemo</code>	147
15.9.7	Sistema di diff per il batch di modifiche	148
15.9.8	Toast system: visibilità overflow-safe	149
15.9.9	Toast di scroll: rilevamento overflow dinamico	149
15.9.10	Rilevamento modifiche non salvate (<code>hasUnsavedChanges</code>)	150
15.9.11	Chiusura automatica post-salvataggio	151
15.10	Sistema di design CSS — Token, convenzioni e componenti	151
15.10.1	Paletta cromatica e token impliciti	152
15.10.2	Convenzioni di naming CSS	152
15.10.3	<code>WorkersPage.css</code> — analisi approfondita	153
15.10.4	<code>JobAnalysisView.css</code> — analisi approfondita	157
15.10.5	<code>HealthConditionWizard.css</code> — analisi approfondita	159

16	Configurazione del progetto frontend	164
16.1	<code>src/api/semanticService.ts</code> — Layer di accesso al microservizio	164
16.1.1	Ruolo e posizione architetturale	164
16.1.2	Costante <code>BASE_URL</code> e protocollo	164
16.1.3	Tipi TypeScript — specchio dei modelli Pydantic	165
16.1.4	Helper generico <code>apiFetch<T></code>	165
16.1.5	Funzioni API — analisi per gruppo	166
16.1.6	Uso di <code>encodeURIComponent</code>	168
16.2	<code>package.json</code> — Dipendenze e configurazione build	168
16.2.1	Metadata e entry point	168
16.2.2	Script npm — flusso di sviluppo e build	169
16.2.3	Dipendenze di produzione	170
16.2.4	Dipendenze di sviluppo chiave	171
16.2.5	Configurazione electron-builder	171
16.3	<code>vite.config.ts</code> — Configurazione del bundler	172
16.4	<code>tsconfig.json</code> — Configurazione TypeScript	173
16.5	<code>eslint.config.js</code> — Qualità del codice	174
16.6	<code>start.sh</code> — Script di avvio rapido	175
16.7	Script di utilità — <code>replace.js</code> , <code>fix_box.py</code> , <code>restructure.py</code> .	176
16.7.1	<code>replace.js</code> — patch dell’espansore skill	176
16.7.2	<code>fix_box.py</code> — ristrutturazione layout wizard	177
16.7.3	<code>restructure.py</code> — spostamento header wizard	177
Appendice A	Struttura sintetica di <code>main.py</code>	178
Appendice B	Struttura sintetica di <code>models.py</code>	179
Appendice C	Struttura sintetica di <code>reasoner.py</code>	180
Appendice D	Tabella delle query SPARQL	183
Appendice E	Namespace IRI dell’ontologia	184
Appendice F	Struttura sintetica di <code>importer.py</code>	184
Appendice G	Schema SQL del dataset di import	187
Appendice H	Struttura sintetica dei file Electron	189
Glossario		191
	Classificazione ICF e dominio clinico	191
	Metriche di matching	192

Terminologia ontologica e semantica	193
Terminologia RDB2RDF e pipeline di import	194
Terminologia Electron e frontend	195
Terminologia API e protocolli	196
17 Scenari d'uso e sequenza operativa	198
17.1 Scenario tipico: valutazione idoneità di un nuovo paziente	198
17.2 Sequenza operativa: aggiornamento dei job di valutazione	199
17.3 Diagramma di sequenza UML semplificato	199
18 Known issues e debiti tecnici	201
18.1 Known issues	201
18.2 Debiti tecnici	202
19 Validazione e testing	203
19.1 Validazione dell'ontologia	203
19.2 Test rapido del microservizio	203
20 Note su provenienza dati, licenze e privacy	205
20.1 Provenienza dati O*NET	205
20.2 Dati ICF e OMS	205
20.3 Dati clinici e privacy	205
21 Configurazione IDE e ambiente di sviluppo consigliato	207
21.1 Strumenti consigliati	207
21.2 Estensioni VSCode raccomandate	208
21.3 Configurazione <code>.vscode/settings.json</code> consigliata	208
22 Changelog versioni del documento	209

List of Figures

1	Posizione del microservizio nell'architettura di sistema.	16
2	Sequenza temporale di avvio del microservizio.	22
3	Raggruppamento dei modelli Pydantic per dominio funzionale.	37
4	Relazioni di composizione tra i modelli Pydantic principali.	45
5	Strategia di avvio rapido con snapshot cache.	54
6	Struttura del sottografo HC nell'ontologia Rientr@.	65
7	Flusso dati da endpoint a risposta JSON, con cache bypass.	71
8	Struttura del grafo di matching: il join sul codice ICF collega la condizione di salute del lavoratore alle skill richieste dal job.	79
9	Pipeline completa di <code>import_sql_dataset()</code>	104
10	Architettura a tre processi di Electron nel progetto Rientr@.	107
11	Flusso end-to-end dell'import dataset SQL da "Add Worker".	115
12	Ciclo di vita dell'applicazione Electron con gestione del processo Python.	116
13	Struttura ad albero della directory <code>src/</code>	118
14	Flusso di navigazione gestito da <code>App.tsx</code>	121
15	Diagramma di sequenza semplificato per lo scenario di valutazione (parte 1/2).	200
16	Diagramma di sequenza semplificato per lo scenario di valutazione (parte 2/2).	200

List of Tables

3	Funzioni importate da reasoner	19
4	Classi Pydantic importate da models	20
5	Codici HTTP restituiti dalla guard in base allo stato.	26
7	Mapping eccezioni → codici HTTP.	34
8	Import di models.py	37
9	Mapping suitability → colore hex.	42
11	Costanti IRI principali e loro significato ontologico.	49
12	Metodi di transizione di stato di ReasonerState	51
13	Lock di concorrenza e risorse protette.	70
14	Sotto-ontologie e namespace dell'ontologia Rientr@ v3.1.	72
15	Distribuzione degli individui OWL per namespace.	73
16	Classi OWL principali dell'ontologia.	74
17	ICF Core Set presenti nell'ontologia Rientr@ v3.1.	75
18	Object Properties chiave dell'ontologia.	75
19	Datatype Properties rilevanti per il microservizio.	76
20	Regole SWRL 10–13: soglie O*NET e proprietà inferita.	78
21	Requisiti software del microservizio.	80
22	Dipendenze Python con versioni minime e motivazioni.	81
23	Comportamento degli endpoint durante il warm-up.	83
24	Motivazione di ogni campo pyrightconfig.json	84
25	Confronto Direct Mapping vs R2RML per il caso Rientr@.	86
26	Confronto engine R2RML.	87
27	Le 5 TriplesMap di rientra_mapping.ttl	92
28	VIEW SQL ausiliarie e loro scopo.	95
29	Quando rigenerare il mapping.	102
30	Pattern IRI per un paziente con person_id = "MarioBianchi"	102
31	Struttura delle directory del progetto grafico.	106
32	Motivazione delle opzioni BrowserWindow	110
33	Differenze tra modalità development e production.	111
34	Handler IPC di main.js accessibili tramite electronAPI	112
35	API disponibili in window.electronAPI nel renderer React.	113
36	Struttura della directory src/ con contenuto.	117
37	Mapping ID navigazione → vista iniziale in WorkersPage	121
38	Tutte le variabili di stato di WorkersPage con tipo e scopo.	132
39	Variabili di stato di HealthConditionWizard	145
40	Token cromatici impliciti condivisi tra tutti i CSS.	152
41	Prefissi CSS e namespace corrispondenti.	153

42	Corrispondenza tipi TypeScript $\leftrightarrow$ modelli Pydantic.	165
43	Script npm con motivazione di ogni scelta.	169
44	Dipendenze di produzione e motivazioni.	170
45	DevDependencies chiave con motivazioni.	171
46	Plugin ESLint configurati e motivazioni.	175
49	Problemi noti e workaround.	201
50	Debiti tecnici identificati con priorità e impatto.	202
51	Strumenti di validazione dell'ontologia e risultati.	203
52	Dati O*NET nell'ontologia Rientr@.	205
53	Strumenti consigliati per lo sviluppo del progetto Rientr@.	207
54	Estensioni VSCode per il progetto Rientr@.	208

Listings

1	Lifespan handler — startup sequence (<code>main.py</code> , righe 76–104).	21
2	Creazione dell'istanza FastAPI.	23
3	Configurazione del CORS middleware.	23
4	Guard per lo stato del reasoner.	25
5	Selezione del lavoratore attivo.	28
6	Request body per <code>POST /match/detail</code>	30
7	Request body per l'update delle condizioni ICF.	31
8	Struttura della risposta di <code>POST /import/workers</code>	32
9	Pattern di gestione degli errori degli endpoint.	34
10	Avvio diretto con Uvicorn per lo sviluppo.	35
11	Comando di avvio in produzione.	35
12	<code>OntologyStats</code> — statistiche dell'ontologia.	38
13	<code>StatusResponse</code> — risposta di <code>GET /status</code>	38
14	<code>WorkerSummary</code> e <code>WorkerDetail</code>	38
15	Selezione del lavoratore attivo.	39
16	Aggiornamento dei job assegnati a un lavoratore.	39
17	<code>JobSummary</code> — elemento di lista job.	40
18	<code>JobSkillProfileEntry</code> — skill con punteggio O*NET.	40
19	Importanza skill per job (Q4).	41
20	<code>MatchResult</code> — coppia (worker, job) con punteggi.	41
21	Request body per <code>POST /match/detail</code>	42
22	<code>SkillDetail</code> — dettaglio per singola skill (Q3).	42
23	Condizione di salute ICF di un lavoratore.	43
24	<code>IcfCodeEntry</code> — catalogo ICF per la procedura guidata.	44
25	Modelli per l'aggiornamento batch delle condizioni ICF.	44
26	Silencing dei log rumorosi di owlready2 e Java.	47
27	Costanti per la classificazione di idoneità.	49
28	Classe <code>ReasonerState</code> — singleton thread-safe.	50
29	Variabili globali di stato e cache.	51
30	Estrazione del nome locale da un IRI.	52
31	Mappatura CS → etichetta (Fig. 2 del paper).	52
32	Classificazione di idoneità per coppia (GCS%, AISA%).	52
33	Conversione punteggio O*NET in anchor [0-3	53
34	Fingerprint del file ontologico per invalidazione della cache.	53
35	Chiavi obbligatorie nella snapshot cache.	55
36	Scrittura atomica della cache su disco.	55
37	Condizione di utilizzo della cache.	56

38	Caricamento del file RDF in owlready2.	56
39	Avvio del reasoner Pellet con monkey-patch anti-ciclo.	57
40	Parsing del tempo di esecuzione di Pellet dal log.	58
41	Sequenza completa di <code>load_and_reason()</code>	58
42	Pattern comune delle funzioni di query.	59
43	SPARQL per la lista dei workers.	60
44	SPARQL per la lista dei job.	60
45	SPARQL per le condizioni ICF di un worker.	60
46	Parsing dei compound IRI ICF.	61
47	Pattern delle query Q4 per importanza.	61
48	SPARQL principale per GCS% e AISA% (Q1+Q2).	62
49	Calcolo di GCS% e AISA% dall'aggregazione.	63
50	SPARQL Q3 per il dettaglio skill.	64
51	Mutazione atomica di <code>isSelected</code>	64
52	Assegnazione sicura di proprietà funzionali in owlready2.	65
53	Routing del qualificatore in base al prefisso ICF.	67
54	Sostituzione atomica dei link <code>isEvaluatedForJob</code>	67
55	Sequenza di re-reasoning dopo una mutazione.	68
56	Flusso di <code>inject_imported_workers()</code>	68
57	SPARQL per la mappa ICF code $\rightarrow$ Core Set.	69
58	Setup completo dell'ambiente di sviluppo.	82
59	Esempio di chiamata da Spring Boot con RestTemplate.	83
60	<code>pyrightconfig.json</code> — configurazione del type checker.	84
61	Importazione condizionale con fallback graceful.	88
62	Namespace <code>rdflib</code> e costanti di modulo.	88
63	<code>ImportResult</code> — struttura dati di ritorno.	89
64	<code>ensure_mapping()</code> — generazione idempotente del mapping.	90
65	Classificazione automatica delle datatype properties.	90
66	Alias colonna ZIPcode $\rightarrow$ <code>zip_code</code>	91
67	Estratto da <code>rientra_mapping.ttl</code> — MAP 1 e MAP 3B.	92
68	Creazione delle VIEW ausiliarie nel DB SQLite in-memory.	94
69	Caricamento del dataset SQL in SQLite in-memory.	95
70	Schema SQL del dataset di import.	96
71	Debug della costante booleana in Morph-KGC.	97
72	Scrittura manuale di <code>isSelected</code> compatibile con Pellet.	98
73	Iniezione testuale dei blocchi RDF/XML prima del tag di chiusura.	98
74	Escaping XML dei valori anagrafici.	99
75	Estrazione codici ICF validi con regex sul testo RDF.	100
76	Estrazione job validi con tre strategie di fallback.	100

77	Controllo esistenza persona con ricerca stringa.	101
78	Sanificazione dell'ID per IRI validi.	101
79	Sequenza completa dell'entry point pubblico.	103
80	Risoluzione cross-platform dell'eseguibile Python.	108
81	Avvio di uvicorn come sottoprocesso Node.js.	108
82	Terminazione del sottoprocesso Python.	109
83	Configurazione della BrowserWindow principale.	110
84	Caricamento condizionale dev/production.	111
85	Rimozione del menu nativo.	111
86	preload.js — esposizione sicura delle API native.	112
87	main.tsx — montaggio dell'applicazione React.	118
88	App.tsx — routing client-side e gestione navigazione.	119
89	Tipi TypeScript esportati da App.tsx	120
90	index.css — CSS globale dell'applicazione.	122
91	start.sh — contenuto completo.	175
92	replace.js — obiettivo della patch.	176
93	Pseudo-outline di main.py	178
94	Pseudo-outline di models.py	179
95	Pseudo-outline di reasoner.py	180
96	Pseudo-outline di importer.py	184
97	Schema completo del dataset SQL di import (v0.1-test).	187
98	Pseudo-outline di electron/main.js	189
99	Pseudo-outline di electron/preload.js	190
100	Test rapido degli endpoint principali.	203
101	Impostazioni VSCode raccomandate per Rientr@.	208

1 Introduzione e contesto di progetto

1.1 Il sistema Rientr@

Rientr@ è un sistema di supporto decisionale per la valutazione dell'*idoneità lavorativa* di persone con condizioni di salute documentate secondo la classificazione ICF (*International Classification of Functioning, Disability and Health*).

Il sistema opera su un'ontologia OWL 2 DL arricchita con regole SWRL. L'inferenza viene eseguita dal reasoner **Pellet**, che calcola relazioni implicite tra individui di classe **Person** (i lavoratori) e individui di classe **Job** (i profili lavorativi), attraverso punteggi derivati da dati O*NET e qualificatori ICF.

Il file `main.py` è il **punto di ingresso** dell'intero sottosistema semantico: istanzia l'applicazione FastAPI, configura il middleware, avvia il reasoner e dichiara tutti gli endpoint REST che espongono i risultati del ragionamento ontologico.

1.2 Posizione nell'architettura generale

Il microservizio semantico si colloca all'interno di un'architettura a più livelli:

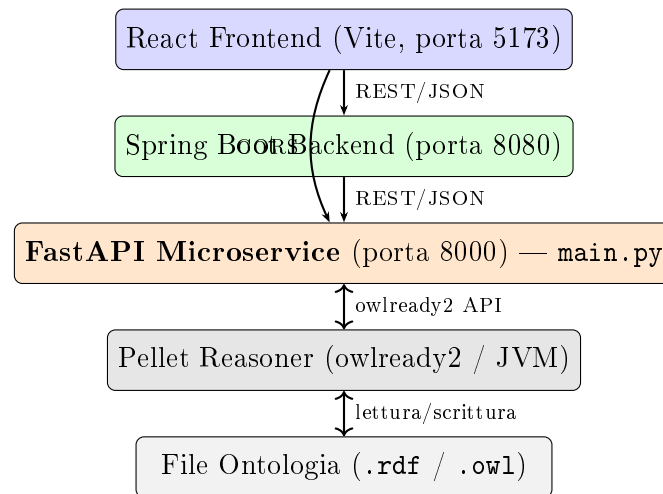


Figure 1: Posizione del microservizio nell'architettura di sistema.

Il frontend React comunica *direttamente* con il microservizio per alcune operazioni (scatter plot, aggiornamenti in tempo reale), mentre lo Spring Boot Backend funge da orchestratore per i flussi applicativi complessi.

2 Dipendenze e import

2.1 Librerie standard Python

Modulo	Motivazione d'uso
<code>os</code>	Lettura della variabile d'ambiente <code>ONTOLOGY_PATH</code> ; enumerazione dei file nella directory di servizio.
<code>threading</code>	Avvio del reasoner Pellet in un <i>daemon thread</i> separato, in modo da non bloccare il ciclo di eventi ASGI di FastAPI durante il caricamento dell'ontologia.
<code>tempfile</code>	Creazione di file temporanei <code>.sql</code> per il pipeline di import RDB2RDF, evitando di mantenere dati in memoria per l'intera durata dell'operazione.
<code>contextlib.</code> <code>asynccontextmanager</code>	Necessario per implementare il <i>lifespan handler</i> asincrono richiesto dall'API di FastAPI 0.95+.
<code>pathlib.Path</code>	Gestione robusta e cross-platform dei percorsi di file, in alternativa alla concatenazione con <code>os.path.join</code> .
<code>typing.Optional</code>	Annotazione di tipi per i parametri di query opzionali negli endpoint.

2.2 Dipendenze esterne

Libreria	Motivazione d'uso
<code>fastapi</code>	Framework ASGI ad alte prestazioni per la definizione degli endpoint REST, la validazione automatica degli input (via Pydantic) e la generazione della documentazione OpenAPI. Scelto rispetto a Flask o Django per il supporto nativo all'async, la tipizzazione forte e le prestazioni.

Libreria	Motivazione d'uso
<code>fastapi.</code> <code>CORSMiddleware</code>	Middleware per la gestione delle <i>Cross-Origin Resource Sharing</i> policy. Necessario perché frontend React (porta 5173) e backend Spring Boot (porta 8080) risiedono su origini diverse rispetto al microservizio (porta 8000).
<code>fastapi.</code> <code>responses.</code> <code>JSONResponse</code>	Risposta JSON personalizzata usata nell'endpoint <code>/import/workers</code> , dove il body di ritorno non segue un <code>response_model</code> Pydantic fisso.
<code>uvicorn</code>	Server ASGI di produzione (e sviluppo) che esegue l'applicazione FastAPI. Configurato per ascoltare su tutte le interfacce (0.0.0.0) alla porta 8000.

2.3 Moduli interni

`main.py` importa da due moduli interni al progetto:

`reasoner`

Contiene l'intera logica di accesso all'ontologia: caricamento, ragionamento con Pellet, esecuzione delle query SPARQL e mutazione in-memory degli individui. Le funzioni importate sono elencate nella Tabella 3.

`models`

Definisce tutti i modelli Pydantic (`BaseModel`) usati come `response_model` degli endpoint. Garantisce validazione automatica dell'output e la generazione dello schema OpenAPI. Le classi importate sono elencate nella Tabella 4.

Table 3: Funzioni importate da **reasoner**.

Funzione	Responsabilità
<code>state</code>	Oggetto singleton che espone lo stato corrente del reasoner (<code>loading</code> , <code>ready</code> , <code>error</code>).
<code>load_and_reason</code>	Carica l'ontologia e avvia Pellet; eseguita nel thread in background.
<code>load_snapshot_cache</code>	Pre-carica la cache degli snapshot per ridurre la latenza delle prime query.
<code>get_workers</code>	SPARQL query Q0: restituisce tutti i Person con almeno un <code>isEvaluatedForJob</code> .
<code>get_jobs</code>	Restituisce tutti i Job con almeno una relazione <code>requires</code> .
<code>get_health_conditions</code>	Recupera i codici ICF e i qualificatori di un lavoratore.
<code>get_importance_summary</code>	Q4: importanza skill/abilità per job.
<code>get_match_results</code>	Q1+Q2: calcola GCS% e AISA%.
<code>get_skill_detail</code>	Q3: dettaglio per coppia (worker, job).
<code>get_job_skill_profile</code>	Profilo skill di un job (senza worker).
<code>set_selected_worker</code>	Muta <code>isSelected</code> in-memory.
<code>get_all_icf_codes</code>	Tutti i codici ICF presenti nell'ontologia.
<code>get_all_core_sets</code>	Nomi distinti degli ICF core set.
<code>update_health_conditions</code>	Applica batch di modifiche ICF.
<code>update_worker_jobs</code>	Sostituisce i link <code>isEvaluatedForJob</code> .
<code>inject_imported_workers</code>	Integra in-memory i nuovi individui importati via SQL.

Table 4: Classi Pydantic importate da `models`.

Classe	Utilizzo
StatusResponse	Risposta GET <code>/status</code> .
OntologyStats	Statistiche dell'ontologia incluse nello status.
WorkerSummary	Elemento di lista in GET <code>/workers</code> .
WorkerDetail	Risposta GET <code>/workers/{id}</code> .
JobSummary	Elemento di lista in GET <code>/jobs</code> .
JobSkillProfileEntry	Risposta GET <code>/jobs/{id}/profile</code> .
HealthConditionsResponse	Risposta GET <code>/health-conditions</code> .
ImportanceEntry	Elemento di lista in GET <code>/jobs/{id}/importance</code> .
MatchResult	Risposta GET <code>/match</code> e varianti.
MatchDetailRequest	Request body POST <code>/match/detail</code> .
SkillDetailResponse	Risposta POST <code>/match/detail</code> .
SelectWorkerRequest	Request body POST <code>/workers/select</code> .
SelectWorkerResponse	Risposta POST <code>/workers/select</code> .
IcfCodeEntry	Elemento di lista in GET <code>/icf-codes</code> .
HcChangeItem	Singola modifica ICF nel batch update.
UpdateHCRequest	Request body POST <code>/health-conditions/.../update</code> .
UpdateHCResponse	Risposta POST <code>/health-conditions/.../update</code> .
UpdateWorkerJobsRequest	Request body POST <code>/workers/.../jobs/update</code> .
UpdateWorkerJobsResponse	Risposta POST <code>/workers/.../jobs/update</code> .

3 Ciclo di vita dell'applicazione (Lifespan)

3.1 Motivazione del pattern lifespan

FastAPI, a partire dalla versione 0.95, ha deprecato gli event handler `on_event("startup")` in favore del pattern basato su `@asynccontextmanager` e il parametro `lifespan=` del costruttore `FastAPI()`. Questo approccio è preferibile per due ragioni:

1. È compatibile con la gestione di risorse asincrone (connection pool, client HTTP, ecc.) tramite la sintassi `async with`.
2. Garantisce che il cleanup venga sempre eseguito, anche in caso di eccezioni durante lo startup.

Scelta progettuale — Thread vs. Task asincrono

Il reasoner Pellet viene avviato in un `threading.Thread`, non in una `asyncio.Task`. La motivazione è che owlready2 e la JVM sottostante eseguono operazioni bloccanti di I/O e CPU che non sono compatibili con il loop asincrono di `asyncio`. Usare un thread nativo permette al loop ASGI di restare reattivo durante il caricamento dell'ontologia (che può durare 30–120 secondi), consentendo al server di rispondere già al polling di `GET /status`.

3.2 Sequenza di inizializzazione

```

1 @asynccontextmanager
2 async def lifespan(app: FastAPI):
3     rdf_path = os.environ.get("ONTOLOGY_PATH", "")
4
5     # 1) Verifica/genera il file di mapping R2RML
6     try:
7         from import_workers.importer import ensure_mapping
8         ...
9         ensure_mapping(ontology_file, mapping_file)
10    except Exception as _map_err:
11        logging.warning("[startup] Could not ensure mapping: %s",
12                        ↪ _map_err)
13
14    # 2) Pre-carica la snapshot cache (prima che Pellet finisca)
15    load_snapshot_cache(rdf_path)
16
17    # 3) Avvia Pellet in un daemon thread
18    thread = threading.Thread(

```

```

18     target=load_and_reason ,
19     args=(rdf_path,) ,
20     daemon=True ,
21     name="pellet-reasoner" ,
22 )
23 thread.start()
24 yield    # <-- l'applicazione gira qui
25 # cleanup: nulla da fare; owlready2 gestisce la JVM

```

Listing 1: Lifespan handler — startup sequence (`main.py`, righe 76–104).

I tre passi dello startup hanno il seguente significato:

1. **Generazione del mapping R2RML** — Prima che qualsiasi import SQL possa avvenire, deve esistere un file di mapping (`rientra_mapping.ttl`) che descrive come trasformare le tabelle relazionali in triple RDF. La funzione `ensure_mapping` lo genera a partire dall'ontologia se non è già presente. Eventuali errori in questa fase non bloccano l'avvio (solo un `warning`), perché la funzionalità di import è opzionale rispetto al ragionamento.
2. **Snapshot cache** — La funzione `load_snapshot_cache` carica una versione pre-computata dei risultati SPARQL (se disponibile), in modo che le prime richieste degli endpoint non debbano aspettare il completamento di Pellet.
3. **Avvio di Pellet** — Il thread `pellet-reasoner` è marcato come *daemon*: viene terminato automaticamente quando il processo principale termina, evitando zombie JVM.

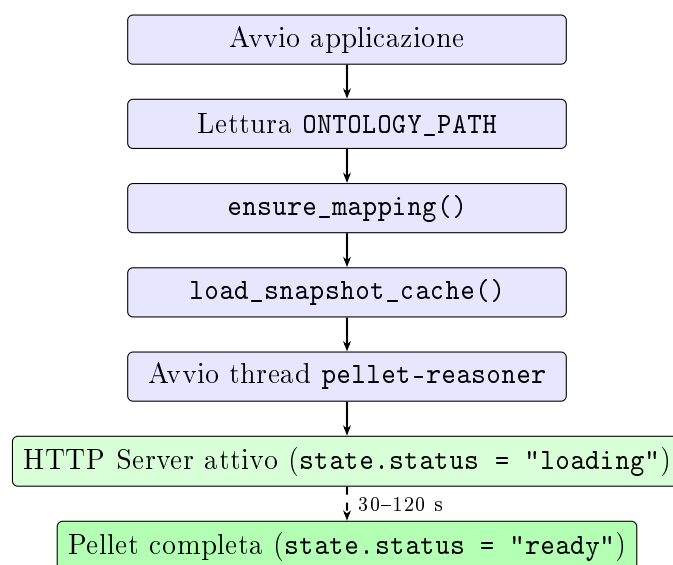


Figure 2: Sequenza temporale di avvio del microservizio.

4 Configurazione dell'applicazione FastAPI

4.1 Istanza dell'applicazione

```
1 app = FastAPI(  
2     title="Rientr@ Semantic Microservice",  
3     description=("REST API exposing the Rientr@ ontology reasoning  
4         ↪ results. "  
5         "Loads an RDF ontology, runs Pellet (DL reasoner)  
6         ↪ with SWRL rules, "  
7         "and serves SPARQL query outputs as structured  
8         ↪ JSON."),  
9     version="1.0.0",  
10    lifespan=lifespan,  
11 )
```

Listing 2: Creazione dell'istanza FastAPI.

I metadati `title`, `description` e `version` popolano automaticamente la documentazione OpenAPI disponibile a `http://localhost:8000/docs` (Swagger UI) e `http://localhost:8000/redoc` (ReDoc). Dichiararli esplicitamente invece di usare i default (`"FastAPI"`, `"", "0.1.0"`) serve a tre scopi concreti: (1) i client generati automaticamente dalla specifica OpenAPI includono il titolo nel nome del package; (2) Spring Boot e il frontend possono rilevare la versione tramite `/openapi.json` senza parsing di header HTTP; (3) la descrizione appare nel campo `info.description` della specifica, che strumenti come Postman e Insomnia mostrano agli sviluppatori che integrano il servizio.

4.2 Middleware CORS

```
1 app.add_middleware(  
2     CORSMiddleware,  
3     allow_origins=[  
4         "http://localhost:5173",    # Vite / React dev server  
5         "http://localhost:3000",    # alternative React port  
6         "http://localhost:8080",    # Spring Boot backend  
7         "http://127.0.0.1:5173",  
8         "http://127.0.0.1:8080",  
9     ],  
10    allow_credentials=True,  
11    allow_methods=["*"],  
12    allow_headers=["*"],  
13 )
```

Listing 3: Configurazione del CORS middleware.

Nota di sicurezza — CORS in produzione

La configurazione attuale consente credenziali (`allow_credentials=True`) e tutti i metodi/header per le origini elencate. Per un deploy in produzione è necessario:

1. Sostituire le origini hardcoded con variabili d'ambiente.
2. Limitare `allow_methods` e `allow_headers` ai soli valori necessari.
3. Valutare l'uso di HTTPS.

La scelta di listare esplicitamente le origini anziché usare `allow_origins=["*"]` è intenzionale: con `allow_credentials=True` i browser rifiutano il wildcard, quindi le origini devono essere elencate esplicitamente.

5 Guard di disponibilità del reasoner

5.1 La funzione `_require_ready()`

```
1 def _require_ready() -> None:
2     if state.status == "loading":
3         raise HTTPException(
4             status_code=503,
5             detail={
6                 "status" : "loading",
7                 "message": "Ontology is still being loaded...",
8             },
9         )
10    if state.status == "error":
11        raise HTTPException(
12            status_code=500,
13            detail={"status": "error", "message": state.message},
14        )
```

Listing 4: Guard per lo stato del reasoner.

Ogni endpoint (tranne GET `/status`) chiama `_require_ready()` come prima operazione. Questa scelta garantisce:

- **Fail-fast:** il client riceve immediatamente un errore significativo invece di attendere una timeout o ricevere dati parziali.
- **Coerenza semantica:** un 503 `Service Unavailable` segnala al client che la risorsa *esiste* ma non è ancora disponibile (a differenza di un 404), permettendo una strategia di retry con backoff esponenziale.
- **Separazione stato/logica:** tutta la gestione dello stato è centralizzata in una sola funzione, rendendo gli endpoint focalizzati sulla logica di business.

Il mapping eccezione $\rightarrow$ codice HTTP segue criteri semantici precisi: `KeyError` produce 404 (la risorsa non esiste nell'ontologia, non è un errore del client), `RuntimeError` produce 503 (la risorsa esiste ma non è temporaneamente disponibile, ad esempio per un re-reasoning in corso). Un 422 `Unprocessable Entity` sarebbe semanticamente scorretto per questi casi: 422 indica che la sintassi della richiesta è valida ma il contenuto non può essere elaborato, mentre qui il problema è sempre nella disponibilità della risorsa, non nel formato della richiesta.

Table 5: Codici HTTP restituiti dalla guard in base allo stato.

stato	HTTP	Significato
loading	503	Pellet non ha ancora completato; riprova più tardi.
error	500	Errore irrecuperabile durante il caricamento.
ready	(nessun errore)	Elaborazione continua normalmente.

6 Endpoint REST

6.1 Panoramica

Il microservizio espone **17 endpoint** raggruppati in 5 tag tematici. La tabella seguente fornisce una visione sinottica.

I cinque tag (*Service*, *Workers*, *Jobs*, *Matching*, *Health Conditions*, *Import*) rispecchiano la struttura delle risorse ontologiche, non la struttura interna del codice. Questa scelta garantisce che la documentazione OpenAPI sia leggibile da un clinico o un ontology engineer, non solo da un programmatore. Il tag *Import* è separato dagli altri perché l'operazione di import è distruttiva (modifica il file RDF) e lenta (avvia la pipeline RDB2RDF): isolarla in un tag distinto segnala visivamente la sua natura eccezionale rispetto alle operazioni di lettura.

Tag	Path	Metodo	Descrizione
Svc	/status	GET	Stato e metriche del servizio.
	/workers	GET	Lista tutti i lavoratori.
Wk.	/workers/{id}	GET	Dettaglio singolo lavoratore.
	/workers/select	POST	Seleziona il lavoratore attivo.
	/workers/{id}/	POST	Aggiorna i job assegnati.
	jobs/update		
Jobs	/jobs	GET	Lista tutti i job.
	/jobs/{id}/	GET	Importanza skill (Q4).
	importance		
	/jobs/{id}/profile	GET	Profilo skill del job.
M.	/match	GET	GCS%/AISA% per tutti i selezionati.
	/match?worker_id=	GET	Filtro su worker specifico.
	/match/{id}	GET	Matching per worker (path param).
	/match/detail	POST	Dettaglio skill per coppia (Q3).
HC	/health-cond./{id}	GET	Codici ICF del lavoratore.
	/health-conditions/{id}/update	POST	Aggiorna le condizioni ICF.
	/icf-codes	GET	Tutti i codici ICF disponibili.
	/core-sets	GET	Nomi dei core set ICF.
Imp.	/import/workers	POST	Import da file SQL (RDB2RDF).

6.2 Tag: Service

6.2.1 GET /status

Scopo: Unico endpoint che non richiede lo stato **ready**. Restituisce le metriche correnti del reasoner ed è progettato per il polling periodico da parte del frontend durante il warm-up.

Response model: `StatusResponse`

Campi notevoli della risposta:

- `status`: "loading" | "ready" | "error"
- `elapsed_pellet`: tempo in secondi impiegato da Pellet.
- `stats`: oggetto `OntologyStats` (numero classi, individui, proprietà, ecc.).

6.3 Tag: Workers

6.3.1 GET /workers

Restituisce tutti gli individui di classe `Person` che hanno almeno una tripla `isEvaluatedForJob`. Questo filtro esclude le persone che esistono nell'ontologia ma non sono state ancora associate a nessun profilo lavorativo, garantendo che la lista sia sempre actionable dal punto di vista applicativo.

6.3.2 GET /workers/{worker_id}

Navigazione per ID. Se il lavoratore non esiste nell'ontologia viene restituito un **404 Not Found** con un messaggio esplicito. Il pattern usato (`next(..., None)` seguito da controllo esplicito) è preferito a una ricerca con eccezione perché rende il codice più leggibile e l'intento più chiaro.

6.3.3 POST /workers/select

```
1 # Request body
2 { "worker_id": "Patient1" }
```

Listing 5: Selezione del lavoratore attivo.

Muta in-memory il flag `isSelected` dell'ontologia `owlready2`: deselecta il lavoratore precedentemente attivo e seleziona il nuovo. Deve essere chiamato *prima* di `GET /match/{worker_id}`, perché la query SPARQL di matching filtra su `FILTER(?selected = true)`.

Scelta progettuale — mutazione in-memory

La modifica di `isSelected` non viene persistita su file RDF. Questo è intenzionale: la selezione è uno stato di sessione effimero, non una proprietà dati permanente. Persistere ogni selezione comporterebbe riscrittura frequente del file RDF e potenziali race condition nel caso di accessi concorrenti.

6.3.4 POST /workers/{worker_id}/jobs/update

Sostituisce atomicamente tutti i link `isEvaluatedForJob` del lavoratore. Dopo la mutazione, l'ontologia viene salvata su disco, Pellet viene rieseguito e la snapshot cache viene aggiornata, in modo che le chiamate successive a `/match` riflettano immediatamente i nuovi assegnamenti. Questo garantisce la coerenza dei dati tra operazioni successive senza richiedere un riavvio del servizio.

6.4 Tag: Jobs

Gli endpoint del tag Jobs usano esclusivamente il metodo **GET** perché sono operazioni di *sola lettura* sul grafo ontologico: non modificano l'ontologia, sono idempotenti e cacheabili. Questa conformità alla semantica HTTP permette a proxy, CDN e browser di mettere in cache le risposte (anche se nel contesto desktop corrente non ci sono proxy intermedi, la conformità facilita future integrazioni web).

6.4.1 GET /jobs/{job_id}/importance

Implementa la **query semantica Q4**: restituisce la ripartizione dell'importanza delle skill/abilità per un dato job, organizzata per livello di importanza (`isVeryImportantFor`, `isImportantFor`, ecc.), inferita da Pellet attraverso le regole SWRL 9–12.

6.4.2 GET /jobs/{job_id}/profile

Restituisce il profilo completo delle skill richieste da un job con i relativi punteggi O*NET grezzi (0–100). Non coinvolge alcun lavoratore: è pensato per costruire grafici radar o di "inclinazione" indipendentemente da qualsiasi condizione di salute.

6.5 Tag: Matching

Il matching è il cuore funzionale del sistema. Il microservizio espone tre livelli di dettaglio invece di un solo endpoint generico, per due ragioni. Prima ragione: **granularità delle query SPARQL**. Le tre query (Q0, Q1+Q2, Q3) hanno costo

computazionale molto diverso: Q3 (`hasSpecificCriticality` per ogni skill di una coppia) è dieci volte più lenta di Q1+Q2 (aggregato per worker). Esporre granularità separate permette al frontend di caricare prima la visione d'insieme (scatter plot) e poi il dettaglio on-demand. Seconda ragione: **separazione degli use case**. `GET /match` alimenta la KPI strip e lo scatter plot; `GET /match/{id}` permette il deep link a un worker specifico da Spring Boot; `POST /match/detail` è chiamato solo quando l'utente apre la tabella skill.

1. `GET /match` — risultati aggregati per tutti i lavoratori selezionati (o filtrati per `worker_id`).
2. `GET /match/{worker_id}` — risultati per un singolo lavoratore (path parameter, restituisce 404 se assente).
3. `POST /match/detail` — breakdown per-skill per una specifica coppia (worker, job). Usa POST anziché GET perché il body della richiesta contiene due ID che semanticamente formano un “oggetto di ricerca”, non filtri di una collezione.

6.5.1 Query Q1+Q2 — GCS% e AISA%

I due indicatori principali del sistema Rientr@ sono:

- **GCS% (General Criticality Score)**: misura la criticità complessiva del profilo di salute del lavoratore rispetto al job.
- **AISA% (Amount of Impaired Skill/Abilities)**: percentuale di skill/abilità richieste dal job che risultano compromesse dalla condizione di salute del lavoratore.

Il campo `suitability` viene derivato dalle soglie definite in Fig. 4 del paper Rientr@:

$$\text{SUITABLE} : \text{GCS} < -0.5 \cdot \text{AISA} + 15.5 \quad (1)$$

$$\text{NOT SUITABLE} : \text{GCS} > -0.5 \cdot \text{AISA} + 21 \quad (2)$$

$$\text{SUITABLE WITH PRECAUTIONS} : \text{altrimenti} \quad (3)$$

6.5.2 Query Q3 — Dettaglio per coppia (worker, job)

```
1 { "worker_id": "Patient1", "job_id": "Job_AssemblyWorker" }
```

Listing 6: Request body per `POST /match/detail`.

Restituisce per ogni skill/abilità:

- Punteggio O*NET di importanza (0–100).
- Anchor di importanza [0–3].
- Qualificatore ICF del lavoratore.
- Criticality Score: $CS = \text{qualifier} \times \text{anchor}$.
- CS normalizzato: $\widehat{CS} = CS/12$.
- Label di criticità (da *not critical* a *extremely critical*).

6.6 Tag: Health Conditions

6.6.1 GET /health-conditions/{worker_id}

Alimenta la tabella "Current Health Conditions" nel frontend React. Restituisce i codici ICF con i rispettivi qualificatori (0–4) associati al profilo di salute del lavoratore.

6.6.2 POST /health-conditions/{worker_id}/update

Applica un batch di modifiche (add, remove, modify) al profilo ICF del lavoratore *in-memory*. Nessun file RDF viene riscritto in questa operazione, che è quindi reversibile tra sessioni.

```

1 {
2   "worker_id": "Patient1",
3   "changes": [
4     { "icf_code": "b1408", "action": "add", "qualifier": 2 },
5     { "icf_code": "d410", "action": "modify", "qualifier": 3 },
6     { "icf_code": "b280", "action": "remove", "qualifier": null }
7   ]
8 }
```

Listing 7: Request body per l'update delle condizioni ICF.

6.6.3 GET /icf-codes e GET /core-sets

/icf-codes è un endpoint separato da /health-cond./{id} per una ragione di frequenza di accesso: le condizioni di salute di un worker vengono ricaricate ad ogni selezione del worker (potenzialmente decine di volte per sessione); il catalogo ICF completo viene caricato solo quando l'utente apre la wizard di modifica (una volta per sessione al massimo). Accoppiarli in un singolo endpoint significherebbe trasferire centinaia di entry ICF ad ogni cambio worker, aumentando la latenza percepita. La separazione rispetta il principio di *caricamento lazy*: il catalogo pesante viene caricato solo quando serve.

Questi due endpoint forniscono i dati necessari al frontend per popolare le interfacce di selezione nella procedura guidata "Modify Health Condition":

- `/icf-codes`: restituisce tutti i codici ICF presenti nell'ontologia via triple `involvesICFCode`.
- `/core-sets`: lista ordinata dei core set ICF distinti (es. *Stroke*, *Hearing Loss*, *Low Back Pain*).

6.7 Tag: Import

6.7.1 POST `/import/workers`

Pipeline RDB2RDF via R2RML

Questo è l'endpoint più complesso del microservizio. Implementa una pipeline completa di importazione dati strutturati in formato SQL nell'ontologia RDF.

Flusso operativo:

1. Il client invia il file `.sql` come body raw (`Content-Type: application/octet-stream`). Il nome del file può essere trasmesso nell'header `X-Filename`.
2. Il body viene letto con `await request.body()`, senza richiedere `python-multipart` (scelta che riduce le dipendenze).
3. Il file viene scritto in un file temporaneo con `tempfile.NamedTemporaryFile`.
4. Viene verificata la presenza del file di mapping R2RML (`rientra_mapping.ttl`); se assente, viene generato da `ensure_mapping()`.
5. La funzione `import_sql_dataset()` esegue la trasformazione RDB2RDF e aggiorna il file ontologico.
6. Il file temporaneo viene eliminato nel blocco `finally`, garantendo la pulizia anche in caso di errore.
7. Se l'import ha aggiunto nuovi individui, viene chiamata `inject_imported_workers()` per aggiornare l'ontologia in-memory senza riavviare Pellet.

```

1 {
2   "persons_added" : 3,
3   "persons_skipped": 0,
4   "icf_valid"      : 42,

```

```
5  "icf_skipped"      : 2,  
6  "jobs_valid"       : 7,  
7  "jobs_skipped"     : 1,  
8  "new_person_ids"   : ["LucaMartini", "ElenaConti", "GiuseppeFerrari  
↔  "],  
9  "skipped_ids"      : [],  
10 "backup_path"      : "/abs/path/Rientra.rdf.bak",  
11 "error"            : null  
12 }
```

Listing 8: Struttura della risposta di POST `/import/workers`.

Il campo `backup_path` indica dove è stato salvato il backup dell'ontologia prima della modifica, permettendo un eventuale rollback manuale.

7 Gestione degli errori

7.1 Strategia generale

Tutti gli endpoint seguono il pattern:

```

1 def endpoint(...):
2     _require_ready()
3     try:
4         data = get_something(...)
5         return ResponseModel(**data)
6     except HTTPException:
7         raise # propaga errori gia' tipizzati
8     except KeyError as exc:
9         raise HTTPException(status_code=404, detail=str(exc)) from
        ↪ exc
10    except RuntimeError as exc:
11        raise HTTPException(status_code=503, detail=str(exc)) from
        ↪ exc
12    except Exception as exc:
13        raise HTTPException(status_code=500, detail=str(exc)) from
        ↪ exc

```

Listing 9: Pattern di gestione degli errori degli endpoint.

7.2 Gerarchia dei codici di errore

Table 7: Mapping eccezioni → codici HTTP.

Eccezione Python	HTTP	Significato applicativo
state == "loading"	503	Reasoner ancora in esecuzione.
state == "error"	500	Errore irrecuperabile del reasoner.
KeyError	404	Risorsa (worker/job) non trovata.
RuntimeError	503	Risorsa temporaneamente non disponibile.
Exception (generico)	500	Errore interno non previsto.

8 Entrypoint di sviluppo

```
1 if __name__ == "__main__":
2     import uvicorn
3     uvicorn.run(
4         "main:app",
5         host="0.0.0.0",
6         port=8000,
7         reload=False,    # True solo in sviluppo
8         log_level="info",
9     )
```

Listing 10: Avvio diretto con Uvicorn per lo sviluppo.

Il flag `reload=False` è impostato di default anche nell'entrypoint di sviluppo, perché il reload automatico di Uvicorn riavvierebbe il thread Pellet ad ogni modifica del codice, con tempi inaccettabili (30–120 s). In fase di sviluppo attivo, si raccomanda di:

1. Modificare solo i file che non toccano il reasoner (`main.py`, `models.py`).
2. Usare `-reload` solo quando si lavora su parti che non richiedono il warm-up di Pellet.

Il comando raccomandato per il deploy è invece:

```
1 uvicorn main:app --host 0.0.0.0 --port 8000 --reload
```

Listing 11: Comando di avvio in produzione.

9 Modulo `models.py` — Schemi Pydantic v2

9.1 Ruolo e motivazione del modulo

`models.py` centralizza la definizione di **tutti i modelli di dati** del microservizio, sia per le risposte (*response models*) che per i corpi delle richieste (*request bodies*). La scelta di isolare questi schemi in un modulo dedicato, anziché dichiararli inline negli endpoint, risponde a tre obiettivi:

1. **Riusabilità:** lo stesso modello può essere importato da `main.py` e, in futuro, da test automatici o da client generati.
2. **Documentazione automatica:** FastAPI legge le annotazioni Pydantic e genera automaticamente lo schema JSON (OpenAPI 3.1) esposto a `/docs` e `/openapi.json`, comprese le descrizioni dei campi via `Field(description=...)`.
3. **Validazione in ingresso:** i *request bodies* definiti come `BaseModel` vengono validati da Pydantic prima che la funzione endpoint venga eseguita; un payload non valido produce automaticamente un `422 Unprocessable Entity` con un messaggio di errore strutturato.

Pydantic v2 vs. v1

Il modulo usa esplicitamente **Pydantic v2** (importazione da `pydantic` con `BaseModel` e `Field`). Le differenze principali rispetto alla v1 che impattano questo codice sono:

- `model_dump()` sostituisce `dict()` — usato in `main.py` per serializzare i `HcChangeItem` prima di passarli al reasoner.
- La validazione di default è più rigida sui tipi (es. `int` non accetta stringhe numeriche senza coercizione).
- `default_factory=list` sostituisce il vecchio pattern `default=[]`, che in v1 causava il classico bug di mutabilità degli argomenti di default.

9.2 Dipendenze del modulo

Table 8: Import di `models.py`.

Import	Motivazione
<code>from __future__ import annotations</code>	Abilita la valutazione lazy delle annotazioni di tipo (PEP 563), necessaria per riferimenti forward e compatibilità con Python 3.10+ prima che diventasse default.
<code>typing.Optional</code>	Usato per i campi che possono essere <code>None</code> (es. <code>elapsed_pellet</code> , <code>qualifier</code>). In Python 3.10+ si potrebbe usare <code>X None</code> , ma <code>Optional</code> mantiene la compatibilità con toolchain più datate.
<code>pydantic.BaseModel</code>	Classe base di tutti i modelli; fornisce parsing, validazione e serializzazione automatica.
<code>pydantic.Field</code>	Permette di arricchire i campi con metadati (descrizione, esempi, vincoli numerici) che vengono inclusi nello schema OpenAPI.

9.3 Panoramica dei gruppi di modelli

I modelli sono organizzati in sette gruppi logici, corrispondenti ai tag degli endpoint di `main.py`:



Figure 3: Raggruppamento dei modelli Pydantic per dominio funzionale.

9.4 Gruppo: Status

9.4.1 OntologyStats

```
1 class OntologyStats(BaseModel):
2     classes: int
3     individuals: int
4     properties: int
```

Listing 12: OntologyStats — statistiche dell'ontologia.

Modello ausiliario annidato in `StatusResponse`. Espone i tre contatori fondamentali dell'ontologia caricata, utili per diagnostica e monitoraggio. La scelta di usare un oggetto annidato anziché tre campi flat in `StatusResponse` migliora la leggibilità del JSON e permette di estendere le statistiche senza modificare la struttura radice.

9.4.2 StatusResponse

```
1 class StatusResponse(BaseModel):
2     status: str = Field(..., examples=["ready", "loading",
3     ↪ "error"])
4     message: str
5     ontology_path: str = ""
6     elapsed_pellet: Optional[float] = Field(
7     None, description="Pellet runtime in seconds")
8     stats: Optional[OntologyStats] = None
```

Listing 13: StatusResponse — risposta di GET /status.

Il campo `status` è una stringa libera (non un `Literal` o un `Enum`) per mantenere la flessibilità in caso di stati futuri. Il valore `elapsed_pellet` è `Optional[float]` perché non è disponibile finché Pellet non ha completato; `stats` è anch'esso opzionale per lo stesso motivo.

`ontology_path` ha come default una stringa vuota anziché `None` per evitare null nel JSON, che richiederebbe gestione esplicita nel frontend.

9.5 Gruppo: Workers

9.5.1 WorkerSummary e WorkerDetail

```
1 class WorkerSummary(BaseModel):
2     id: str
3     first_name: str
4     surname: str
5     is_selected: bool
6     evaluated_for_jobs: list[str]
```



```

7
8 class WorkerDetail(BaseModel):
9     id: str
10    first_name: str
11    surname: str
12    is_selected: bool
13    evaluated_for_jobs: list[str]

```

Listing 14: WorkerSummary e WorkerDetail.

Nota di design — WorkerSummary vs WorkerDetail

I due modelli hanno attualmente la stessa struttura. Questa duplicazione è **intenzionale**: separa concettualmente i due contesti d'uso (lista vs. dettaglio singolo) e permette di aggiungere campi specifici al dettaglio (es. storico delle valutazioni, metadati clinici) senza modificare la struttura della lista, che potrebbe essere cacheable o paginata in futuro.

I campi `first_name` e `surname` sono distinti (non concatenati) per permettere ordinamenti e filtri lato client senza parsing. Il campo `evaluated_for_jobs` è una lista di stringhe (IRI locali dei job) anziché oggetti `JobSummary` annidati, per evitare risposte troppo verbose nella lista workers.

9.5.2 SelectWorkerRequest e SelectWorkerResponse

```

1 class SelectWorkerRequest(BaseModel):
2     worker_id: str = Field(..., examples=["Patient1"])
3
4 class SelectWorkerResponse(BaseModel):
5     previous: Optional[str] = Field(
6         None, description="ID del worker precedentemente
7         ↳ selezionato.")
8     selected: str = Field(..., description="ID del worker ora
9         ↳ selezionato.")

```

Listing 15: Selezione del lavoratore attivo.

La risposta include `previous` per permettere al frontend di aggiornare lo stato UI (es. deselectare il marker precedente nello scatter plot) senza dover mantenere lo stato localmente o fare una chiamata aggiuntiva a `GET /workers`.

9.5.3 UpdateWorkerJobsRequest e UpdateWorkerJobsResponse

```

1 class UpdateWorkerJobsRequest(BaseModel):
2     """Replace the complete set of isEvaluatedForJob links."""
3     job_ids: list[str] = Field(

```

```

4         ...,
5         description="Lista ordinata di job ID da assegnare.",
6         examples=[["Job_AssemblyWorker", "Job_Carpenter"]],
7     )
8
9     class UpdateWorkerJobsResponse(BaseModel):
10         worker_id: str
11         previous: list[str] = Field(..., description="Job assegnati
12             ↪ prima.")
13         assigned: list[str] = Field(..., description="Job assegnati
14             ↪ dopo.")
15         unresolved: list[str] = Field(
16             default_factory=list,
17             description="ID richiesti non trovati nell'ontologia (
18                 ↪ ignorati).",
19         )

```

Listing 16: Aggiornamento dei job assegnati a un lavoratore.

La semantica di **sostituzione atomica** (non append) semplifica la logica client: il frontend invia sempre la lista completa desiderata, senza dover calcolare diff. Il campo `unresolved` segnala silenziosamente gli ID non validi senza far fallire l'intera operazione, implementando una politica di *partial success*.

`default_factory=list` (anziché `default=[]`) è la pratica corretta in Pydantic v2 per i campi con default mutabile, evitando la condivisione accidentale dello stesso oggetto lista tra istanze diverse.

9.6 Gruppo: Jobs

9.6.1 JobSummary

```

1 class JobSummary(BaseModel):
2     id: str
3     label: str

```

Listing 17: JobSummary — elemento di lista job.

Modello minimo per la lista dei job. Il campo `label` contiene la denominazione leggibile del profilo lavorativo (es. “*Assembly Worker*”) distinta dall’IRI locale (`id`).

9.6.2 JobSkillProfileEntry

```

1 class JobSkillProfileEntry(BaseModel):
2     """One skill/ability required by a job, with its raw 0*NET
3     ↪ score."""
4     id: str

```

```

4     score: int = Field(..., description="O*NET importance score
    ↪ [0-100]")

```

Listing 18: JobSkillProfileEntry — skill con punteggio O*NET.

Usato da GET /jobs/{id}/profile. Il punteggio O*NET grezzo (0–100) viene qui esposto direttamente, a differenza dell’anchor [0–3] usato nel matching, perché questo endpoint è pensato per visualizzazioni radar/inclinazione che richiedono la scala completa.

9.6.3 ImportanceEntry e SkillScore

```

1 class SkillScore(BaseModel):
2     id: str
3     score: int
4
5 class ImportanceEntry(BaseModel):
6     job_id: str
7     importance_level: str = Field(
8         ...,
9         examples=["isVeryImportantFor", "isImportantFor",
10                  "isSomewhatImportantFor", "isLessImportantFor"],
11     )
12     skills: list[SkillScore]

```

Listing 19: Importanza skill per job (Q4).

ImportanceEntry aggrega le skill per **livello di importanza**, rispecchiando la struttura delle regole SWRL 9–12. Il valore di `importance_level` corrisponde direttamente al nome della proprietà OWL inferita da Pellet, mantenendo trasparenza tra ontologia e API.

9.7 Gruppo: Matching

9.7.1 MatchResult

```

1 class MatchResult(BaseModel):
2     worker_id: str
3     job_id: str
4     gcs_pct: float = Field(..., description="General
    ↪ Criticality Score %")
5     aisa_pct: float = Field(..., description="Amount
    ↪ Impaired SkAb %")
6     n_total: int = Field(..., description="Numero
    ↪ totale di SkAb valutate")
7     n_critical: int = Field(..., description="SkAb con CS
    ↪ > 0")

```

```

8      suitability:      str      = Field(...,
9          examples=["SUITABLE", "SUITABLE WITH PRECAUTIONS", "NOT
            ↪ SUITABLE"])
10     suitability_color: str      = Field(...,
11         examples=["#22c55e", "#f59e0b", "#ef4444"])

```

Listing 20: MatchResult — coppia (worker, job) con punteggi.

Il campo `suitability_color` merita attenzione: incorpora la logica di colore direttamente nella risposta API. Questa scelta accoppia leggermente il backend alla presentazione, ma semplifica il frontend che non deve reimplementare le soglie di Fig. 4 del paper Rientr@. I tre colori corrispondono al sistema di semaforo verde / giallo / rosso:

Table 9: Mapping suitability → colore hex.

Suitability	Colore	Hex
SUITABLE	Verde	#22c55e
SUITABLE WITH PRECAUTIONS	Ambra	#f59e0b
NOT SUITABLE	Rosso	#ef4444

I colori hex usati corrispondono ai valori *Tailwind CSS* standard (green-500, amber-400, red-500), coerenti con la palette del frontend React.

9.7.2 MatchDetailRequest

```

1 class MatchDetailRequest(BaseModel):
2     worker_id: str = Field(..., examples=["Patient1"])
3     job_id:     str = Field(..., examples=["Job_AssemblyWorker"])

```

Listing 21: Request body per POST /match/detail.

La coppia (worker, job) viene trasmessa come **POST body** anziché come query string o path parameter per due ragioni: (1) semantica REST più corretta per un'operazione di ricerca complessa; (2) facilitazione di eventuali estensioni future (aggiunta di filtri, opzioni di calcolo).

9.7.3 SkillDetail e SkillDetailResponse

```

1 class SkillDetail(BaseModel):
2     id: str
3     score: int = Field(..., description="0*NET score
            ↪ [0-100]")
4     importance_label: str = Field(..., examples=["
            ↪ isVeryImportantFor"])

```

```

5     anchor:          int    = Field(..., description="Importance
    ↪ anchor [0-3]")
6     qualifier:       int    = Field(..., description="ICF
    ↪ qualifier value")
7     cs:              int    = Field(..., description="CS =
    ↪ qualifier x anchor")
8     cs_normalized:   float  = Field(..., description="CS / 12 (
    ↪ normalized)")
9     criticality_label: str    = Field(..., examples=["MODERATELY
    ↪ CRITICAL"])
10    description:      str    = Field("", description="Descrizione
    ↪ ICF della skill")
11
12    class SkillDetailResponse(BaseModel):
13        worker_id: str
14        job_id:   str
15        skills:   list[SkillDetail]

```

Listing 22: SkillDetail — dettaglio per singola skill (Q3).

SkillDetail è il modello più ricco del sistema: espone l'intera catena di calcolo del Criticality Score per ogni skill, dalla fonte grezza (O*NET) alla classificazione finale. Questo permette al frontend di costruire tabelle di dettaglio trasparenti e verificabili dall'utente clinico.

Il campo `description` ha come default una stringa vuota (non `None`) per coerenza con la serializzazione: il frontend riceve sempre un campo stringa, senza dover gestire il caso null.

9.8 Gruppo: Health Conditions

9.8.1 HealthCondition e HealthConditionsResponse

```

1    class HealthCondition(BaseModel):
2        icf_code:      str
3        icf_name:      str = ""
4        description:   str = ""
5        core_sets:     list[str] = []
6        bf_qualifier:  Optional[int] = None
7        ap1_qualifier: Optional[int] = None
8
9    class HealthConditionsResponse(BaseModel):
10        worker_id: str
11        conditions: list[HealthCondition]

```

Listing 23: Condizione di salute ICF di un lavoratore.

HealthCondition modella un singolo codice ICF con i suoi qualificatori. La distinzione tra **bf_qualifier** e **ap1_qualifier** riflette la struttura ICF che prevede qualificatori separati per *Body Functions/Structures* (b/s) e *Activities & Participation* (d). Entrambi sono **Optional[int]** perché un codice potrebbe non avere entrambi i qualificatori valorizzati nell'ontologia.

Il campo **core_sets** è una lista di stringhe che identifica a quali core set ICF appartiene il codice (es. "Stroke", "Low Back Pain"), utilizzato dal frontend per filtrare la visualizzazione.

9.8.2 IcfCodeEntry

```

1 class IcfCodeEntry(BaseModel):
2     icf_code: str
3     icf_name: str = ""
4     description: str = ""
5     category: str = ""
6     core_sets: list[str] = []
7     iri: str = ""

```

Listing 24: IcfCodeEntry — catalogo ICF per la procedura guidata.

Rispetto a **HealthCondition**, questo modello aggiunge:

- **category**: categoria ICF di primo livello (es. "Body Functions", "Activities"), utile per raggruppare i codici nella tabella di selezione della wizard.
- **iri**: IRI completo dell'individuo nell'ontologia, necessario per operazioni di lookup preciso nel reasoner.

I qualificatori sono assenti perché **IcfCodeEntry** rappresenta il *catalogo* dei codici disponibili, non l'istanza specifica di un lavoratore.

9.8.3 HcChangeItem, UpdateHealthConditionsRequest e UpdateHealthConditionsResponse

```

1 class HcChangeItem(BaseModel):
2     icf_code: str
3     action: str = Field(..., examples=["add", "remove", "modify"])
4     qualifier: Optional[int] = Field(None, ge=0, le=4)
5
6 class UpdateHealthConditionsRequest(BaseModel):
7     worker_id: str
8     changes: list[HcChangeItem]
9
10 class UpdateHealthConditionsResponse(BaseModel):
11     worker_id: str

```

```

12     added:      int
13     removed:   int
14     modified:  int

```

Listing 25: Modelli per l'aggiornamento batch delle condizioni ICF.

Il vincolo `ge=0, le=4` su `qualifier` rispecchia la scala ICF ufficiale (0 = nessun problema, 4 = problema completo). Pydantic v2 applica questo vincolo in fase di parsing e restituisce un 422 se violato, senza che il codice dell'endpoint debba effettuare validazione manuale.

Il campo `qualifier` è `None` per l'azione `remove` (dove non ha senso specificare un valore) ed è obbligatorio per `add` e `modify` a livello applicativo (il reasoner lo gestisce). Non è stato reso obbligatorio a livello Pydantic per mantenere un unico modello per tutte e tre le azioni.

La risposta riporta i contatori delle tre operazioni (`added/removed/modified`) anziché la lista completa delle condizioni aggiornata, per leggerezza. Il frontend può ricaricare il profilo completo con `GET /health-conditions/{worker_id}` se necessario.

9.9 Diagramma delle relazioni tra modelli

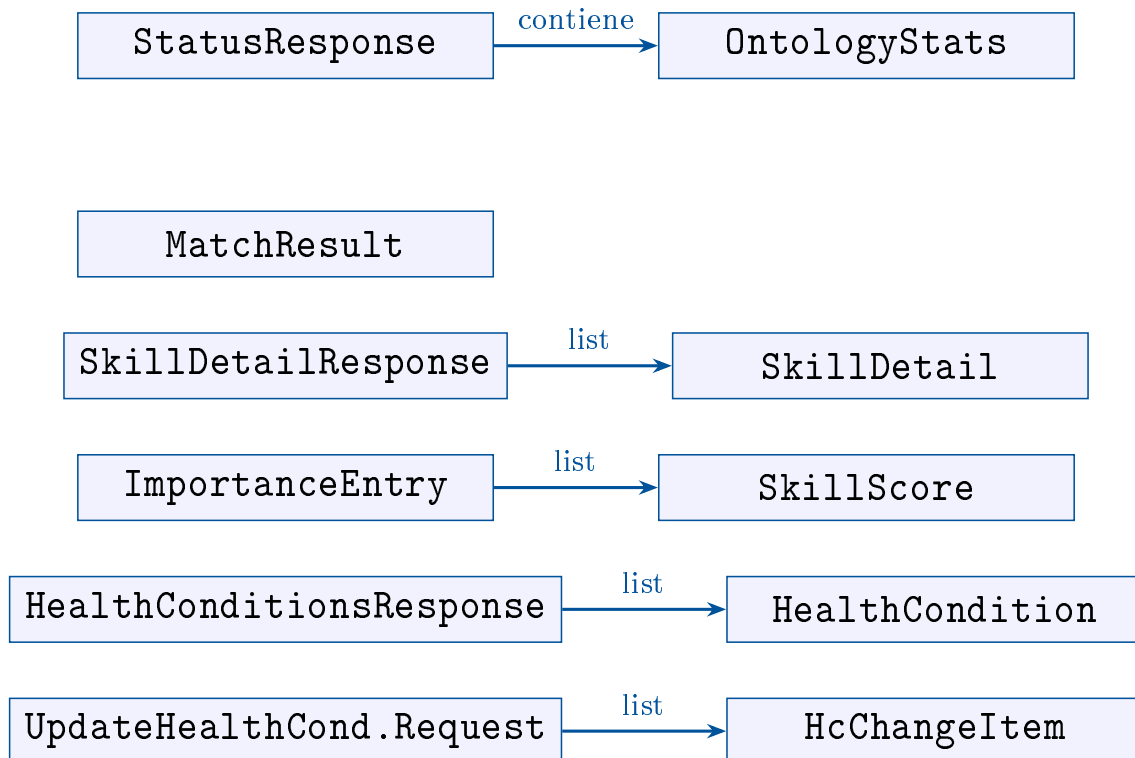


Figure 4: Relazioni di composizione tra i modelli Pydantic principali.

9.10 Convenzioni adottate nel modulo

Convenzione	Motivazione
Campi stringa con default "" anziché None	Evita <code>null</code> nel JSON per campi che il frontend tratta sempre come stringhe (nomi, descrizioni, IRI).
Liste con default []	Coerenza: il frontend non deve gestire <code>null</code> vs. lista vuota. In Pydantic v2 le liste di default sono sicure (non condivise tra istanze).
<code>Optional[int]</code> per i qualificatori	I qualificatori ICF possono essere assenti se il codice è presente nell'ontologia senza un valore esplicito.
<code>Field(description=...)</code> su tutti i campi non ovvi	Popola la documentazione OpenAPI con testo leggibile, riducendo la necessità di documentazione separata.
<code>Field(examples=...)</code>	Fornisce valori di esempio nello schema OpenAPI/Swagger UI, facilitando il testing manuale degli endpoint.
Nessun <code>model_config</code> o <code>Config</code> class	I default di Pydantic v2 (<code>strict mode off</code> , <code>from_attributes off</code>) sono adeguati; la serializzazione avviene tramite <code>model_dump()</code> esplicito dove necessario.

10 Modulo `reasoner.py` — Logica semantica e ragionamento

10.1 Ruolo e responsabilità del modulo

`reasoner.py` è il **nucleo computazionale** dell'intero microservizio. La separazione da `main.py` è una scelta architetturale deliberata: `owlready2`, `Pellet` e le query SPARQL non dipendono in alcun modo da FastAPI, Uvicorn o dal protocollo HTTP. Mantenerli in moduli distinti permette di testare la logica semantica in isolamento (importando solo `reasoner.py` senza avviare il server), di sostituire il framework HTTP in futuro senza toccare il codice di reasoning, e di ragionare sulla complessità dei due domini (web vs. ontologico) separatamente. Implementa quattro macro-responsabilità distinte:

1. **Caricamento e ragionamento ontologico:** carica il file RDF in `owlready2` ed esegue il reasoner `Pellet` (DL reasoner con supporto SWRL), ottenendo la chiusura inferenziale completa dell'ontologia.
2. **Query SPARQL:** espone funzioni pubbliche che interrogano l'`owlready2 default_world` per restituire dati strutturati (workers, jobs, condizioni ICF, risultati di matching).
3. **Gestione dello stato del servizio:** mantiene il singleton `ReasonerState` che traccia lo stato del processo di caricamento (`loading` / `ready` / `error`).
4. **Mutazioni in-memory e persistenza:** applica modifiche alle condizioni di salute ICF e agli assegnamenti job, riesegue `Pellet`, e aggiorna sia il file RDF su disco sia la cache locale JSON.

Separazione delle responsabilità

Il modulo è definito come *pure logic layer*: non dipende da FastAPI, non esegue output su console (nessun `print`/`Rich`/`matplotlib`) e non importa `main.py`. Questo design a bassa dipendenza facilita il test unitario delle funzioni di reasoning in isolamento rispetto all'applicazione HTTP.

10.2 Dipendenze e silenziamento dei log

```
1 warnings.filterwarnings("ignore")
2 logging.disable(logging.WARNING)
3 owlready2.set_log_level(0)
```

Listing 26: Silencing dei log rumorosi di `owlready2` e Java.

owlready2 e la JVM sottostante producono un volume elevato di messaggi di log (deprecation warning, messaggi di stato di Pellet, eccezioni catturate internamente). Queste tre righe li sopprimono a livello globale prima che qualsiasi endpoint possa rispondere, evitando inquinamento dei log dell'applicazione. Il logging applicativo rimane disponibile tramite il modulo `logging` standard.

10.3 Costanti IRI dell'ontologia Rientr@

Le proprietà OWL dell'ontologia Rientr@ vengono referenziate tramite i loro IRI completi. La scelta di definire queste costanti a livello di modulo (anziché recuperarle dinamicamente a ogni query) serve a:

- Evitare stringhe magiche sparse nel codice.
- Centralizzare la manutenzione in caso di cambio del namespace.
- Documentare esplicitamente quali proprietà usa il sistema.

Table 11: Costanti IRI principali e loro significato ontologico.

Costante	Proprietà OWL / Significato
IRI_IS_VERY_IMP	<code>isVeryImportantFor</code> — skill critica per un job (SWRL r. 9)
IRI_IS_IMP	<code>isImportantFor</code> — skill importante (SWRL r. 10)
IRI_IS_SOMEWHAT	<code>isSomewhatImportantFor</code> — skill moderatamente imp. (SWRL r. 11)
IRI_IS_LESS	<code>isLessImportantFor</code> — skill poco importante (SWRL r. 12)
IRI_HAS_CRIT	<code>hasSpecificCriticality</code> — CS inferito da Pellet per coppia (SkAb, Persona)
IRI_REQUIRES	<code>requires</code> — relazione Job → JobDescEntry
IRI_CONCERNS	<code>concerns</code> — relazione JobDescEntry → SkAb
IRI_HAS_SCORE	<code>hasScore</code> — punteggio O*NET di un JobDescEntry
IRI_IS_EVAL_JOB	<code>isEvaluatedForJob</code> — Persona valutata per un Job
IRI_IS_IN_HC	<code>isInHealthCondition</code> — Persona → HC
IRI_IS_SELECTED	<code>isSelected</code> — flag di selezione attiva del worker
IRI_IS_TRANSL	<code>isTranslatedWithICFCode</code> — SkAb → codice ICF
IRI_BFQUAL	<code>BFqual</code> — qualificatore ICF Body Functions/Structures
IRI_AP1QUAL	<code>AP1qual</code> — qualificatore ICF Activities & Participation
IRI_FIRST_NAME	<code>first_name</code> (FOAF) — nome del lavoratore
IRI_SURNAME	<code>surname</code> (FOAF) — cognome del lavoratore
IRI_IS_DESCRIBED_BY	<code>isDescribedBy</code> — HC → Descriptor
IRI_INVOLVES_ICF	<code>involvesICFCode</code> — Descriptor → individuo ICF
IRI_ICF_DESCRIPTION	<code>description</code> — testo descrittivo del codice ICF
IRI_ONET_DEFINITION	<code>ONet_definition</code> — definizione testuale O*NET di una SkAb

10.3.1 Soglie di idoneità (Fig. 4 del paper)

1	JS_RED_INTERCEPT	= 21.0	# intercetta soglia NOT SUITABLE
---	------------------	--------	----------------------------------

```

2 JS_YELLOW_INTERCEPT = 15.5      # intercetta soglia SUITABLE WITH
  ↪ PRECAUTIONS
3 JS_SLOPE                = -0.5     # pendenza comune delle due rette di
  ↪ soglia

```

Listing 27: Costanti per la classificazione di idoneità.

Le due rette che delimitano le tre zone del piano GCS%-AISA% sono:

$$\text{Soglia rossa (NOT SUITABLE)} : \text{GCS} = -0.5 \cdot \text{AISA} + 21.0 \quad (4)$$

$$\text{Soglia gialla (PRECAUTIONS)} : \text{GCS} = -0.5 \cdot \text{AISA} + 15.5 \quad (5)$$

Un lavoratore con $\text{GCS} > \text{soglia rossa}$ non è idoneo; tra le due soglie è idoneo con precauzioni; sotto la soglia gialla è idoneo.

10.4 Classe ReasonerState — Stato globale del servizio

```

1 class ReasonerState:
2     def __init__(self) -> None:
3         self._lock          = threading.Lock()
4         self.status         = "loading"
5         self.message        = "Ontology not yet loaded."
6         self.ontology_path  = ""
7         self.elapsed_pellet: Optional[float] = None
8         self.stats: dict    = {}

```

Listing 28: Classe ReasonerState — singleton thread-safe.

`ReasonerState` è un singleton a livello di modulo che espone il ciclo di vita del reasoner agli endpoint FastAPI. Ogni transizione di stato è protetta da un `threading.Lock` per garantire visibilità corretta tra il thread Pellet e i thread dei worker ASGI.

Il lock è necessario perché Python non garantisce l'atomicità dell'assegnamento di attributi di oggetto in un contesto multi-thread: anche `self.status = "ready"` non è atomico in CPython (coinvolge più bytecode). Senza lock, un thread ASGI potrebbe leggere `status = "ready"` mentre `message` è ancora il vecchio valore (*torn read*), producendo risposte JSON internamente inconsistenti a `GET /status`.

Table 12: Metodi di transizione di stato di `ReasonerState`.

Metodo	Transizione / Effetto
<code>set_ready(path, elapsed, stats)</code>	<code>loading</code> → <code>ready</code> ; registra percorso, tempo Pellet e statistiche ontologia.
<code>set_ready_from_cache(path, stats)</code>	<code>loading</code> → <code>ready</code> da cache JSON; nessun elapsed (Pellet non ha girato).
<code>set_loading(message)</code>	Forza ritorno a <code>loading</code> durante un re-reasoning (es. dopo update HC o job).
<code>set_error(error)</code>	<code>loading</code> → <code>error</code> ; messaggio di errore visibile da <code>GET /status</code> .
<code>is_ready()</code>	Controllo booleano rapido usato dai lock di mutazione.

10.4.1 Variabili globali di modulo

```

1 state = ReasonerState() # singleton esposto a
  ↳ main.py
2 _selection_lock = threading.Lock() # serializza mutazioni
  ↳ isSelected
3 _icf_core_set_map : dict[str, list[str]] # ICF code -> [core set
  ↳ labels]
4 _snapshot_cache : dict | None # cache JSON in-memory
5 _live_reasoner_ready : bool # True dopo Pellet
  ↳ completo
6 _pending_selected_worker_id : Optional[str] # richiesta selezione
  ↳ durante warmup
7 _loaded_ontology = None # oggetto ontologia
  ↳ owlready2

```

Listing 29: Variabili globali di stato e cache.

Il flag `_live_reasoner_ready` è distinto da `state.status == "ready"` perché lo stato può diventare `ready` anche grazie alla snapshot cache, mentre le operazioni di *mutazione* (update HC, update jobs) richiedono la presenza dell'ontologia viva in `owlready2` — condizione garantita solo da `_live_reasoner_ready = True`.

10.5 Funzioni di utilità

10.5.1 local_name(entity)

```

1 def local_name(entity) -> str:
2     if hasattr(entity, "name"):
3         return entity.name
4     s = str(entity)
5     return s.split("#")[-1] if "#" in s else s.split("/")[-1]

```

Listing 30: Estrazione del nome locale da un IRI.

Converte un oggetto owlready2 o una stringa IRI nel suo frammento locale (es. `http://...JobList#Job_Carpenter` → `Job_Carpenter`). La priorità all'attributo `.name` di owlready2 è più efficiente della manipolazione stringa.

10.5.2 criticality_label(cs)

```

1 def criticality_label(cs: int) -> str:
2     if cs >= 7: return "EXTREMELY CRITICAL"
3     elif cs >= 5: return "RELEVANTLY CRITICAL"
4     elif cs >= 3: return "MODERATELY CRITICAL"
5     elif cs >= 1: return "SLIGHTLY CRITICAL"
6     else: return "not critical"

```

Listing 31: Mappatura CS → etichetta (Fig. 2 del paper).

Il Criticality Score (CS = $\text{qualifier} \times \text{anchor}$) ha range $[0, 12]$. Le soglie sono definite in Fig. 2 del paper Rientr@ e codificano la gravità dell'impatto di una condizione di salute su una specifica skill. CS = 0 significa nessun impatto; CS = 12 ($\text{qualifier}=4$, $\text{anchor}=3$) è il caso massimo di impatto estremo.

10.5.3 job_suitability(gcs, aisa)

```

1 def job_suitability(gcs: float, aisa: float) -> tuple[str, str]:
2     thr_red = JS_SLOPE * aisa + JS_RED_INTERCEPT # 21 - 0.5*
3     ↪ aisa
4     thr_yellow = JS_SLOPE * aisa + JS_YELLOW_INTERCEPT # 15.5 -
5     ↪ 0.5*aisa
6     if gcs > thr_red:
7         label = "NOT SUITABLE"
8     elif gcs >= thr_yellow:
9         label = "SUITABLE WITH PRECAUTIONS"
10    else:
11        label = "SUITABLE"
12    return label, SUITABILITY_COLORS[label]

```

Listing 32: Classificazione di idoneità per coppia (GCS%, AISA%).

Restituisce una tupla (`label`, `hex_color`). La doppia soglia lineare è dipendente da AISA%: al crescere della percentuale di skill compromesse, la soglia di idoneità si abbassa (GCS critico più basso). Questo modella il fatto che molte compromissioni moderate sono complessivamente più limitanti di poche compromissioni gravi.

10.5.4 `_score_to_anchor(score)`

```

1 def _score_to_anchor(score: int) -> int:
2     if score >= 75: return 3
3     if score >= 50: return 2
4     if score >= 26: return 1
5     return 0

```

Listing 33: (Tabella 2 del paper).]Conversione punteggio O*NET in anchor [0-3] (Tabella 2 del paper).

L'anchor è il fattore moltiplicativo dell'importanza della skill per un dato job. Il punteggio O*NET (0–100) viene discretizzato in quattro livelli secondo la Tabella 2 del paper. Questo permette al sistema di amplificare l'impatto delle skill ad alta importanza nel calcolo del CS.

10.6 Sistema di cache snapshot

Il sistema di cache è uno degli aspetti architetturali più importanti del modulo. Risolve un problema fondamentale: Pellet impiega 30–120 secondi per completare il ragionamento, rendendo inutilizzabile il servizio al primo avvio.

10.6.1 Strategia generale

10.6.2 `_ontology_fingerprint(path)`

```

1 def _ontology_fingerprint(path: str) -> str:
2     abs_path = os.path.abspath(path)
3     stat = os.stat(abs_path)
4     payload = f"{abs_path}|{stat.st_size}|{stat.st_mtime_ns}".
5     ↪ encode()
6     return hashlib.sha256(payload).hexdigest()[:16]

```

Listing 34: Fingerprint del file ontologico per invalidazione della cache.

Il fingerprint combina percorso assoluto, dimensione e timestamp di modifica in nanosecondi. L'hash SHA-256 troncato a 16 caratteri è sufficientemente unico da evitare collisioni pratiche. Il fingerprint viene usato come suffisso del nome del file cache: `inference_{fingerprint}.json`.

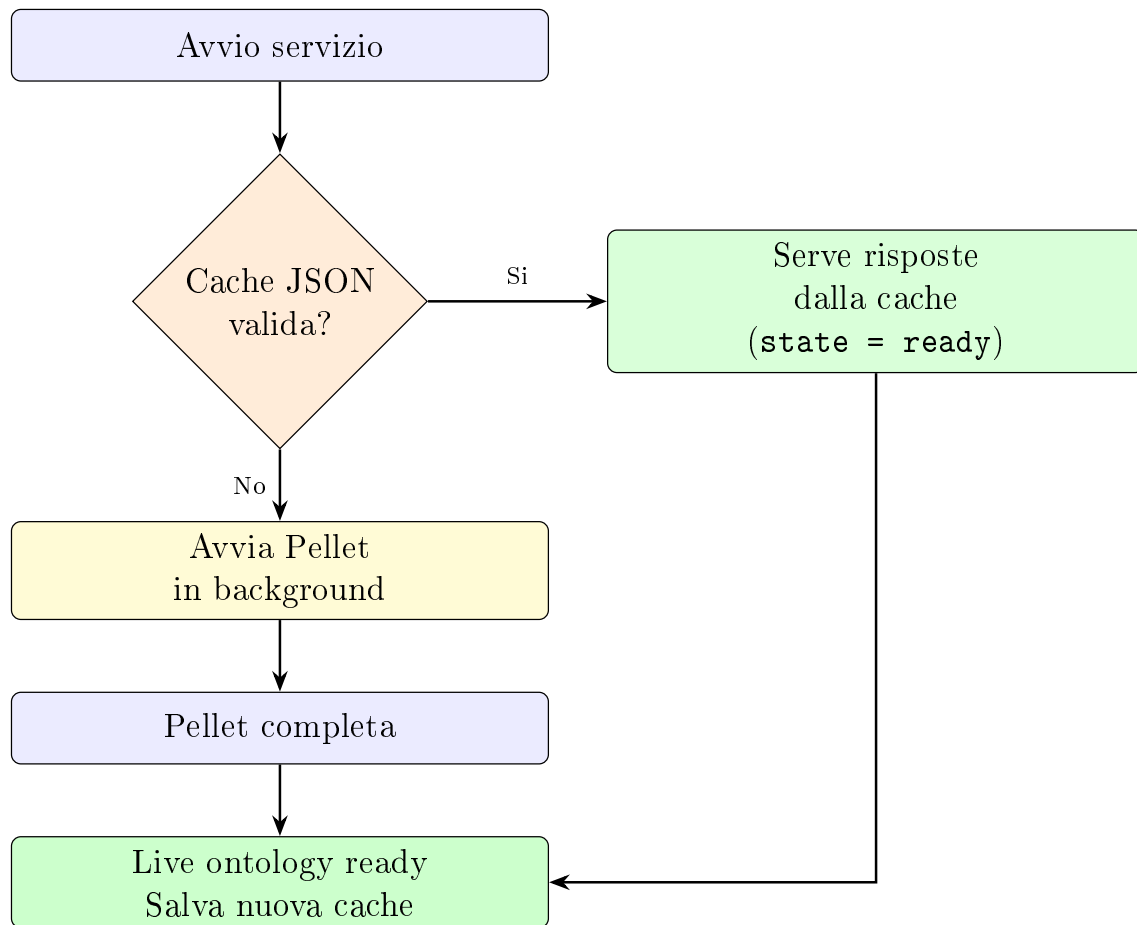


Figure 5: Strategia di avvio rapido con snapshot cache.

La scelta di non fare hash del contenuto del file (che sarebbe più affidabile ma molto più lento su file RDF di decine di MB) è un compromesso consapevole: il timestamp di modifica cattura il 99.9% dei casi reali di cambiamento.

10.6.3 `load_snapshot_cache(rdf_path, update_state)`

Tenta di caricare la cache JSON corrispondente all'ontologia corrente. Valida che il JSON contenga tutte le chiavi obbligatorie prima di accettarlo; un file di cache corrotto o incompleto viene semplicemente ignorato (senza errore), facendo ricadere il sistema sul percorso normale con Pellet.

```
1 required_keys = {  
2     "source_hash", "source_path", "stats",  
3     "workers", "jobs", "core_sets", "icf_codes",  
4     "health_conditions", "importance_by_job",  
5     "job_profiles", "match_results_by_worker",  
6     "skill_details_by_worker_job",  
7 }
```

Listing 35: Chiavi obbligatorie nella snapshot cache.

Il parametro `update_state=True` permette di caricare la cache senza aggiornare il `ReasonerState` — utile quando la funzione viene chiamata internamente dopo un re-reasoning per aggiornare la cache in-memory senza interferire con lo stato visibile agli endpoint.

10.6.4 `_save_snapshot_cache(source_path, elapsed, stats)`

Questa funzione è la più onerosa del modulo in termini di computazione: itera su tutti i worker, seleziona ciascuno temporaneamente con `set_selected_worker`, esegue tutte le query di matching e skill detail, poi ripristina la selezione originale. Questo garantisce che la cache contenga dati precalcolati per ogni combinazione (worker, job).

La scrittura su disco avviene in modalità atomica:

```
1 tmp_path = f"{cache_path}.tmp"  
2 with open(tmp_path, "w", encoding="utf-8") as fh:  
3     json.dump(payload, fh, indent=2, ensure_ascii=True)  
4 os.replace(tmp_path, cache_path) # atomico su POSIX
```

Listing 36: Scrittura atomica della cache su disco.

`os.replace()` è atomico su filesystem POSIX: se il processo termina durante la scrittura, il file originale rimane intatto. `ensure_ascii=True` garantisce compatibilità con ogni sistema di file indipendentemente dall'encoding locale.

10.6.5 `_use_snapshot_cache()`

```

1 def _use_snapshot_cache() -> bool:
2     return _snapshot_cache is not None and not _live_reasoner_ready

```

Listing 37: Condizione di utilizzo della cache.

La cache viene usata **solo** finché il reasoner live non ha terminato. Una volta che Pellet completa (`_live_reasoner_ready = True`), tutte le query tornano a interrogare direttamente l'ontologia owlready2, che ha i dati aggiornati dall'inferenza più recente. Questa transizione avviene automaticamente senza riavvio del servizio.

10.7 Fase 1 — Caricamento dell'ontologia

10.7.1 `_find_rdf_file()` e `_resolve_rdf_path()`

Il percorso del file ontologico viene risolto secondo questa gerarchia di priorità:

1. Argomento esplicito alla funzione.
2. Variabile d'ambiente `ONTOLOGY_PATH`.
3. Auto-detection: primo file `.rdf/.owl/.ttl/.n3` nella directory del servizio o nella directory padre.

La ricerca si ferma alla prima directory che contiene un candidato valido, evitando scan inutili.

10.7.2 `_load_ontology(path)`

```

1 def _load_ontology(path: str):
2     abs_path = os.path.abspath(path)
3     iri = f"file://{abs_path}"
4     buf = io.StringIO()
5     with contextlib.redirect_stderr(buf):           # cattura stderr di
6         onto = get_ontology(iri).load()             ↪ owlready2
7     return onto

```

Listing 38: Caricamento del file RDF in owlready2.

Il caricamento usa l'IRI `file://` anziché il percorso nudo perché owlready2 accetta IRI OWL standard come identificatore. Il `redirect_stderr` cattura i messaggi di avviso di owlready2 (es. triplo malformata, namespace sconosciuto) senza mostrarli a console.

10.8 Fase 2 — Esecuzione di Pellet

10.8.1 `_run_pellet(onto)`

```

1 def _run_pellet(onto) -> Optional[float]:
2     ThingMeta = type(Thing)
3     orig_setattr = ThingMeta.__setattr__
4
5     def patched_setattr(self, name, value):
6         if name == "__bases__":
7             try:
8                 orig_setattr(self, name, value)
9             except TypeError as exc:
10                 if "inheritance cycle" not in str(exc):
11                     raise # propaga solo errori non legati a
12                         ↪ cicli
13         else:
14             orig_setattr(self, name, value)
15
16     try:
17         ThingMeta.__setattr__ = patched_setattr
18         with contextlib.redirect_stderr(io.StringIO()) as
19             ↪ stderr_buf:
20             with onto:
21                 sync_reasoner_pellet(
22                     infer_property_values=True,
23                     infer_data_property_values=True,
24                     ...
25         finally:
26             ThingMeta.__setattr__ = orig_setattr # ripristino SEMPRE
27             ↪ garantito

```

Listing 39: Avvio del reasoner Pellet con monkey-patch anti-ciclo.

Monkey-patch per cicli di ereditarietà ICF

L'ontologia ICF contiene gerarchie di classi molto profonde (centinaia di livelli) che Pellet può interpretare come cicli di ereditarietà (*inheritance cycle*) durante la materializzazione delle classi in owlready2. Questo genera una `TypeError` che interrompe il ragionamento prima del completamento.

Il monkey-patch sostituisce temporaneamente `__setattr__` del metaclass `ThingMeta` (la metaclass di tutte le classi OWL in owlready2) con una versione che cattura e ignora silenziosamente i `TypeError` relativi a cicli, lasciando passare tutti gli altri. Il blocco `finally` garantisce che il metaclass venga sempre ripristinato, anche in caso di errore, evitando effetti collaterali su altre parti

del codice.

Questa tecnica è necessaria perché owlready2 non espone un parametro configurabile per questo comportamento.

Configurazione di Pellet:

- `infer_property_values=True`: Pellet materializza le triple inferite da regole SWRL (es. `isVeryImportantFor`, `hasSpecificCriticality`).
- `infer_data_property_values=True`: materializza anche i valori di proprietà dati (es. qualificatori numerici inferiti).

Estrazione del tempo di esecuzione:

```
1 time_match = re.search(r"Pellet took ([\d.]+)", pellet_log)
2 return float(time_match.group(1)) if time_match else None
```

Listing 40: Parsing del tempo di esecuzione di Pellet dal log.

Il tempo viene estratto tramite regex dal log di stderr di Pellet (se presente) e restituito per essere incluso nella risposta GET `/status`.

10.9 Entry point pubblico — `load_and_reason()`

```
1 def load_and_reason(rdf_path: str = "") -> None:
2     path = _resolve_rdf_path(rdf_path)
3     # 1. Caricamento ontologia
4     onto = _load_ontology(path)
5     _loaded_ontology = onto
6     # 2. Ragionamento Pellet
7     elapsed = _run_pellet(onto)
8     # 3. Statistiche
9     stats = {classes, individuals, properties}
10    _live_reasoner_ready = True
11    # 4. Selezione pending (se arrivata durante il warmup)
12    if _pending_selected_worker_id:
13        set_selected_worker(_pending_selected_worker_id)
14    # 5. Stato pronto
15    state.set_ready(path, elapsed, stats)
16    # 6. Build mappa ICF -> Core Set
17    _build_icf_core_set_map()
18    # 7. Salvataggio cache (best-effort)
19    _save_snapshot_cache(path, elapsed, stats)
20    load_snapshot_cache(path, update_state=False)
```

Listing 41: Sequenza completa di `load_and_reason()`.

Il punto 4 gestisce la **selezione pending**: se durante il warmup di Pellet il frontend ha chiamato `POST /workers/select` (quando era in uso la snapshot cache), la richiesta viene buffered in `_pending_selected_worker_id` e applicata all'ontologia live appena Pellet completa. L'ordine è critico: la selezione deve avvenire *prima* di `state.set_ready()` (punto 5) perché appena lo stato diventa `ready`, il frontend potrebbe chiamare immediatamente `GET /match/{workerId}`, che usa `FILTER(?selected = true)`. Se la selezione fosse applicata dopo `set_ready`, ci sarebbe una finestra temporale in cui la query SPARQL restituirebbe risultati del worker precedente. Questo garantisce la consistenza dello stato.

Il punto 7 è *best-effort*: se la scrittura della cache fallisce (es. disco pieno, permessi), l'eccezione viene silenziata e il servizio continua a funzionare normalmente in modalità live.

10.10 Query SPARQL — Pattern generale

Ogni funzione pubblica di query segue questo pattern strutturato:

```

1 def get_something(param: str) -> list[dict]:
2     # 1) Cache check (se Pellet non ha ancora finito)
3     if _use_snapshot_cache():
4         return list(cast(dict, _snapshot_cache)["key"])
5
6     # 2) Lookup individuo nell'ontologia (se necessario)
7     ind = default_world.search_one(iri=f"*#{param}")
8     if ind is None:
9         raise KeyError(f"'{param}' not found.")
10
11    # 3) Query SPARQL
12    rows = _sparql("""SELECT ...""")
13
14    # 4) Aggregazione e costruzione del risultato
15    result = []
16    for row in rows:
17        result.append({...})
18
19    return sorted(result, key=...)

```

Listing 42: Pattern comune delle funzioni di query.

Il lookup `default_world.search_one(iri="*#" + id)` usa un pattern wildcard che corrisponde a qualsiasi IRI il cui frammento locale è uguale all'ID, indipendentemente dal namespace. Questo rende le query robuste a piccole variazioni del namespace tra versioni diverse dell'ontologia.

10.11 Query Q0 — Workers e Jobs

10.11.1 get_workers()

```

1 SELECT DISTINCT ?person WHERE {
2     ?person <isEvaluatedForJob> ?job .
3 }

```

Listing 43: SPARQL per la lista dei workers.

Il filtro `isEvaluatedForJob` è intenzionale: l'ontologia può contenere individui `FOAF:Person` che non sono ancora stati collegati ad alcun job (es. importati ma non ancora configurati). Restituire queste persone nella lista produrrebbe elementi non actionable: selezionarli porterebbe a `/match` vuoto e a una `JobAnalysisView` con errore. Il filtro garantisce che la lista contenga solo worker su cui il sistema può effettivamente operare.

Per ogni individuo `Person` trovato, le proprietà `first_name`, `surname` e `isSelected` vengono recuperate tramite accesso diretto all'oggetto `owlready2` (più efficiente di una seconda query SPARQL). La lista viene ordinata per ID per garantire output deterministico.

10.11.2 get_jobs()

```

1 SELECT DISTINCT ?job WHERE {
2     ?job <requires> ?jde .
3 }

```

Listing 44: SPARQL per la lista dei job.

Il filtro `requires` esclude i job “vuoti” (senza profilo skill). Il `label` viene generato dal local name sostituendo `_` con spazio, senza overhead di query aggiuntive.

10.12 Query — Condizioni di salute ICF

10.12.1 get_health_conditions(worker_id)

La query naviga la catena ontologica:

`Person` $\xrightarrow{\text{isInHealthCondition}}$ `HC` $\xrightarrow{\text{isDescribedBy}}$ `Descriptor` $\xrightarrow{\text{involvesICFCode}}$ `ICF_Individual`

```

1 SELECT ?icf ?desc ?bfq ?ap1q WHERE {
2     <person_iri> <isInHealthCondition> ?hc .
3     ?hc <isDescribedBy> ?des .
4     ?des <involvesICFCode> ?icf .
5     OPTIONAL { ?icf <description> ?desc }
6     OPTIONAL { ?des <BFqual> ?bfq }

```

```

7   OPTIONAL { ?des <APIqual> ?ap1q }
8 }

```

Listing 45: SPARQL per le condizioni ICF di un worker.

Il local name degli individui ICF nell'ontologia può avere il formato composto **b1408-AuditoryAttention**. Il codice gestisce questo caso con una split sul carattere - e una regex di separazione CamelCase:

```

1 icf_code, embedded_name = full_id.split('-', 1)
2 embedded_name = re.sub(r'(?<=[a-z0-9])(?=[A-Z])', ' ',
   ↪ embedded_name)
3 # "AuditoryAttention" -> "Auditory Attention"

```

Listing 46: Parsing dei compound IRI ICF.

10.12.2 get_all_icf_codes()

Restituisce il catalogo completo dei codici ICF per la wizard di modifica. Quando più individui ontologici si riducono allo stesso codice puro (es. **b1408-Var1** e **b1408-Var2** entrambi codice **b1408**), vengono **fusi** in un unico record, preferendo il primo nome e la prima descrizione non-vuota. Questo è necessario perché la wizard opera sul codice ICF, non sulle varianti ontologiche.

La categoria ICF (*Body Functions, Activities and Participation, Body Structures, Environmental Factors*) viene derivata dal prefisso del codice (b, d, s, e).

10.13 Query Q4 — Importanza skill per job

10.13.1 get_importance_summary(job_id)

Esegue quattro query SPARQL distinte, una per ogni livello di importanza SWRL (regole 9–12). Una singola query con UNION sarebbe semanticamente equivalente ma renderebbe impossibile determinare a quale livello di importanza appartiene ogni skill senza aggiungere una variabile **?level** e un'ulteriore logica di decodifica. Le quattro query separate producono direttamente strutture dati partizionate per livello, pronte per la serializzazione in **ImportanceEntry** senza post-processing:

```

1 for label, iri in IMPORTANCE_IRIS.items():
2     rows = _sparql(f"""
3         SELECT ?skab ?job ?score WHERE {{
4             ?skab <{iri}> ?job .           # la skill e' importante
   ↪ per il job
5             ?jde <concerns> ?skab .       # il descriptor riguarda la
   ↪ skill
6             ?job <requires> ?jde .       # il job richiede il
   ↪ descriptor

```

```

7         ?jde <hasScore> ?score .    # punteggio O*NET
8     }}
9     """)

```

Listing 47: Pattern delle query Q4 per importanza.

Le proprietà `isVeryImportantFor`, `isImportantFor`, ecc. sono inferite da Pellet tramite le regole SWRL 9–12. Non esistono nel file RDF originale ma sono materializzate nell'`owlready2 default_world` dopo `sync_reasoner_pellet`. Questa è la dimostrazione concreta del valore del reasoning: senza Pellet, queste proprietà non esisterebbero e non sarebbe possibile rispondere alla domanda “quali skill sono molto importanti per questo job?” senza ricalcolare le soglie O*NET a runtime in Python. Con Pellet, la risposta è una semplice query SPARQL su triple esistenti.

L’output aggrega le skill per livello di importanza e le ordina per score decrescente, così il frontend può mostrarle in ordine di rilevanza.

10.14 Query Q1+Q2 — GCS% e AISA%

10.14.1 `get_match_results(worker_id)`

La query principale Q1+Q2 è la più complessa del sistema:

```

1 SELECT ?person ?job ?skab ?score ?bfq ?ap1q WHERE {
2     ?person <isEvaluatedForJob> ?job .
3     ?person <isSelected> ?selected .
4     FILTER(?selected = true)
5     ?job <requires> ?jde .
6     ?jde <concerns> ?skab .
7     ?jde <hasScore> ?score .
8     ?skab <isTranslatedWithICFCode> ?icf .
9     ?person <isInHealthCondition> ?hc .
10    ?hc <isDescribedBy> ?des .
11    ?des <involvesICFCode> ?icf .
12    OPTIONAL { ?des <BFqual> ?bfq }
13    OPTIONAL { ?des <APIqual> ?ap1q }
14 }

```

Listing 48: SPARQL principale per GCS% e AISA% (Q1+Q2).

Il `FILTER(?selected = true)` è il nodo centrale della query: seleziona solo i lavoratori marcati come attivi nell’ontologia. Il join tra `?skab isTranslatedWithICFCode ?icf` e `?des involvesICFCode ?icf` realizza il **matching semantico** tra la skill richiesta dal job e la condizione di salute del lavoratore: due entità diverse (una skill e una condizione) sono collegate dal codice ICF che le descrive entrambe.

10.14.2 Calcolo aggregato `_compute_metrics(agg)`

```

1 def _compute_metrics(agg: dict) -> list[dict]:
2     for (p_id, j_id), data in sorted(agg.items()):
3         cs_values = []
4         for s_id, entry in data["skabs"].items():
5             anchor = _score_to_anchor(entry["score"])
6             cs = entry["qual"] * anchor          # CS =
           ↪ qualifier x anchor
7             cs_values.append(cs)
8         n_total = len(cs_values)
9         n_crit = sum(1 for cs in cs_values if cs > 0)
10        sum_norm = sum(cs / 12.0 for cs in cs_values)
11        gcs_pct = (sum_norm / n_total) * 100.0    # GCS%
12        aisa_pct = (n_crit / n_total) * 100.0    # AISA%
13        suit, color = job_suitability(gcs_pct, aisa_pct)

```

Listing 49: Calcolo di GCS% e AISA% dall'aggregazione.

Le formule implementate sono:

$$CS_i = \text{qualifier}_i \times \text{anchor}_i \in [0, 12] \quad (6)$$

$$\text{GCS\%} = \frac{\sum_i \widehat{CS}_i}{N} \times 100 = \frac{\sum_i CS_i / 12}{N} \times 100 \quad (7)$$

$$\text{AISA\%} = \frac{|\{i : CS_i > 0\}|}{N} \times 100 \quad (8)$$

dove N è il numero totale di skill/abilità valutate per la coppia (worker, job).

Le due metriche misurano aspetti *complementari* della compatibilità: **GCS%** misura la *gravità media* dell'impatto delle condizioni di salute sulle skill richieste — un valore alto significa che molte skill sono molto compromesse. **AISA%** misura la *diffusione* dell'impatto — un valore alto significa che molte skill (anche leggermente) sono interessate.

La normalizzazione $CS_i/12$ in GCS% è necessaria per portare tutti i CS sulla stessa scala $[0, 1]$ prima della media: senza normalizzazione, una skill con anchor 3 e qualifier 4 (CS=12) peserebbe tre volte una skill con anchor 1 e qualifier 4 (CS=4), anche se entrambe rappresentano una limitazione completa per i rispettivi livelli di importanza.

La soglia $CS_i > 0$ in AISA% è intenzionalmente inclusiva: anche un CS=1 (lieve) contribuisce al conteggio. Questo riflette la filosofia del sistema: qualsiasi impatto, per quanto minore, è clinicamente rilevante nel contesto della riabilitazione vocazionale.

10.14.3 Fallback `_match_results_fallback()`

Se la query principale restituisce zero righe (es. l'ontologia usa `hasSpecificCriticality` direttamente anziché i qualificatori ICF separati), il sistema ricade su una query alternativa che usa `hasSpecificCriticality` come sorgente del CS. Questo rende il codice resiliente a variazioni nella struttura dell'ontologia.

10.15 Query Q3 — Dettaglio skill per coppia (worker, job)

10.15.1 `get_skill_detail(worker_id, job_id)`

```

1 SELECT ?skab ?cs ?score ?def WHERE {
2     ?skab <hasSpecificCriticality> ?cs .
3     ?jde <concerns> ?skab .
4     <job_iri> <requires> ?jde .
5     ?jde <hasScore> ?score .
6     <person_iri> <isEvaluatedForJob> <job_iri> .
7     OPTIONAL { ?skab <ONet_definition> ?def . }
8 }

```

Listing 50: SPARQL Q3 per il dettaglio skill.

Usa `hasSpecificCriticality` (proprietà inferita da Pellet per coppia SkAb-Persona) come sorgente del CS, ricostruendo `anchor` e `qualifier` dalla formula inversa:

$$\text{qualifier} = \frac{CS}{\text{anchor}} \quad (\text{anchor} = \text{_score_to_anchor(score)})$$

Il risultato include anche la definizione testuale O*NET della skill (`ONet_definition`), che nel frontend è mostrata come tooltip o testo descrittivo nella tabella di dettaglio.

La deduplicazione usa la strategia *keep highest CS*: se la stessa skill appare più volte con CS diversi (possibile per individui ICF multipli), viene mantenuta solo la riga con CS massimo.

10.16 Mutazione — Selezione del worker (`set_selected_worker`)

```

1 def set_selected_worker(worker_id: str) -> dict:
2     # Path: cache mode (Pellet non ancora pronto)
3     if _use_snapshot_cache():
4         ... # aggiorna _snapshot_cache["workers"][*]["is_selected
           ↪ " ]

```

```

5     _pending_selected_worker_id = worker_id
6     return {"previous": old_id, "selected": worker_id}
7
8     # Path: live mode
9     sel_prop = default_world.search_one(iri=IRI_IS_SELECTED)
10    with _selection_lock:
11        # 1. Deseleziona tutti
12        for ind in default_world.individuals():
13            if sel_prop[ind] and bool(sel_prop[ind][0]):
14                old_id = local_name(ind)
15                sel_prop[ind] = [False]
16        # 2. Seleziona il target
17        sel_prop[target] = [True]
18    return {"previous": old_id, "selected": worker_id}

```

Listing 51: Mutazione atomica di isSelected.

Il lock `_selection_lock` garantisce che due richieste concorrenti non possano lasciare l'ontologia con due worker selezionati contemporaneamente. Il pattern deseleziona-tutti-poi-seleziona-uno è $O(N_{\text{individui}})$ ma necessario perché owlready2 non espone un indice inverso per la proprietà `isSelected`.

In **cache mode** (durante il warmup), la selezione viene applicata alla cache in-memory e memorizzata in `_pending_selected_worker_id` per essere applicata all'ontologia live quando Pellet completa.

10.17 Mutazione — Aggiornamento condizioni ICF

10.17.1 Struttura del grafo HC nell'ontologia

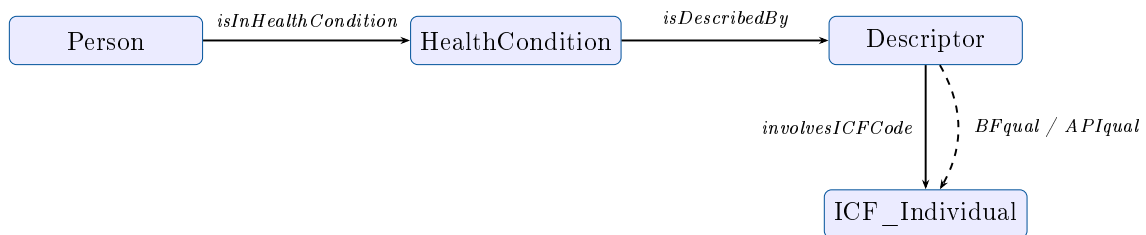


Figure 6: Struttura del sottografo HC nell'ontologia Rientr@.

10.17.2 _safe_set_prop(prop, ind, value)

```

1 def _safe_set_prop(prop, ind, value: int | None) -> None:
2     prop[ind] = [] # 1. flush tutti i triple esistenti
3     if value is not None:
4         prop[ind] = [value] # 2. assegna il nuovo valore

```

Listing 52: Assegnazione sicura di proprietà funzionali in owlready2.

Bug owlready2 — Proprietà funzionali

owlready2 non svuota automaticamente i triple esistenti quando si assegna `prop[ind] = [new_value]` su un individuo che ha già un valore per quella proprietà. Il risultato è la coesistenza di due triple per la stessa proprietà funzionale (BFqual o AP1qual), che causa una `InconsistentOntologyException` in Pellet alla successiva esecuzione.

La soluzione adottata è il flush esplicito `prop[ind] = []` prima di ogni assegnazione, che rimuove tutti i triple esistenti.

10.17.3 Logica delle tre azioni (ADD / REMOVE / MODIFY)**ADD**

1. Risolve l'individuo ICF dalla cache o dall'ontologia.
2. Se il codice è già presente nel profilo HC del worker, lo tratta come **MODIFY** (idempotenza).
3. Crea un nuovo individuo `Descriptor`.
4. Imposta `involvesICFCode`, `BFqual` o `AP1qual`.
5. Se il worker non ha ancora una `HealthCondition`, ne crea una nuova.

REMOVE

1. Localizza il `Descriptor` corrispondente.
2. Lo rimuove dalla lista `isDescribedBy` dell'HC.
3. Chiama `owlready2.destroy_entity()` per rimuovere tutti i triple del descriptor dal grafo.
4. Se l'HC rimane senza descriptor, viene anch'essa distrutta e scollegata dal worker (pulizia del ciclo di vita HC).

MODIFY

1. Localizza il `Descriptor` corrispondente.
2. Aggiorna `BFqual` o `AP1qual` (in base al prefisso del codice) tramite `_safe_set_prop`.
3. Azzera l'altra proprietà per evitare stale data.

10.17.4 Routing BFqual / AP1qual

```

1 def _is_ap1_code(icf_code: str) -> bool:
2     return icf_code.lower().startswith("d")
3
4 # Uso:
5 use_ap1      = _is_ap1_code(icf_code)
6 q_prop       = ap1qual_prop if use_ap1 else bfqual_prop
7 other_prop   = bfqual_prop  if use_ap1 else ap1qual_prop

```

Listing 53: Routing del qualificatore in base al prefisso ICF.

I codici d* (Activities & Participation) usano AP1qual; tutti gli altri (b*, s*, e*) usano BFqual. Questa distinzione rispecchia la struttura del sistema ICF, in cui attività/partecipazione ha un qualificatore separato rispetto alle funzioni corporee.

10.18 Mutazione — Aggiornamento job assegnati

10.18.1 update_worker_jobs(worker_id, job_ids)

```

1 def update_worker_jobs(worker_id: str, job_ids: list[str]) -> dict:
2     ...
3     with _job_lock:
4         previous = [local_name(j) for j in (eval_prop[person_ind]
5         ⇨ or [])]
6         # Risolvi job IDs -> individui owlready2
7         resolved, unresolved = [], []
8         for jid in job_ids:
9             ind = default_world.search_one(iri="*#" + jid)
10            if ind: resolved.append(ind)
11            else:  unresolved.append(jid)
12            # Sostituisce atomicamente
13            eval_prop[person_ind] = resolved
14            _refresh_reasoning_artifacts()

```

Listing 54: Sostituzione atomica dei link isEvaluatedForJob.

`_job_lock` è un lock **separato** da `_selection_lock` perché le due mutazioni non hanno conflitti di risorsa tra loro: `_selection_lock` protegge la proprietà `isSelected` su tutti gli individui, mentre `_job_lock` protegge `isEvaluatedForJob` su un singolo individuo. Usare un unico lock globale serializzerebbe operazioni che potrebbero coesistere in sicurezza (selezione di un worker mentre si aggiornano i job di un altro), riducendo inutilmente il throughput in scenari multi-utente. `_job_lock` serializza le mutazioni degli assegnamenti job. Dopo la sostituzione atomica in-memory, `_refresh_reasoning_artifacts()` esegue la pipeline completa: salva il file RDF, riesegue Pellet, ricalcola la mappa ICF core set, aggiorna le statistiche e sovrascrive la cache JSON.

10.19 _refresh_reasoning_artifacts() — Pipeline di re-reasoning

```

1 def _refresh_reasoning_artifacts() -> None:
2     state.set_loading("Applying changes and recomputing inference
   ↪ ...")
3     _live_reasoner_ready = False
4     _snapshot_cache = None
5     try:
6         _save_live_ontology(ontology_path)      # 1. Salva RDF su
   ↪ disco
7         elapsed = _run_pellet(_loaded_ontology) # 2. Riesegue
   ↪ Pellet
8         _build_icf_core_set_map()              # 3. Ricostruisce
   ↪ mappa ICF
9         _build_icf_ind_cache()                 # 4. Ricostruisce
   ↪ cache ICF ind.
10        stats = {classes, individuals, properties}
11        _live_reasoner_ready = True
12        state.set_ready(...)                   # 5. Stato pronto
13        _save_snapshot_cache(...)              # 6. Salva nuova
   ↪ cache
14        load_snapshot_cache(..., update_state=False) # 7. Ricarica
   ↪ cache
15    except Exception as exc:
16        state.set_error(str(exc))
17    raise

```

Listing 55: Sequenza di re-reasoning dopo una mutazione.

Durante il re-reasoning (passi 1–5), il servizio torna temporaneamente a stato `loading`: gli endpoint restituiranno 503 fino al completamento. Questo garantisce che nessun client riceva dati stale durante il re-reasoning.

10.20 inject_imported_workers() — Integrazione post-import

```

1 def inject_imported_workers(new_person_ids, ontology_path) -> dict:
2     # 1. Usa rdflib (non owlready2) per leggere l'RDF aggiornato
3     g = Graph()
4     g.parse(ontology_path, format="xml")
5
6     for person_id in new_person_ids:
7         # 2. Cerca l'individuo nel live owlready2
8         person_ind = default_world.search_one(iri=str(person_iri))
9

```

```

10      # 3. Inietta isEvaluatedForJob dal file aggiornato
11      new_jobs = [owlready2_ind for iri in g.objects(person_iri,
12      ↪ EVAL_PROP)]
13      eval_prop[person_ind] = existing + new_jobs
14
15      # 4. Imposta isSelected = false
16      sel_prop[person_ind] = [False]
17
18      # 5. Aggiorna snapshot cache
19      _save_snapshot_cache(...)
20      load_snapshot_cache(..., update_state=False)

```

Listing 56: Flusso di inject_imported_workers().

Uso di rdflib per la lettura post-import

L'import SQL scrive nuovi individui nel file RDF su disco, ma il grafo owlready2 in memoria non viene aggiornato automaticamente. Rieseguire Pellet (operazione da 30–120 s) solo per leggere i nuovi link `isEvaluatedForJob` sarebbe eccessivo. La soluzione adottata usa `rdflib` — una libreria Python per la lettura di grafi RDF — per estrarre solo i triple rilevanti dal file aggiornato, e li inietta direttamente nell'oggetto owlready2 in memoria. Questo approccio ibrido (`rdflib` per la lettura, owlready2 per la scrittura live) permette di aggiornare il grafo in pochi millisecondi senza un re-reasoning completo.

10.21 Mappa ICF → Core Set

10.21.1 _build_icf_core_set_map()

```

1 SELECT DISTINCT ?icf ?coreset WHERE {
2     ?icf <rdfs:subClassOf+> ?coreset .
3     ?coreset <rdfs:subClassOf> <ICF_Core_set> .
4 }

```

Listing 57: SPARQL per la mappa ICF code → Core Set.

Il `+` nella proprietà `rdfs:subClassOf+` indica la chiusura transitiva: recupera l'appartenenza indiretta anche attraverso gerarchie di classi profonde. La mappa risultante associa il codice ICF locale (es. `b1408`) alla lista ordinata dei core set di appartenenza (es. `["Hearing Loss", "Stroke"]`).

Questa mappa viene costruita **una sola volta** dopo Pellet, invece di eseguire la query `rdfs:subClassOf+` ad ogni chiamata di endpoint. La motivazione è prestazionale: la chiusura transitiva di `rdfs:subClassOf` su ontologie ICF di grandi dimensioni richiede una traversata ricorsiva del grafo che

può costare centinaia di millisecondi. Eseguirla a runtime per ogni chiamata di GET `/health-conditions/{id}` (che può essere chiamata decine di volte durante una sessione) sarebbe inaccettabile. Il costo del calcolo una-tantum è ammortizzato su tutte le richieste successive, riutilizzata da tutte le query successive (`get_health_conditions`, `get_all_icf_codes`), eliminando migliaia di query SPARQL ripetute a runtime.

10.22 Concorrenza e thread safety

Table 13: Lock di concorrenza e risorse protette.

Lock	Risorsa protetta	Motivazione
<code>State._lock</code>	Campi <code>status</code> , <code>message</code> , <code>stats</code>	Visibilità corretta tra thread Pellet e thread ASGI.
<code>_selection_lock</code>	Proprietà <code>isSelected</code> in <code>owlready2</code>	Evita doppia selezione con richieste concorrenti.
<code>_hc_lock</code>	Mutazioni di <code>Descriptor/HC</code>	Atomicità del batch di modifiche ICF.
<code>_job_lock</code>	Proprietà <code>isEvaluatedForJob</code>	Sostituzione atomica dei job assegnati.

Le query di lettura SPARQL (`get_workers`, `get_match_results`, ecc.) non usano lock perché operano su strutture dati immutabili dopo che Pellet ha completato (il grafo `owlready2` non viene modificato durante le letture normali).

10.23 Flusso dati completo — Dalla query al JSON

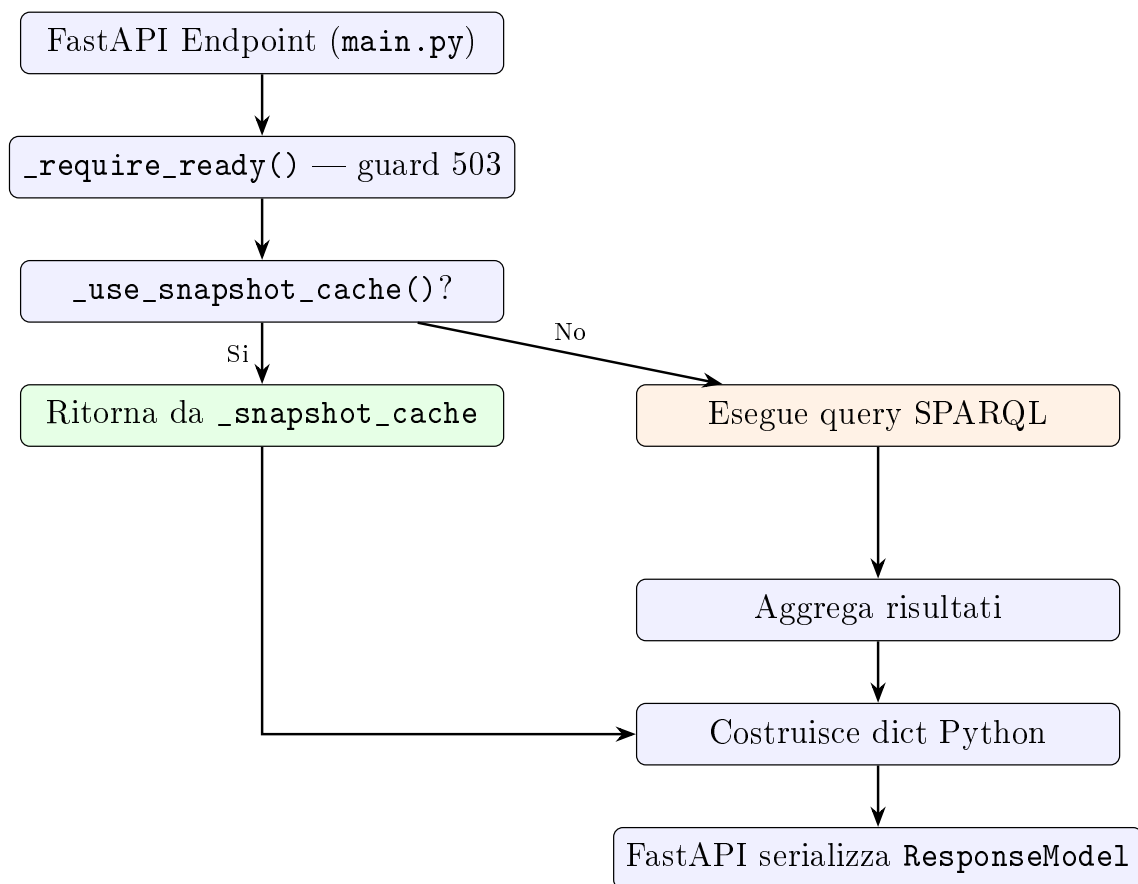


Figure 7: Flusso dati da endpoint a risposta JSON, con cache bypass.

11 Ontologia Rientr@ — Struttura e contenuto

11.1 Panoramica dell'ontologia

L'ontologia Rientr@ è un'ontologia OWL 2 DL sviluppata da **STIIMA-CNR** (Istituto di Sistemi e Tecnologie Industriali Intelligenti per il Manifatturiero Avanzato del CNR). Il file RDF analizzato (**Rientra.rdf**) corrisponde alla versione **3.1**, con ultima revisione al **30 aprile 2024**.

L'ontologia è una *box* composita che importa e integra sette sotto-ontologie distinte, ciascuna con un namespace dedicato:

Table 14: Sotto-ontologie e namespace dell'ontologia Rientr@ v3.1.

Prefisso XML	IRI base	Contenuto
Rien3 (RientraHC)	stiima.cnr.it/RientraHC	Modello condizioni di salute (HC, Descriptor)
FOAF	stiima.cnr.it/ FOAF-excerpt	Estratto FOAF per la descrizione delle persone
ICF-	stiima.cnr.it/ ICF-exc-coreset	Estratto ICF con core set (4 originali + 4 aggiuntivi)
JobL	stiima.cnr.it/JobList	Lista dei profili lavorativi
SkAb	stiima.cnr.it/SkAb	Skills e Abilities da O*NET
JobD	stiima.cnr.it/ JobDescription	Descrizione job (Job Descriptor con score O*NET)
Rien2	stiima.cnr.it/ RientraOnt3Merged	Core set aggiuntivi (Hearing Loss, Low Back Pain, ecc.)

11.2 Statistiche degli individui

Il file RDF contiene **1.458 individui nominati** distribuiti come segue:

Table 15: Distribuzione degli individui OWL per namespace.

Namespace	Individui	Tipo di entità
JobList	971	Job e Job_Descriptor (profili O*NET)
RientraHC	445	HC, Descriptor, Person istanze
ICF-exc-coreset	26	Codici ICF nominati nel core set
Person-CommonBox	10	Individui Person (lavoratori campione)
SkAb	5	Radici Skills/Abilities
RientraOnt3	1	Individuo Match
Totale	1.458	

11.3 Classi principali

Table 16: Classi OWL principali dell'ontologia.

Classe	Significato
FOAF:Person	Individuo lavoratore con nome, cognome e dati anagrafici
FOAF:Person_with_impairment	Sottoclasse di Person per lavoratori con disabilità
RientraHC:Health_Condition	Condizione di salute complessiva di un lavoratore
RientraHC:Health_Condition_with_impairment	HC specifica per condizioni con menomazione
RientraHC:HC_Descriptor	Singola voce ICF con qualificatori nella HC
ICF:ICF	Classe radice dei codici ICF nell'ontologia
ICF:Activities_and_participation	Codici ICF di tipo “d”
ICF:Body_functions	Codici ICF di tipo “b”
ICF:Body_structures	Codici ICF di tipo “s”
ICF:ICF_Core_set	Classe radice dei core set ICF
JobList:Job	Profilo lavorativo (es. Assembly Worker)
JobList:Job_Descriptor	Coppia (SkAb, score O*NET) per un Job
SkAb:Skills	Competenze trasversali da O*NET
SkAb:Abilities	Capacità individuali da O*NET
RientraOnt3:Match	Classe ausiliaria per il matching (legacy)

11.4 ICF Core Set inclusi

L'ontologia include estratti ICF organizzati in **8 core set** clinicamente rilevanti per la riabilitazione vocazionale:

Table 17: ICF Core Set presenti nell'ontologia Rientr@ v3.1.

Core Set	Note
Spinal Cord Injury (Long Term)	Core set originale ICF
Stroke	Core set originale ICF
Traumatic Brain Injury	Core set originale ICF
Vocational Rehabilitation	Core set originale ICF
Hearing Loss	Aggiunto in Rientr@ (RientraOnt3Merged)
Low Back Pain	Aggiunto in Rientr@ (RientraOnt3Merged)
Musculoskeletal Conditions (Post-Acute Care)	Aggiunto in Rientr@
Hand Conditions	Aggiunto in Rientr@

11.5 Proprietà OWL

11.5.1 Proprietà oggetto (Object Properties)

L'ontologia definisce 25 proprietà oggetto. Le più rilevanti per il ragionamento sono:

Table 18: Object Properties chiave dell'ontologia.

Proprietà	Dominio	Range
<code>isEvaluatedForJob</code>	Person	Job
<code>isInHealthCondition</code>	Person	Health_Condition
<code>isDescribedBy</code>	Health_Condition	HC_Descriptor
<code>involvesICFCode</code>	HC_Descriptor	ICF
<code>requires</code>	Job	Job_Descriptor
<code>concerns (funzionale)</code>	Job_Descriptor	Skills $\cup$ Abilities
<code>isTranslatedWithICFCode</code>	Skills $\cup$ Abilities	ICF
<code>isVeryImportantFor</code>	Skills $\cup$ Abilities	Job
<code>isImportantFor</code>	Skills $\cup$ Abilities	Job
<code>isSomewhatImportantFor</code>	Skills $\cup$ Abilities	Job
<code>isLessImportantFor</code>	Skills $\cup$ Abilities	Job
<code>matchedJob</code>	Person	Job

La proprietà **concerns** è dichiarata **funzionale**: ogni Job_Descriptor descrive esattamente una Skill o Ability, garantendo che il grafo dei requisiti di un job sia una struttura senza ambiguità.

11.5.2 Proprietà dato (Datatype Properties)

L'ontologia definisce 20 datatype properties. Quelle usate attivamente dal microservizio sono:

Table 19: Datatype Properties rilevanti per il microservizio.

Proprietà	Tipo XSD	Uso
first_name	string	Nome del lavoratore (FOAF)
surname	string	Cognome del lavoratore (FOAF)
hasScore	integer	Punteggio O*NET [0–100] del Job_Descriptor
BFqual	integer	Qualificatore ICF Body Functions/Structures [0–4]
AP1qual	integer	Qualificatore ICF Activities & Participation [0–4]
hasSpecificCriticality	integer	CS inferito da Pellet per coppia (SkAb, Person)
isSelected	boolean	Flag di selezione attiva del worker
selectedJob	boolean	Flag job selezionato (legacy, non usato dal svc)

Le proprietà AP2qual, AP3qual, AP4qual, BS1qual, BS2qual, BS3qual sono presenti nell'ontologia ma non sono al momento utilizzate dal microservizio (previste per estensioni future del modello ICF).

11.6 Regole SWRL

L'ontologia contiene **13 regole SWRL** che costituiscono il cuore inferenziale del sistema. La scelta di usare SWRL invece di codice Python per il calcolo del CS ha due motivazioni fondamentali: (1) le regole vivono nell'*ontologia*, non nel codice — un ontology engineer può modificarle in Protégé senza toccare Python; (2) Pellet garantisce la consistenza logica delle regole tramite il ragionamento DL, rilevando contraddizioni che codice imperativo non individuerrebbe.

Le 13 regole sono **separate** (non una sola con condizioni complesse) perché SWRL non supporta costrutti condizionali (**if/else**) nella testa delle regole: ogni regola ha esattamente una conclusione. La ramificazione logica (prefisso ICF, livello di importanza) viene espressa moltiplicando le regole. Si dividono in due famiglie:

11.6.1 Famiglia A — Calcolo del Criticality Score (regole 1–9)

Queste 9 regole calcolano **hasSpecificCriticality** (CS) per ogni coppia (SkAb, Person) rispetto al job selezionato. La struttura generale è:

$$\text{Persona}(p) \wedge \text{isSelected}(p, \text{true}) \wedge \text{isEvaluatedForJob}(p, j) \wedge \dots \wedge \text{multiply}(q, a, \text{cs}) \\ \Rightarrow \text{hasSpecificCriticality}(\text{skab}, \text{cs})$$

dove q è il qualificatore ICF e a è l'anchor di importanza (valore fisso per ogni livello). Le regole si differenziano per:

- **Tipo di qualificatore:** **BFqual** (regole 3, 5, 7, 9) vs. **AP1qual** (regole 2, 4, 6, 8) — routing b/d ICF.
- **Livello di importanza:** **isImportantFor** (r.2–3), **isLessImportantFor** (r.4–5), **isSomewhatImportantFor** (r.6–7), **isVeryImportantFor** (r.8–9).
- **Regola 1:** crea l'individuo **Match** associando **Person** a **Job** tramite **matchedJob** (struttura legacy).

Le regole della famiglia A sono **doppie per ogni livello di importanza** (una per **BFqual**, una per **AP1qual**) perché SWRL non supporta la disgiunzione nella testa delle regole. Non si può scrivere “se il codice ICF inizia con b usa **BFqual**, altrimenti usa **AP1qual**” in una singola regola SWRL: la condizione sul prefisso del codice ICF non è esprimibile con i built-in SWRL standard (che operano su valori numerici e stringhe, non su frammenti IRI). La separazione per tipo di qualifier è quindi una necessità architetturale del linguaggio, non una scelta di stile.

Importanza dell'ordine SWRL e isSelected

Tutte le regole della famiglia A verificano la condizione **isSelected(p, true)**. Questo significa che Pellet calcola **hasSpecificCriticality** **solo per il worker correntemente selezionato**. Cambiare il worker selezionato e rieseguire Pellet (tramite **set_selected_worker + _refresh_reasoning_artifacts**) aggiorna i CS per il nuovo worker.

11.6.2 Famiglia B — Classificazione dell'importanza per job (regole 10–13)

Queste 4 regole classificano ogni SkAb rispetto a ciascun Job in base al punteggio O*NET del corrispondente Job_Descriptor, secondo la Tabella 2 del paper. Le soglie estratte direttamente dall'ontologia sono:

Table 20: Regole SWRL 10–13: soglie O*NET e proprietà inferita.

Regola	Soglia score	Anchor	Proprietà inferita
13	≤ 24	0	isLessImportantFor
10	$25 \leq x \leq 49$	1	isSomewhat- ImportantFor
11	$50 \leq x \leq 74$	2	isImportantFor
12	≥ 75	3	isVeryImportantFor

Coerenza SWRL-codice

I valori di anchor [0–3] derivati dalle regole SWRL 10–13 corrispondono esattamente alla funzione `_score_to_anchor()` in `reasoner.py` (Sezione 8.5). Le soglie 25, 50, 75 sono hardcoded nell'ontologia RDF; qualunque modifica alle soglie richiede la modifica del file ontologico, non del codice Python.

11.7 Struttura del grafo di matching

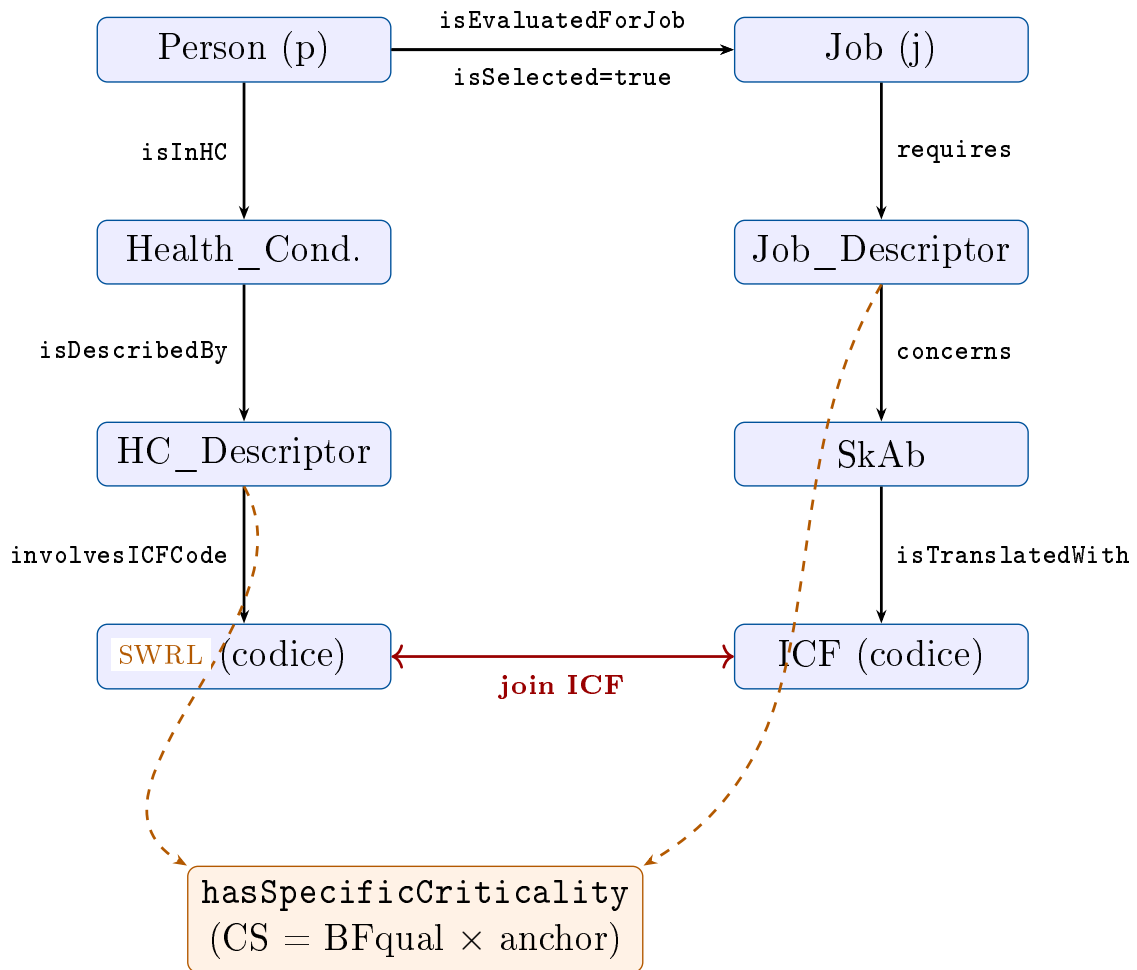


Figure 8: Struttura del grafo di matching: il join sul codice ICF collega la condizione di salute del lavoratore alle skill richieste dal job.

12 Setup, deployment e configurazione

12.1 Requisiti di sistema

Table 21: Requisiti software del microservizio.

Componente	Versione minima	Note
Python	3.11+	Necessario per <code>match-case</code> , type hints avanzati
Java	11+	Richiesto da Pellet via owlready2; deve essere nel PATH
Sistema operativo	Linux / macOS / Windows	Test primari su Linux

Java obbligatorio nel PATH

owlready2 avvia Pellet come sottoprocesso JVM. Se `java` non è disponibile nel PATH del processo Python, il caricamento dell'ontologia fallisce con un errore poco descrittivo. Verificare sempre con `java -version` (deve restituire versione ≥ 11) prima di avviare il servizio.

12.2 Dipendenze Python (requirements.txt)

Table 22: Dipendenze Python con versioni minime e motivazioni.

Pacchetto	Versione	Motivazione
<code>fastapi</code>	$\geq 0.111.0$	Framework ASGI; 0.111 introduce il <code>lifespan</code> stabile e Pydantic v2 come default.
<code>uvicorn[standard]</code>	$\geq 0.29.0$	Server ASGI; <code>[standard]</code> include <code>httptools</code> e <code>uvloop</code> per prestazioni ottimali su Linux.
<code>pydantic</code>	$\geq 2.0.0$	Validazione e serializzazione; v2 richiesta dai modelli di <code>models.py</code> .
<code>owlready2</code>	≥ 0.46	Interfaccia Python a OWL/Pellet; 0.46 risolve bug critici nella gestione di SWRL built-in.
<code>morph-kgc</code>	$\geq 2.6.0$	Materializzatore R2RML W3C per la pipeline di import SQL.
<code>rdflib</code>	$\geq 6.3.2$	Lettura/scrittura di grafi RDF; usato da <code>importer.py</code> e da <code>inject_imported_workers()</code> .
<code>python-multipart</code>	$\geq 0.0.9$	Richiesto da FastAPI per <code>UploadFile</code> / <code>File(...)</code> .
<code>sqlalchemy</code>	$\geq 2.0.0$	Richiesto da morph-kgc per le connessioni ai DB relazionali.

owlready2 e rdflib: coesistenza

owlready2 usa internamente rdflib per il parsing RDF, ma la versione interna può differire da quella installata esplicitamente. L'installazione esplicita di `rdflib>=6.3.2` garantisce che il codice di `inject_imported_workers()` usi la versione corretta dell'API (`Graph.parse()`, `Graph.objects()`) senza conflitti.

12.3 Procedure di installazione

Il virtual environment è **fortemente raccomandato** per due ragioni specifiche al progetto: owlready2 installa binari nativi (JNI bridge) che possono entrare in conflitto con altre versioni installate a livello di sistema; morph-kgc ha dipendenze (SQLAlchemy, rdflib) con versioni minime precise che potrebbero non coincidere con quelle del sistema. Il venv isola il progetto e garantisce riproducibilità tra ambienti diversi.

```
1 # 1. Creare e attivare il virtual environment
2 python3 -m venv .venv
3 source .venv/bin/activate          # Linux/macOS
4 # .venv\Scripts\activate           # Windows
5
6 # 2. Installare le dipendenze
7 pip install -r requirements.txt
8
9 # 3. Verificare Java
10 java -version # deve essere >= 11
11
12 # 4a. Avvio con auto-detection del file RDF
13 #      (posizionare il .rdf nella directory del servizio)
14 uvicorn main:app --host 0.0.0.0 --port 8000
15
16 # 4b. Avvio con percorso esplicito
17 ONTOLOGY_PATH=/path/to/ontology.rdf \
18     uvicorn main:app --host 0.0.0.0 --port 8000
19
20 # 5. Polling della readiness (Pellet impiega 30-120 s)
21 curl http://localhost:8000/status
22 # Attendere {"status": "ready", ...}
```

Listing 58: Setup completo dell'ambiente di sviluppo.

12.4 Modalità di avvio e readiness

Il servizio implementa un **warm-up asincrono**: risponde immediatamente (HTTP 200) alla rotta `/status` con `{"status": "loading"}` sin dal primo istante, mentre Pellet lavora in background. Il codice 503 (*Service Unavailable*) è stato scelto rispetto ad alternative come 425 (*Too Early*) o 202 (*Accepted*) perché: 503 è universalmente supportato dai client HTTP e segnala inequivocabilmente che il servizio esiste ma non è ancora operativo, incoraggiando il retry; 425 non è supportato da tutte le versioni di Spring Boot e dai client HTTP generici; 202 implica che la richiesta è stata accettata e sarà processata in asincrono, semantica non corretta per un endpoint di lettura sincrono.

Il **polling HTTP** è preferito a WebSocket o Server-Sent Events perché: (1) il warm-up è un evento una-tantum, non un flusso continuo; (2) il polling su `/status` non richiede gestione della connessione persistente né autenticazione aggiuntiva; (3) in contesto Electron/localhost la latenza del polling è trascurabile (<1ms). Tutte le altre rotte restituiscono HTTP 503 fino a completamento. Il frontend deve implementare un ciclo di polling su `GET /status` e sbloccare la UI solo quando `status == "ready"`.

Table 23: Comportamento degli endpoint durante il warm-up.

Condizione	HTTP	Note
Pellet in esecuzione	200 (solo <code>/status</code>)	Risponde <code>status</code> : "loading"
Pellet in esecuzione	503 (tutti gli altri)	Cache snapshot disponibile se già esistente
Pellet completato	200	Tutti gli endpoint operativi
Errore Pellet	500	<code>GET /status</code> descrive l'errore

12.5 Integrazione con Spring Boot

Il servizio è progettato per essere chiamato da uno Spring Boot Backend sulla porta 8080. L'integrazione tipica usa `RestTemplate` o `WebClient`:

```

1 @Service
2 public class SemanticService {
3
4     private final RestTemplate restTemplate;
5     private final String baseUrl = "http://localhost:8000";
6
7     public String getWorkers() {
8         return restTemplate.getForObject(
9             baseUrl + "/workers", String.class);
10    }
11
12    public String getMatchResults(String workerId) {
13        return restTemplate.getForObject(
14            baseUrl + "/match/" + workerId, String.class);
15    }
16 }

```

Listing 59: Esempio di chiamata da Spring Boot con `RestTemplate`.

Il CORS è pre-configurato per accettare richieste da `http://localhost:8080` (Spring Boot), `http://localhost:5173` (React/Vite) e `http://localhost:3000`

(React alternativo). Non è implementata autenticazione a livello del microservizio: la gestione auth è delegata interamente a Spring Boot.

12.6 Configurazione Pyright (pyrightconfig.json)

```

1 {
2   "venvPath": ".",
3   "venv": ".venv",
4   "pythonVersion": "3.14",
5   "typeCheckingMode": "off"
6 }
```

Listing 60: pyrightconfig.json — configurazione del type checker.

Table 24: Motivazione di ogni campo pyrightconfig.json.

Campo	Motivazione
<code>venvPath: "."</code>	Pyright cerca il virtual environment nella directory corrente del progetto.
<code>venv: ".venv"</code>	Nome standard del virtual environment generato da <code>python -m venv .venv</code> .
<code>pythonVersion: "3.14"</code>	Impostato alla versione futura per evitare falsi positivi su sintassi moderna (type union <code>X Y</code> , <code>match-case</code>) già usata nel codice ma non ancora pienamente supportata da Pyright per versioni precedenti. Il valore non influenza il runtime, solo l'analisi statica.
<code>typeCheckingMode: "off"</code>	La type-checking è disabilitata perché <code>owlready2</code> non ha stub di tipo (<code>.pyi</code>) completi. Pyright genererebbe centinaia di errori su attributi dinamici degli oggetti <code>owlready2</code> , oscurando problemi reali. La configurazione attuale permette l'uso di Pyright per il completamento automatico e la navigazione del codice senza il rumore dei type errors.

pythonVersion: "3.14" vs runtime

Il valore 3.14 in `pyrightconfig.json` è la versione *di analisi* di Pyright, non la versione di runtime. Il codice richiede Python **3.11**+ in esecuzione (come specificato in `requirements.txt` e `README.md`). Impostare 3.11 o 3.12 in `pyrightconfig` non cambierebbe il comportamento runtime ma potrebbe causare warning su sintassi che Pyright considera ancora sperimentale.

13 Modulo `import_workers/importer.py` — Pipeline RDB2RDF

13.1 Ruolo, contesto e versione

`importer.py` implementa la **pipeline RDB2RDF** del progetto Rientr@: converte dataset di pazienti in formato SQL in individui OWL compatibili con Pellet, iniettandoli nell'ontologia `Rientra.rdf` senza alterarne la struttura originale.

Il modulo è la *versione libreria* di `rientra_import.py` (strumento CLI della cartella Union), adattata per essere chiamata programmaticamente dagli endpoint FastAPI (`POST /import/workers`) invece che da riga di comando. La versione documentata corrisponde al **prototipo strategico v0.1-test** (aprile 2026), sviluppato per validare la strategia implementativa RDB2RDF. Le funzionalità legate all'interfaccia grafica (selezione interattiva dei job, avvio matching, validazione in tempo reale) non sono incluse in questa versione.

Il modulo appartiene al package `import_workers/`, segnalato dal file `__init__.py` (contenuto: sola riga di commento identificativa). Questa struttura a package permette di importare il modulo con `from import_workers.importer import ...` da `main.py`.

13.2 Perché RDB2RDF: motivazione del paradigma

Scelta paradigmatica — RDB2RDF vs manipolazione testuale

RDB2RDF garantisce **interoperabilità, riproducibilità e manutenibilità** del processo di import, in linea con gli standard del Web Semantico W3C. Un approccio ad-hoc basato su manipolazione testuale del file RDF non sarebbe documentabile, verificabile né riusabile con altri strumenti ontologici.

Il W3C definisce due approcci RDB2RDF:

Table 25: Confronto Direct Mapping vs R2RML per il caso Rientr@.

Aspetto	Direct Mapping	R2RML (adottato)
Struttura IRI	Generata dai nomi colonne	Template espliciti (Section 13.15)
Costanti	Non supportate	<code>rr:constant</code>
Logica condizionale	Non supportata	VIEW SQL ausiliarie
Filtraggio	Non supportato	JOIN su tabelle di controllo
Datatype	Inferiti	Dichiarati esplicitamente
Auditabilità	Nessuna	File <code>.ttl</code> versionabile in Git

Il Direct Mapping è inapplicabile per quattro ragioni specifiche al dominio Rientr@:

1. Gli IRI devono rispettare la struttura `Person-CommonBox#{person_id} / RientraHC#des_{id}_{icf}` non derivabile automaticamente dai nomi delle colonne SQL.
2. La proprietà `isSelected` deve essere sempre `false` come costante applicativa (non da colonna).
3. I qualifier ICF (`BFqual/AP1qual/BS1qual`) dipendono dal prefisso del codice (b/d/s): logica condizionale non esprimibile in Direct Mapping.
4. I codici `e*` e i job non presenti nell'ontologia devono essere esclusi automaticamente.

13.3 Perché Morph-KGC come engine R2RML

Scelta engine — Morph-KGC vs RMLMapper vs D2RQ

Morph-KGC è stato scelto per la sua **integrazione nativa con Python**, il supporto a SQLite via SQLAlchemy, e la restituzione diretta di un grafo rdflib senza file intermedi su disco.

Table 26: Confronto engine R2RML.

Engine	Linguaggio	Motivo esclusione / adozione
RMLMapper	Java	Richiede JVM + invocazione <code>subprocess</code> ; integrazione complessa in pipeline Python.
D2RQ	Java	Non più mantenuto attivamente; supporto DB limitato.
Morph-KGC	Python	Adottato: API Python nativa; <code>materialize(config)</code> restituisce <code>rdflib.Graph</code> in memoria; supporto SQLite; conformità R2RML W3C; sviluppo attivo (UPM Madrid).

Limitazione nota su macOS

Su macOS, Morph-KGC non supporta la parallelizzazione quando invocato come libreria Python. Il messaggio *“Parallelization is not supported for darwin when running as a library”* è atteso e non influisce sulla correttezza dei risultati.

13.4 Dipendenze e importazione condizionale

13.4.1 Pattern `_DEPS_OK` e stub `_SAFE_ANY`

```

1 _SAFE_ANY: Any = None
2
3 try:
4     import morph_kgc
5     from rdflib import Graph, URIRef, Namespace
6     from rdflib.namespace import RDF, OWL, RDFS, XSD
7     _DEPS_OK = True
8 except ImportError:
9     _DEPS_OK = False
10    morph_kgc: Any = _SAFE_ANY # type: ignore[no-redef]
11    Graph: Any = _SAFE_ANY # ...
12    # ... altri stub
13    logger.warning("morph_kgc or rdflib not installed...")

```

Listing 61: Importazione condizionale con fallback graceful.

Le dipendenze heavy (`morph_kgc`, `rdflib`) sono importate in un blocco `try/except ImportError`. Se assenti, il modulo si carica comunque con `_DEPS_OK = False`: ogni funzione che le richiede fallisce con un `RuntimeError` descrittivo anziché con un `ImportError` al momento del caricamento del modulo.

Il tipo `_SAFE_ANY` (alias di `Any`) viene assegnato agli stub per i nomi non importati. Pyrefly/Pyright tratta `Any` come compatibile con ogni operazione (chiamata, accesso attributo, subscript), evitando che gli errori di tipo dagli stub si propaghino al resto del codice analizzato staticamente.

Questo pattern garantisce che il microservizio FastAPI si avvii correttamente anche in ambienti dove `morph-kgc` non è installato, degradando gracefully solo la funzionalità di import.

13.4.2 Namespace e costanti di modulo

```

1 if _DEPS_OK:
2     NS_FOAF = Namespace("http://www.stiima.cnr.it/FOAF-excerpt#"
3     ↪ )
4     NS_HC = Namespace("http://www.stiima.cnr.it/RientraHC#")

```

```

4     NS_ICF      = Namespace("http://www.stiima.cnr.it/ICF-exc-
    ↪ coreset#")
5     NS_PERSON  = Namespace("http://www.stiima.cnr.it/Person-
    ↪ CommonBox#")
6     NS_JOB     = Namespace("http://www.stiima.cnr.it/JobList#")
7     NS_RIEONT3 = Namespace("http://www.stiima.cnr.it/RientraOnt3#")
8 else:
9     NS_FOAF = NS_HC = ... = _SAFE_ANY
10
11 CLOSING_TAG   = "</rdf:RDF>" # marcatore per l'iniezione testuale
12 QUALIFIER_MAP = {"b": "BFqual", "d": "AP1qual", "s": "BS1qual"}

```

Listing 62: Namespace rdflib e costanti di modulo.

I namespace rispecchiano esattamente quelli dichiarati nel file RDF (Sezione 10). `QUALIFIER_MAP` codifica il routing `b/d/s` → proprietà ICF, coerente con `_is_ap1_code()` in `reasoner.py`. `CLOSING_TAG` è il punto di iniezione testuale nell'ontologia.

13.4.3 ImportResult — dataclass di risposta

```

1 @dataclass
2 class ImportResult:
3     persons_added: int = 0
4     persons_skipped: int = 0
5     icf_valid: int = 0
6     icf_skipped: int = 0
7     jobs_valid: int = 0
8     jobs_skipped: int = 0
9     backup_path: str = ""
10     new_person_ids: list[str] = field(default_factory=list)
11     skipped_ids: list[str] = field(default_factory=list)
12     error: Optional[str] = None

```

Listing 63: ImportResult — struttura dati di ritorno.

L'uso di `@dataclass` anziché un `Pydantic BaseModel` è intenzionale: `ImportResult` è un oggetto di trasferimento interno tra `importer.py` e `main.py`, non un modello di validazione REST. `main.py` lo converte manualmente in un `JSONResponse` prima di restituirlo al client. Il campo `error = None` indica successo; se valorizzato contiene il messaggio dell'eccezione catturata.

13.5 Funzione pubblica ensure_mapping()

```

1 def ensure_mapping(ontology_path: str, mapping_path: str) -> None:
2     if not _DEPS_OK:
3         raise RuntimeError("morph-kgc / rdflib non installati.")
4     mapping = Path(mapping_path)
5     if mapping.exists():
6         logger.info("[importer] Mapping file already exists: %s",
7             ↪ mapping)
8         return # no-op: già presente
9     logger.info("[importer] Generating mapping from ontology...")
10    _generate_mapping(ontology_path, mapping_path)

```

Listing 64: ensure_mapping() — generazione idempotente del mapping.

Questa funzione è chiamata dal *lifespan handler* di `main.py` a ogni avvio. L'idempotenza (no-op se il file esiste già) è fondamentale perché il *lifespan handler* viene eseguito ad ogni avvio del servizio: senza il controllo di esistenza, il mapping verrebbe rigenerato ad ogni riavvio, causando un parsing completo dell'ontologia (~12.000 triple, diversi secondi) anche quando non è cambiato nulla. Il design idempotente rispetta anche il principio di *least surprise*: chiamare `ensure_mapping` due volte ha lo stesso effetto di chiamarla una sola volta. Il mapping va rigenerato *solo* in caso di modifiche strutturali all'ontologia (nuove proprietà datatype, nuovi namespace, nuove classi), non per ogni import di nuovi pazienti.

13.6 Generazione del mapping R2RML — _generate_mapping()

Questa è la funzione architetturealmente più importante del modulo: analizza l'ontologia OWL e produce un file `rientra_mapping.ttl` derivato automaticamente dalla struttura delle proprietà OWL.

13.6.1 Scoperta automatica delle proprietà FOAF:Person

```

1 for prop in g.subjects(RDF.type, OWL.DatatypeProperty):
2     domain = next(g.objects(prop, RDFS.domain), None)
3     range_ = next(g.objects(prop, RDFS.range), None)
4     if str(domain) != str(foaf_person):
5         continue # solo prop. di FOAF:Person
6     prop_name = str(prop).split("#")[-1]
7     range_str = str(range_) if range_ else str(XSD.string)
8     if range_str == str(XSD.boolean):
9         boolean_props[prop_name] = prop # isSelected,
10        ↪ selectedJob
11    elif range_str in XSD_PLAIN or range_ is None:

```

```

11     plain_props[prop_name]    = prop      # first_name, surname,
    ↪ TIN, ...
12 else:
13     typed_props[prop_name]    = (xsd_type, prop) # birthday,
    ↪ ZIPcode

```

Listing 65: Classificazione automatica delle datatype properties.

Le proprietà vengono partizionate in tre gruppi:

- **plain_props**: stringhe senza datatype esplicito (serializzate come plain literal in R2RML).
- **typed_props**: valori con datatype XSD non stringa (`xsd:dateTime`, `xsd:int`) — serializzati con `rr:datatype`.
- **boolean_props**: proprietà booleane — gestite con `rr:constant` ma **escluse dalla serializzazione XML** per il problema Pellet (Section 13.10).

L’approccio di scoperta automatica garantisce che il mapping sia sempre aggiornato alla struttura OWL corrente senza modifiche manuali al codice.

Analogamente, le proprietà di `HC_Descriptor` (`BFqual`, `AP1qual`, `BS1qual`) vengono scoperte filtrando per dominio `hc:HC_Descriptor`.

13.6.2 Correzione colonna ZIPcode

```

1 col = "zip_code" if prop_name == "ZIPcode" else prop_name

```

Listing 66: Alias colonna ZIPcode → zip_code.

La proprietà OWL si chiama `ZIPcode` (naming OWL), ma la colonna SQL nella tabella `person` si chiama `zip_code` (snake_case SQL). Questo mapping esplicito risolve la discrepanza di naming senza modificare né l’ontologia né lo schema SQL.

13.6.3 Le 5 TriplesMap del mapping R2RML

Il mapping generato contiene esattamente **5 TriplesMap**, ciascuna con una responsabilità precisa:

Table 27: Le 5 TriplesMap di `rientra_mapping.ttl`.

TriplesMap	Sorgente	Triple prodotte
#PersonMap	person	Individuo <code>foaf:Person</code> con tutte le datatype prop. anagrafiche. IRI: <code>Person-CommonBox#{person_id}</code>
#HealthCond.Map	person	Individuo <code>hc:Health_Condition</code> . IRI: <code>Person-CommonBox#HC{person_id}</code>
#DescriptorMapB	v_desc_b	Individui <code>HC_Descriptor</code> per codici b*. Proprietà <code>BFqual</code> .
#DescriptorMapD	v_desc_d	Individui <code>HC_Descriptor</code> per codici d*. Proprietà <code>AP1qual</code> .
#DescriptorMapS	v_desc_s	Individui <code>HC_Descriptor</code> per codici s*. Proprietà <code>BS1qual</code> .
#DescriptorLinkMap	v_hc_links	Triple <code>HC isDescribedBy Descriptor</code> (una per codice ICF).
#JobEvaluationMap	v_job_valid	Triple <code>Person isEvaluatedForJob Job</code> (solo job presenti nell'ontologia).

13.6.4 Il mapping generato: `rientra_mapping.ttl`

Il file prodotto da `_generate_mapping()` è un artefatto documentale autonomo: versionabile in Git, revisionabile da un ontology engineer, e conforme allo standard W3C R2RML. Un estratto del file effettivo generato per l'ontologia Rientr@ v3.1:

```
<#PersonMap>
  rr:logicalTable [ rr:tableName "person" ] ;
  rr:subjectMap [
    rr:template ".../Person-CommonBox#{person_id}" ;
    rr:class foaf:Person ; rr:class owl:NamedIndividual
  ] ;
  rr:predicateObjectMap [
```

```

rr:predicate <.../FOAF-excerpt#first_name> ;
rr:objectMap [ rr:column "first_name" ]
] ;
...
rr:predicateObjectMap [
  rr:predicate <.../RientraOnt3Merged#isSelected> ;
  rr:objectMap [ rr:constant "false"^^xsd:boolean ]
] .

<#DescriptorMapB>
rr:logicalTable [ rr:tableName "v_desc_b" ] ;
rr:subjectMap [
  rr:template ".../RientraHC#des_{person_id}_{icf_code}" ;
  rr:class hc:HC_Descriptor ; rr:class owl:NamedIndividual
] ;
rr:predicateObjectMap [
  rr:predicate hc:involvesICFCode ;
  rr:objectMap [
    rr:template ".../ICF-exc-coreset#{icf_code}" ;
    rr:termType rr:IRI
  ]
] ;
rr:predicateObjectMap [
  rr:predicate hc:BFqual ;
  rr:objectMap [ rr:column "qualifier" ; rr:datatype xsd:
    ↪ integer ]
] .

```

Listing 67: Estratto da rientra_mapping.ttl — MAP 1 e MAP 3B.

Nota sul file reale: datatype hash in luogo di xsd:integer

Nel file `rientra_mapping.ttl` generato, le TriplesMap B, D, S mostrano un datatype con hash (es. `xsd:N1333c4ff5bdc4100a43de2255a2eda34`) anziché `xsd:integer`. Questo è un artefatto di come `rdflib` serializza i blank node del range della proprietà quando il tipo XSD non è riconosciuto come tipo primitivo noto. Il comportamento runtime di Morph-KGC è corretto perché il valore viene passato comunque come intero dalla colonna SQL. Questo è un punto di miglioramento futuro: sostituire il lookup dinamico del range con un mapping fisso `{"BFqual": "integer", ...}`.

13.7 Perché le VIEW SQL invece di query R2RML complesse

Scelta architetturale — VIEW SQL

Le VIEW SQL esprimono logiche di filtraggio e routing che R2RML standard non supporta nativamente (logica condizionale sui prefissi ICF, join su liste di valori validi), mantenendo il mapping Turtle semplice e leggibile.

13.7.1 _create_views(conn, known_icf, known_jobs)

```

1  # Tabelle di controllo (popolate dinamicamente dall'ontologia)
2  conn.execute("CREATE TABLE _known_icf (icf_code TEXT PRIMARY KEY)"
↳ )
3  conn.execute("CREATE TABLE _known_jobs (job_id TEXT PRIMARY KEY)"
↳ )
4  conn.executemany("INSERT OR IGNORE INTO _known_icf VALUES (?)",
↳ ...)
5  conn.executemany("INSERT OR IGNORE INTO _known_jobs VALUES (?)",
↳ ...)
6
7  # VIEW per routing prefisso ICF
8  for view, prefix in [("v_desc_b", "b"), ("v_desc_d", "d"), ("
↳ v_desc_s", "s")]:
9      conn.execute(f"""
10         CREATE VIEW {view} AS
11         SELECT h.person_id, h.icf_code, h.qualifier
12         FROM hc_descriptor h
13         JOIN _known_icf k ON k.icf_code = h.icf_code
14         WHERE lower(substr(h.icf_code,1,1)) = '{prefix}'
15     """)
16
17  # VIEW unione per isDescribedBy
18  conn.execute("""
19     CREATE VIEW v_hc_links AS
20     SELECT person_id, icf_code FROM v_desc_b
21     UNION ALL SELECT ... FROM v_desc_d
22     UNION ALL ... FROM v_desc_s
23 """)
24
25  # VIEW per job validi (con try/except: tabella job_evaluation
↳ opzionale)
26  conn.execute("""
27     CREATE VIEW v_job_valid AS
28     SELECT j.person_id, j.job_id
29     FROM job_evaluation j
30     JOIN _known_jobs k ON k.job_id = j.job_id

```


31 `"""`

Listing 68: Creazione delle VIEW ausiliarie nel DB SQLite in-memory.

Table 28: VIEW SQL ausiliarie e loro scopo.

VIEW	Filtro	Scopo
<code>v_desc_b</code>	codici ICF <code>b*</code> noti	Alimenta MAP 3B → <code>BFqual</code>
<code>v_desc_d</code>	codici ICF <code>d*</code> noti	Alimenta MAP 3D → <code>AP1qual</code>
<code>v_desc_s</code>	codici ICF <code>s*</code> noti	Alimenta MAP 3S → <code>BS1qual</code>
<code>v_hc_links</code>	unione <code>b+d+s</code>	Alimenta MAP 4 → <code>isDescribedBy</code>
<code>v_job_valid</code>	job presenti nell'ontologia	Alimenta MAP 5 → <code>isEvaluatedForJob</code>

Le tabelle `_known_icf` e `_known_jobs` vengono **popolate dinamicamente** dall'ontologia prima di ogni esecuzione (tramite `_extract_known_icf()` e `_extract_known_jobs()`). Questo garantisce che il filtraggio sia sempre aggiornato anche se l'ontologia viene estesa con nuovi codici ICF o nuovi job, senza modificare né il codice né il mapping.

I codici `e*` (Environmental Factors) vengono automaticamente esclusi perché nessuna VIEW li include: non esistono né `v_desc_e` né un match in `_known_icf` per codici con prefisso `e`.

La gestione della tabella `job_evaluation` (opzionale nel dataset) avviene tramite `try/except sqlite3.Error`: se la tabella non esiste, viene creata una VIEW vuota, il che porta a zero link `isEvaluatedForJob` senza errori.

13.8 Perché SQLite in-memory

Scelta database — SQLite in-memory

SQLite in-memory garantisce **zero configurazione, isolamento completo** tra esecuzioni e **riproducibilità totale**: ogni run parte da uno stato pulito, senza residui di esecuzioni precedenti.

13.8.1 `_load_sql(sql_path)`

```

1 def _load_sql(sql_path: str) -> sqlite3.Connection:
2     sql = Path(sql_path).read_text(encoding="utf-8")
3     conn = sqlite3.connect(":memory:")
4     conn.row_factory = sqlite3.Row # accesso per nome colonna

```

```

5     conn.executescript(sql)
6     return conn

```

Listing 69: Caricamento del dataset SQL in SQLite in-memory.

`sqlite3.connect(":memory:")` crea un database completamente in RAM. `conn.row_factory = sqlite3.Row` abilita l'accesso ai risultati per nome di colonna (`row["person_id"]`) anziché per indice, migliorando la leggibilità e la robustezza del codice. `executescript()` esegue il file SQL completo in una singola transazione.

Il vincolo che Morph-KGC richiede un file su disco per la connessione SQLAlchemy viene risolto da `_export_db_to_file()`, che usa `conn.iterdump()` per scrivere il DB in-memory su un file temporaneo, che viene poi eliminato in un blocco `finally`.

13.9 Struttura del dataset SQL di input

Il file SQL importato deve definire le seguenti tabelle. La struttura a tre tabelle separate (`person`, `hc_descriptor`, `job_evaluation`) anziché una tabella piatta riflette la struttura dell'ontologia: `Person`, `HC` e `Descriptor` sono entità distinte con IRI propri e relazioni molti-a-molti (una persona ha molti codici ICF; un codice ICF può appartenere a molte persone). Una tabella piatta con colonne `person_id`, `icf_code`, `qualifier`, `job_id` avrebbe righe duplicate per ogni persona con più codici e più job, complicando il mapping R2RML e rendendo impossibile l'uso delle PRIMARY KEY per la deduplicazione. La tabella `job_evaluation` è opzionale perché l'assegnazione dei job è un'operazione che nella versione grafica sarà gestita interattivamente: non tutti i dataset di import includeranno job pre-assegnati.

```

1  -- Obbligatoria
2  CREATE TABLE person (
3      person_id TEXT PRIMARY KEY,  -- frammento IRI
4      first_name TEXT NOT NULL,
5      surname TEXT NOT NULL,
6      TIN TEXT,                    -- codice fiscale
7      birthday TEXT,               -- formato YYYY-MM-DDTHH:MM:SS
8      city TEXT,
9      country TEXT,
10     zip_code INTEGER              -- xsd:int
11 );
12
13 -- Obbligatoria (anche vuota)
14 CREATE TABLE hc_descriptor (
15     person_id TEXT NOT NULL REFERENCES person(person_id),
16     icf_code TEXT NOT NULL,        -- es. b110, d175, s710 (non e*)
17     qualifier INTEGER NOT NULL CHECK (qualifier BETWEEN 0 AND 4),
18     PRIMARY KEY (person_id, icf_code)

```

```

19 );
20
21 -- Opzionale: se assente, nessun link isEvaluatedForJob viene
   ↳ creato
22 CREATE TABLE job_evaluation (
23     person_id TEXT NOT NULL REFERENCES person(person_id),
24     job_id     TEXT NOT NULL,      -- es. Carpenter, FileClerk
25     PRIMARY KEY (person_id, job_id)
26 );

```

Listing 70: Schema SQL del dataset di import.

13.10 Il problema del boolean con Pellet

Questa sezione documenta uno dei problemi tecnici più insidiosi incontrati durante lo sviluppo, risolto con una soluzione a tre componenti.

13.10.1 Analisi del problema

L'eccezione rilevata a ogni avvio di Pellet dopo un import era:

```

InconsistentOntologyException: Literal value "false"
does not belong to datatype boolean

```

Il debug ha identificato **due cause indipendenti** che si sommarono:

Causa 1: Re-serializzazione rdflib.

Il primo approccio (caricare l'ontologia in rdflib, aggiungere triple, re-serializzare) è stato abbandonato perché rdflib alterava la struttura XML in modo incompatibile con Pellet. Specificamente:

- La proprietà `isSelected` nell'originale è scritta senza prefisso namespace, usando l'`xmlns` di default. rdflib la serializzava con prefisso esplicito (`ns0:isSelected`), che Pellet non riconosceva come equivalente.
- Gli individui esistenti (`FrancaNeri`, `SandraVerdi`) venivano riscritti con `isSelected` come plain literal `Literal('false')` senza datatype.

Causa 2: Bug Morph-KGC su `rr:constant` boolean.

Morph-KGC materializzava `rr:constant "false"^^xsd:boolean` come plain literal senza datatype. Confermato con ispezione diretta:

```

1 for s, p, o in g.triples((None, NS_RIE.isSelected, None)):
2     print(repr(o))
3 # Output: rdflib.term.Literal('false') <-- senza datatype!

```

Listing 71: Debug della costante booleana in Morph-KGC.

13.10.2 Soluzione definitiva

Soluzione al problema boolean — 3 modifiche coordinate

La soluzione adotta tre modifiche coordinate che si rinforzano reciprocamente:

1. **Niente re-serializzazione**: il file originale rimane intatto;
2. **isSelected esclusa dal mapping R2RML**: il grafo materializzato da Morph-KGC non contiene triple `isSelected` (anche se il mapping le genera, non vengono usate per l'iniezione XML);
3. **Scrittura XML manuale**, replicando esattamente la sintassi degli individui esistenti nell'ontologia originale.

```

1 # In _triples_to_rdfxml_blocks():
2 prop_lines.append(
3     f'          <Rien2:isSelected rdf:datatype="{XSD}boolean">'
4     f'false</Rien2:isSelected>'
5 )

```

Listing 72: Scrittura manuale di `isSelected` compatibile con Pellet.

La scrittura **senza prefisso namespace esplicito** (`Rien2:isSelected` anziché un IRI completo) è essenziale: Pellet risolve il nome tramite il namespace di default `xmlns="...RientraOnt3Merged#"` dichiarato nell'intestazione del documento RDF/XML. Usare l'IRI completo potrebbe essere non riconosciuto a seconda di come Pellet gestisce l'espansione del namespace in quel contesto.

13.11 Perché l'iniezione testuale invece della re-serializzazione

Scelta architetturale — iniezione testuale vs re-serializzazione

La re-serializzazione dell'intera ontologia con `rdflib` causa incompatibilità con Pellet. L'iniezione testuale mantiene il file originale intatto, garantendo compatibilità con il reasoner e preservando la struttura XML attesa da Protégé.

```

1 injection = (
2     "\n\n\n    <!-- ===== PERSONE IMPORTATE (RDB2RDF) ===== -->\n\n
   ↪ "
3     + "\n\n".join(blocks) + "\n\n"
4 )
5 updated = rdf_text.replace(CLOSING_TAG, injection + CLOSING_TAG, 1)
6 Path(ontology_path).write_text(updated, encoding="utf-8")

```

Listing 73: Iniezione testuale dei blocchi RDF/XML prima del tag di chiusura.

I nuovi blocchi RDF/XML vengono inseriti immediatamente prima di `</rdf:RDF>` tramite una semplice sostituzione stringa. Il flag `count=1` in `str.replace` garantisce che venga sostituita solo la prima occorrenza, evitando danni in caso di occorrenze multiple.

13.12 Conversione triple $\rightarrow$ blocchi RDF/XML

13.12.1 `_triples_to_rdfxml_blocks(new_graph, persons_to_add)`

Per ogni `person_id` da aggiungere, la funzione costruisce tre blocchi XML ordinati in sequenza inversa rispetto alle dipendenze:

1. **Blocchi HC_Descriptor** (uno per codice ICF valido).
2. **Blocco Health_Condition** (con tutti gli `isDescribedBy`).
3. **Blocco Person** (con dati anagrafici, `isInHealthCondition`, `isEvaluatedForJob` e `isSelected=false`).

L'ordine è determinato dai vincoli di parsing OWL: i descrittori devono esistere prima che la HC li referenzi, e la HC prima che la Person la referenzi. Protegé e Pellet non richiedono questo ordine (risolvono i riferimenti forward), ma rende il file più leggibile e coerente con il pattern degli individui esistenti.

13.12.2 Uso di `xml_escape`

```
1 from xml.sax.saxutils import escape as xml_escape
2 ...
3 prop_lines.append(
4     f'          <{tag}>{xml_escape(str(val))}</{tag}>')
```

Listing 74: Escaping XML dei valori anagrafici.

`xml_escape` converte i caratteri speciali XML (&, <, >, ", ') nelle rispettive entità XML. Necessario per valori anagrafici che possono contenere apostrofi (es. cognomi come *D'Angelo*) o altri caratteri speciali che renderebbero il file RDF/XML malformato.

13.13 Estrazione entità dall'ontologia

13.13.1 `_extract_known_icf(rdf_text)`

```

1 def _extract_known_icf(rdf_text: str) -> set[str]:
2     prefix = "http://www.stiima.cnr.it/ICF-exc-coreset#"
3     return set(re.findall(
4         rf'{re.escape(prefix)}([A-Za-z0-9_\-\-]+)', rdf_text))

```

Listing 75: Estrazione codici ICF validi con regex sul testo RDF.

Usa una regex sul testo grezzo del file RDF anziché parsing `rdflib`. La motivazione è la velocità: il file `Rientra.rdf` ha circa 12.000 triple; parsarlo con `rdflib` impiega diversi secondi, mentre la regex sul testo grezzo opera in millisecondi. La regex è sufficiente perché il formato degli IRI ICF è stabile e non contiene caratteri ambigui.

13.13.2 `_extract_known_jobs(ontology_path)`

```

1 def _extract_known_jobs(ontology_path: str) -> set[str]:
2     g = Graph()
3     g.parse(ontology_path, format="xml")
4     known: set[str] = set()
5
6     # Pattern 1: rdf:type JobList#Job (metaclass/punning pattern)
7     for entity in g.subjects(RDF.type, NS_JOB.Job):
8         known.add(str(entity).split("#")[-1])
9
10    # Pattern 2: subClassOf JobList#Job (gerarchia OWL standard)
11    for subclass in g.subjects(RDFS.subClassOf, NS_JOB.Job):
12        for ind in g.subjects(RDF.type, subclass):
13            known.add(str(ind).split("#")[-1])
14
15    # Pattern 3: qualsiasi soggetto di una tripla requires
16    for job in g.subjects(NS_JOB.requires, None):
17        known.add(str(job).split("#")[-1])
18    return known

```

Listing 76: Estrazione job validi con tre strategie di fallback.

Metaclass/Punning pattern nei Job Rientr@

I job dell'ontologia Rientr@ usano un pattern OWL non standard detto *punning*: la stessa entità è dichiarata sia come `owl:Class` (sottoclasse di `JobList:Job`) sia come individuo con `rdf:type JobList:Job`. Questo permette ai job di essere contemporaneamente classi (per i `Job_Descriptor`) e individui (per `isEvaluatedForJob`).

Il primo approccio implementato (cercare individui istanza di sottoclassi di `Job`) restituiva un insieme vuoto, causando la perdita di tutti i link

`isEvaluatedForJob`. I tre pattern in sequenza gestiscono questo e altri eventuali stili di dichiarazione OWL, rendendo l'estrazione robusta a variazioni nella struttura dell'ontologia.

13.13.3 `_person_exists(rdf_text, person_id)`

```
1 def _person_exists(rdf_text: str, person_id: str) -> bool:
2     return f".../Person-CommonBox#{person_id}" in rdf_text
```

Listing 77: Controllo esistenza persona con ricerca stringa.

Controllo $O(n)$ sul testo grezzo: verifica la presenza dell'IRI dell'individuo nel file. Più veloce del parsing rdflib per questo controllo puntuale. Garantisce idempotenza: i pazienti già presenti nell'ontologia vengono saltati (conteggiati in `persons_skipped`).

13.13.4 `_sanitize_id(name)`

```
1 def _sanitize_id(name: str) -> str:
2     return re.sub(r"[^A-Za-z0-9_\-]", "_", name)
```

Listing 78: Sanificazione dell'ID per IRI validi.

Sostituisce qualsiasi carattere non ammesso negli IRI con `_`. Previene IRI malformati con spazi, apostrofi o caratteri accentati nel `person_id`.

13.14 Separazione -generate-mapping dall'import

Scelta architetturale — separazione generate-mapping / import

La separazione riflette una distinzione fondamentale tra **cambiamenti strutturali** dell'ontologia (rari, competenza dell'ontology engineer) e **aggiornamenti dei dati** (frequenti, operazione routinaria). Generare il mapping richiede il parsing completo dell'ontologia (~12.000 triple); separare evita questo overhead ad ogni import.

Table 29: Quando rigenerare il mapping.

Rigenerare	NON rigenerare
Nuova owl:DatatypeProperty su FOAF:Person	Aggiunta di nuovi pazienti
Nuovo prefisso ICF oltre b/d/s	Aggiunta di nuovi codici ICF esistenti
Modifica dei namespace dell'ontologia	Aggiunta di nuovi job
Nuova classe da mappare	Modifica dei qualificatori ICF

13.15 Struttura degli IRI generati

Gli IRI dei nuovi individui seguono esattamente le convenzioni degli individui esistenti nell'ontologia:

Table 30: Pattern IRI per un paziente con person_id = "MarioBianchi".

Individuo	IRI generato
Person	.../Person-CommonBox#MarioBianchi
Health_Condition	.../Person-CommonBox#HCMarioBianchi
HC_Descriptor (b110)	http://www.stiima.cnr.it/RientraHC#des_MarioBianchi_b110

Motivazioni di naming:

- Person-CommonBox per Person e HC: coerente con gli individui esistenti (FrancaNeri, HCFrancaNeri).
- Prefisso HC per Health_Condition: evita collisioni con la Person stessa e rende riconoscibile il tipo dal solo IRI.

- `des_{person_id}_{icf_code}`: garantisce unicità globale anche quando più pazienti condividono lo stesso codice ICF.

13.16 Entry point pubblico — `import_sql_dataset()`

```

1 def import_sql_dataset(sql_path, ontology_path, mapping_path) ->
  ↳ ImportResult:
2     # 1. Caricamento SQL in SQLite in-memory
3     conn = _load_sql(sql_path)
4     # 2. Lettura ontologia + identificazione nuovi pazienti
5     rdf_text = Path(ontology_path).read_text(encoding="utf-8")
6     known_icf = _extract_known_icf(rdf_text)
7     known_jobs = _extract_known_jobs(ontology_path)
8     new_ids = [id for id if not _person_exists(rdf_text, id)]
9     # Early exit se nessun nuovo paziente
10    if not new_ids: return result
11    # 3. Creazione VIEW ausiliarie
12    _create_views(conn, known_icf, known_jobs)
13    # 4. Export in-memory -> file temporaneo + Morph-KGC
14    db_file = _export_db_to_file(conn)
15    try:
16        new_graph = _run_r2rml(mapping_path, db_file)
17    finally:
18        Path(db_file).unlink(missing_ok=True) # pulizia garantita
19    # 5. Conversione triple -> blocchi RDF/XML
20    blocks = _triples_to_rdfxml_blocks(new_graph, set(new_ids))
21    # 6. Backup + iniezione testuale + salvataggio
22    shutil.copy2(ontology_path, ontology_path + ".bak")
23    updated = rdf_text.replace(CLOSING_TAG, injection + CLOSING_TAG
  ↳ , 1)
24    Path(ontology_path).write_text(updated, encoding="utf-8")

```

Listing 79: Sequenza completa dell'entry point pubblico.

Punti notevoli:

- **Early exit:** se tutti i pazienti del dataset sono già presenti nell'ontologia, la funzione ritorna immediatamente senza avviare Morph-KGC, risparmiando il costo della materializzazione.
- **Backup atomico:** `shutil.copy2()` preserva i metadati del file originale. Il backup ha sempre il suffisso `.bak` ed è il file referenziato in `ImportResult.backup_path`.
- **Pulizia file temporaneo:** il blocco `finally` garantisce che il file `.db` temporaneo venga eliminato anche in caso di errore durante la materializzazione.

- **Gestione eccezioni:** tutte le eccezioni vengono catturate dal `try/except` esterno e memorizzate in `result.error`, permettendo all'endpoint FastAPI di restituire una risposta strutturata anche in caso di errore.

13.17 Architettura della pipeline — diagramma completo

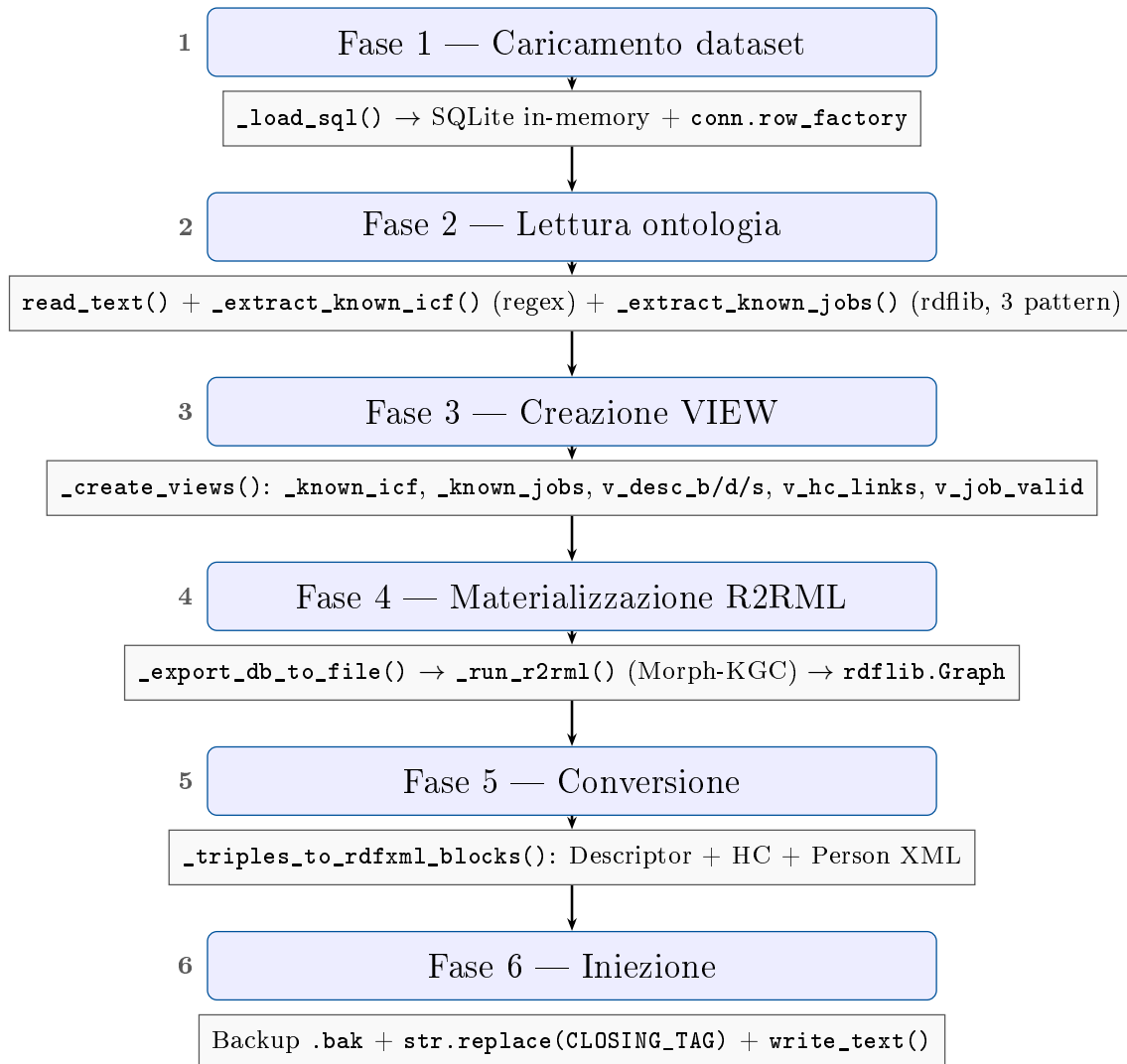


Figure 9: Pipeline completa di `import_sql_dataset()`.

14 Layer applicativo Electron — Struttura del progetto grafico

14.1 Architettura generale dell'applicazione desktop

Il frontend grafico di Rientr@ è un'applicazione desktop costruita con **Electron** + **React** (Vite). La scelta di Electron rispetto alle alternative è motivata da tre fattori concreti:

1. **Gestione del sottoprocesso Python:** Electron espone `child_process.spawn` con pieno controllo di `stdin/stdout/stderr`, pipe di logging e segnali di terminazione (`SIGTERM`). Framework come Tauri (Rust-based) richiedono plugin aggiuntivi o sidecar per gestire processi esterni con questo livello di controllo.
2. **Ecosistema React/Node.js:** il team usa già React e npm. Electron permette di riutilizzare l'intera toolchain (Vite, TypeScript, ESLint) senza aggiungere un secondo linguaggio (Rust per Tauri).
3. **API native immediate:** `dialog.showOpenDialog`, `fs.readFileSync` e `ipcMain.handle` sono disponibili nativamente senza wrapper. NW.js offre funzionalità simili ma con un'architettura IPC meno strutturata e un ecosistema di documentazione più limitato.

Electron funge da *host nativo* che avvolge il frontend web React e gestisce il ciclo di vita del microservizio Python, avviandolo e terminandolo insieme all'applicazione.

La struttura del progetto (directory `Graphic/rientra-app`), visibile dalla struttura fornita, è organizzata come segue:

Table 31: Struttura delle directory del progetto grafico.

Directory / File	Contenuto
<code>electron/</code>	Processo principale Electron (<code>main.js</code> , <code>preload.js</code>). Qui risiede tutta la logica nativa: avvio Python, finestra, IPC, menu.
<code>src/</code>	Sorgenti React/TypeScript del renderer (componenti, pagine, hook, stili).
<code>public/</code>	Asset statici: loghi (<code>logo-rientra.png</code> , <code>logo-stiima.png</code> , <code>logo-inail.png</code>), icone (<code>icons.svg</code> , <code>favicon.svg</code>), immagini UI (<code>Archive.png</code> , <code>User_scan_fill.png</code> , <code>Vector.png</code>).
<code>dist/</code>	Bundle di produzione generato da Vite (<code>vite build</code>), servito da Electron in modalità production tramite <code>win.loadFile()</code> .
<code>python-service/</code>	Directory del microservizio FastAPI (contenuto documentato nelle sezioni precedenti). Referenziata da <code>main.js</code> tramite <code>path.join(__dirname, '..', 'python-service')</code> .
<code>node_modules/</code>	Dipendenze npm (non versionato).
<code>package.json</code>	Manifest npm: dipendenze, script (<code>dev</code> , <code>build</code> , <code>electron</code>), configurazione Electron Forge/Builder.
<code>vite.config.ts</code>	Configurazione Vite per il bundle React.
<code>tsconfig*.json</code>	Configurazioni TypeScript separate per app (<code>tsconfig.app.json</code>), Node (<code>tsconfig.node.json</code>) e root.
<code>eslint.config.js</code>	Regole ESLint per il codice TypeScript/React.
<code>index.html</code>	Entry point HTML di Vite.
<code>start.sh</code>	Script shell di avvio rapido per lo sviluppo.
<code>replace.js</code>	Script di utilità per sostituzioni nel bundle (post-build).
<code>fix_box.py /</code> <code>restructure.py</code>	Script Python di utilità per la gestione del layout di build.

14.2 Architettura a tre processi di Electron

Electron separa l'applicazione in tre processi distinti con privilegi differenti. Questa separazione non è una convenzione arbitraria ma una scelta di sicurezza fondamentale: se il processo `renderer` (che carica HTML/JS potenzialmente da sorgenti esterne) avesse accesso diretto a Node.js, una XSS nel frontend potrebbe leggere il filesystem, avviare processi arbitrari o accedere ai dati clinici dei pazienti. La separazione dei privilegi garantisce che solo il Main Process (che non elabora HTML esterno) abbia accesso alle API di sistema, e che il Preload Script agisca come *mediatore controllato* che espone **esattamente** le operazioni necessarie e nient'altro:

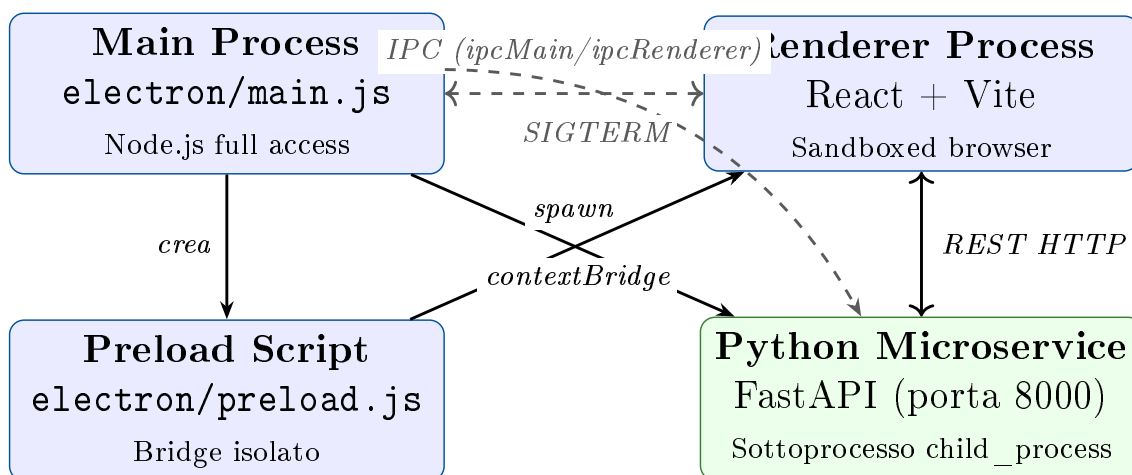


Figure 10: Architettura a tre processi di Electron nel progetto Rientr@.

14.3 Modulo `electron/main.js` — Processo principale

14.3.1 Ruolo e responsabilità

`main.js` è il **processo principale** di Electron: è l'unico processo con accesso completo a Node.js e alle API di sistema. Ha quattro macro-responsabilità:

1. **Avvio e supervisione del microservizio Python** tramite `child_process.spawn`.
2. **Creazione e configurazione della finestra principale** (`BrowserWindow`).
3. **Gestione del ciclo di vita dell'applicazione** (eventi `ready`, `window-all-closed`, `will-quit`).
4. **Esposizione di API native al renderer** tramite handler IPC (`ipcMain.handle`).

14.3.2 Risoluzione dell'eseguibile Python

```

1 const isWindows = process.platform === 'win32';
2
3 const VENV_PYTHON = isWindows
4   ? path.join(SERVICE_DIR, '.venv', 'Scripts', 'python.exe')
5   : path.join(SERVICE_DIR, '.venv', 'bin', 'python3');
6
7 const FALLBACK_PYTHON = isWindows ? 'python' : 'python3';

```

Listing 80: Risoluzione cross-platform dell'eseguibile Python.

La risoluzione dell'eseguibile segue una gerarchia a due livelli:

1. **Virtual environment dedicato:** cerca `python-service/.venv/bin/python3` (Linux/macOS) o `.venv\Scripts\python.exe` (Windows). L'uso del `venv` garantisce che le dipendenze (`owlready2`, `fastapi`, `morph-kgc`) siano isolate dall'ambiente di sistema.
2. **Fallback sistema:** se il `venv` non esiste (`fs.existsSync(VENV_PYTHON) = false`), usa il Python globale del sistema. Questo permette l'avvio anche senza un `venv` pre-configurato, utile in fase di sviluppo.

La scelta di `python3` (non `python`) come fallback su Linux/macOS è intenzionale: su questi sistemi `python` può puntare a Python 2, incompatibile con il microservizio.

14.3.3 startPythonService() — Avvio del sottoprocesso

```

1 pythonProcess = spawn(
2   pythonExe,
3   ['-m', 'uvicorn', 'main:app',
4     '--host', '127.0.0.1',
5     '--port', String(PYTHON_PORT)],
6   {
7     cwd: SERVICE_DIR,
8     stdio: ['ignore', 'pipe', 'pipe'], // cattura stdout/stderr
9   }
10 );
11
12 pythonProcess.stdout.on('data', (data) =>
13   console.log('[Python]', data.toString().trim())
14 );
15 pythonProcess.stderr.on('data', (data) =>
16   console.error('[Python ERR]', data.toString().trim())
17 );

```

Listing 81: Avvio di uvicorn come sottoprocesso Node.js.

Scelta — spawn vs exec vs fork

spawn è preferito rispetto a **exec** perché:

- Non ha limite sulla dimensione dell'output (nessun buffer overflow su log verbosi di Pellet).
- Permette di leggere stdout/stderr in streaming (`on('data')`) per logging in tempo reale nella console Electron.
- Il processo figlio è un riferimento diretto, terminabile con **SIGTERM** preciso.

fork non è applicabile perché riservato ai moduli Node.js, non agli eseguibili Python.

Nota sull'host: il servizio viene avviato su 127.0.0.1 (loopback) anziché 0.0.0.0. Questa scelta limita l'accesso al solo processo locale, prevenendo esposizioni accidentali in rete in modalità desktop.

Cattura degli stream: `stdio: ['ignore', 'pipe', 'pipe']` reindirizza stdout e stderr del processo Python verso i listener `on('data')` di Node.js, che li prefissano con `[Python]` e `[Python ERR]` e li emettono nella console Electron. Questo permette il debug del microservizio direttamente dagli strumenti di sviluppo Electron.

14.3.4 stopPythonService() — Terminazione pulita

```

1 function stopPythonService() {
2   if (pythonProcess) {
3     pythonProcess.kill('SIGTERM'); // graceful shutdown
4     pythonProcess = null;
5   }
6 }

```

Listing 82: Terminazione del sottoprocesso Python.

SIGTERM (anziché **SIGKILL**) permette a uvicorn di eseguire il cleanup del processo ASGI (chiusura delle connessioni HTTP aperte, flush dei buffer di log) prima della terminazione. Il processo Python viene terminato in due momenti del ciclo di vita Electron:

- **window-all-closed:** quando l'utente chiude l'ultima finestra (comportamento standard su Windows/Linux).
- **will-quit:** segnale di uscita dell'app (include il caso macOS dove **window-all-closed** non causa quit).

Il doppio hook garantisce che il processo Python non rimanga *zombie* in background indipendentemente dalla piattaforma.

14.3.5 Creazione della finestra — `createWindow()`

```

1  const win = new BrowserWindow({
2    width: 1280, height: 800,
3    minWidth: 900, minHeight: 600,
4    title: 'RIENTR@ returns -- Decision Support System',
5    backgroundColor: '#1a2a4a',    // blu scuro Rientr@ (evita flash
    ↪ bianco)
6    webPreferences: {
7      preload: path.join(__dirname, 'preload.js'),
8      contextIsolation: true,      // sandbox del renderer
9      nodeIntegration: false,      // blocca accesso Node.js dal
    ↪ renderer
10   },
11 });

```

Listing 83: Configurazione della `BrowserWindow` principale.

Table 32: Motivazione delle opzioni `BrowserWindow`.

Opzione	Motivazione
<code>width/height:</code> 1280x800	Risoluzione minima standard per applicazioni desktop professionali; sufficiente per mostrare scatter plot e tabelle di dettaglio senza scroll.
<code>minWidth/minHeight:</code> 900x600	Soglia sotto cui i layout React collaserebbero; impedisce il ridimensionamento a dimensioni inutilizzabili.
<code>backgroundColor:</code> '#1a2a4a'	Colore di sfondo blu scuro corrispondente al tema dell'applicazione. Evita il “flash bianco” durante il caricamento del renderer.
<code>contextIsolation:</code> <code>true</code>	Isola il contesto JavaScript del renderer dal contesto Node.js del preload. Requisito di sicurezza fondamentale in Electron moderno (default <code>true</code> da Electron 12).
<code>nodeIntegration:</code> <code>false</code>	Impedisce al codice React di accedere direttamente alle API Node.js (<code>fs</code> , <code>path</code> , ecc.), prevenendo attacchi XSS con escalation di privilegi.
<code>preload:</code> <code>preload.js</code>	Specifica lo script di bridge che espone le API native al renderer in modo controllato tramite <code>contextBridge</code> .

14.3.6 Dev vs Production — caricamento dell'interfaccia


```

1 const isDev = process.env.NODE_ENV === 'development';
2
3 if (isDev) {
4   win.loadURL('http://localhost:5173'); // Vite dev server
5 } else {
6   win.loadFile(path.join(__dirname, '../dist/index.html')); //
   ↪ bundle
7 }

```

Listing 84: Caricamento condizionale dev/production.**Table 33:** Differenze tra modalità development e production.

Aspetto	Development	Production
Sorgente UI	Vite dev server (localhost:5173)	Bundle compilato (dist/index.html)
Hot reload	Si (HMR Vite)	No
Build necessaria	No	Si (vite build)
DevTools	Decomentare openDevTools()	Non disponibili
Distribuzione	Solo sviluppo locale	Pacchetto installabile

La variabile `NODE_ENV` viene impostata dallo script `npm` di sviluppo ("`dev`": "`NODE_ENV=development electron .`"), rendendo il branch trasparente allo sviluppatore.

14.3.7 Gestione del menu nativo

```

1 Menu.setApplicationMenu(null);

```

Listing 85: Rimozione del menu nativo.

Il menu nativo di sistema (File, Edit, View, ecc.) viene rimosso. Questa scelta è tipica delle applicazioni *kiosk-like* o professionali con interfaccia custom che non richiedono le voci di menu standard del browser. Tutta la navigazione avviene tramite l'interfaccia React interna.

14.3.8 Handler IPC esposti al renderer

Table 34: Handler IPC di `main.js` accessibili tramite `electronAPI`.

Canale IPC	Funzione e risposta
<code>get-python-port</code>	Restituisce la costante <code>PYTHON_PORT</code> (8000). Permette al renderer di conoscere la porta del microservizio senza hardcodarla nel codice React.
<code>show-open-dialog</code>	Apri il dialog nativo di selezione file del sistema operativo con le opzioni passate dal renderer. Restituisce <code>{canceled, filePaths}</code> . Usato dal pulsante “Add Worker” per selezionare il file <code>.sql</code> di import.
<code>read-file-buffer</code>	Legge un file dal filesystem con <code>fs.readFileSync()</code> e lo restituisce come <code>{ok, data: number[]}</code> (array di byte serializzabile attraverso <code>contextBridge</code>). Permette al renderer di inviare il file SQL al microservizio Python via HTTP senza accesso diretto al filesystem.

Perché `number[]` anziché `Buffer` o `Uint8Array`

`contextBridge` serializza i dati attraverso la boundary di isolamento del contesto usando la *Structured Clone Algorithm*. `Buffer` (Node.js) e `Uint8Array` non sono sempre clonabili in modo affidabile tra contesti isolati in tutte le versioni di Electron. `Array.from(buf)` produce un `number[]` standard, garantito serializzabile. Il renderer lo ricostruisce come `Uint8Array` (o `Blob`) prima di inviarlo via `fetch` al microservizio Python.

14.4 Modulo `electron/preload.js` — Bridge sicuro

14.4.1 Ruolo e principio di funzionamento

`preload.js` è lo **script di bridge** tra il processo principale (Node.js) e il processo renderer (React). Viene eseguito in un contesto JavaScript isolato che ha accesso *sia* alle API Node.js *sia* al DOM del renderer, ma espone al renderer solo le API esplicitamente dichiarate tramite `contextBridge`.

```
1 const { contextBridge, ipcRenderer } = require('electron');
2
```

```

3 contextBridge.exposeInMainWorld('electronAPI', {
4   // Info piattaforma
5   platform: process.platform,
6
7   // Dialog file picker nativo
8   showOpenDialog: (options) =>
9     ipcRenderer.invoke('show-open-dialog', options),
10
11   // Lettura file come buffer di byte
12   readFileBuffer: (filePath) =>
13     ipcRenderer.invoke('read-file-buffer', filePath),
14 });

```

Listing 86: preload.js — esposizione sicura delle API native.

14.4.2 API esposte: window.electronAPI

Il codice React accede alle funzionalità native tramite `window.electronAPI`, che `contextBridge` rende disponibile sul global `window` del renderer:

Table 35: API disponibili in `window.electronAPI` nel renderer React.

Proprietà/Metodo	Tipo	Uso nel frontend
<code>platform</code>	<code>string</code>	Valore di <code>process.platform</code> (es. <code>'darwin'</code> , <code>'win32'</code> , <code>'linux'</code>). Usato per logica OS-specifica (es. separatori di path, stile pulsanti).
<code>showOpenDialog(options)</code>	<code>Promise<{canceled, filePaths}></code>	Apri il file picker nativo per selezionare il dataset <code>.sql</code> da importare nel pulsante “Add Worker”.
<code>readFileBuffer(filePath)</code>	<code>Promise<{ok, data}></code>	Legge il file <code>.sql</code> selezionato e lo restituisce come <code>number[]</code> per l'upload al microservizio Python.

14.4.3 Perché contextBridge è necessario

Sicurezza Electron — contextIsolation e contextBridge

Con `contextIsolation: true` (impostato in `main.js`), il codice JavaScript del renderer non condivide il contesto di esecuzione con il preload. Senza `contextBridge`, il renderer non può accedere a *nessuna* API Node.js.

`contextBridge.exposeInMainWorld('electronAPI', {...})` crea un “**tunnel**” sicuro: solo le funzioni esplicitamente elencate nell’oggetto vengono esposte, con i valori clonati (non per riferimento) attraverso la boundary di isolamento. Il renderer non può accedere a `ipcRenderer` direttamente — può solo chiamare le funzioni pre-approvate definite in `preload.js`.

Questo pattern previene che codice JavaScript malevolo caricato accidentalmente nel renderer (es. da una XSS o da una dipendenza compromessa) possa accedere al filesystem, eseguire processi o fare uso delle API Electron complete.

14.4.4 Flusso di import del dataset SQL: end-to-end

Il seguente diagramma mostra il flusso completo dell’operazione “Add Worker” (import di un dataset `.sql`) dall’interazione utente fino alla risposta del microservizio:

14.5 Ciclo di vita dell’applicazione Electron

La gestione del ciclo di vita macOS merita attenzione particolare: su macOS le applicazioni non terminano quando si chiude l’ultima finestra (il processo rimane nel Dock). L’evento `activate` permette di riaprire la finestra cliccando sull’icona nel Dock, mentre `will-quit` garantisce che il processo Python venga sempre terminato anche in questo scenario.

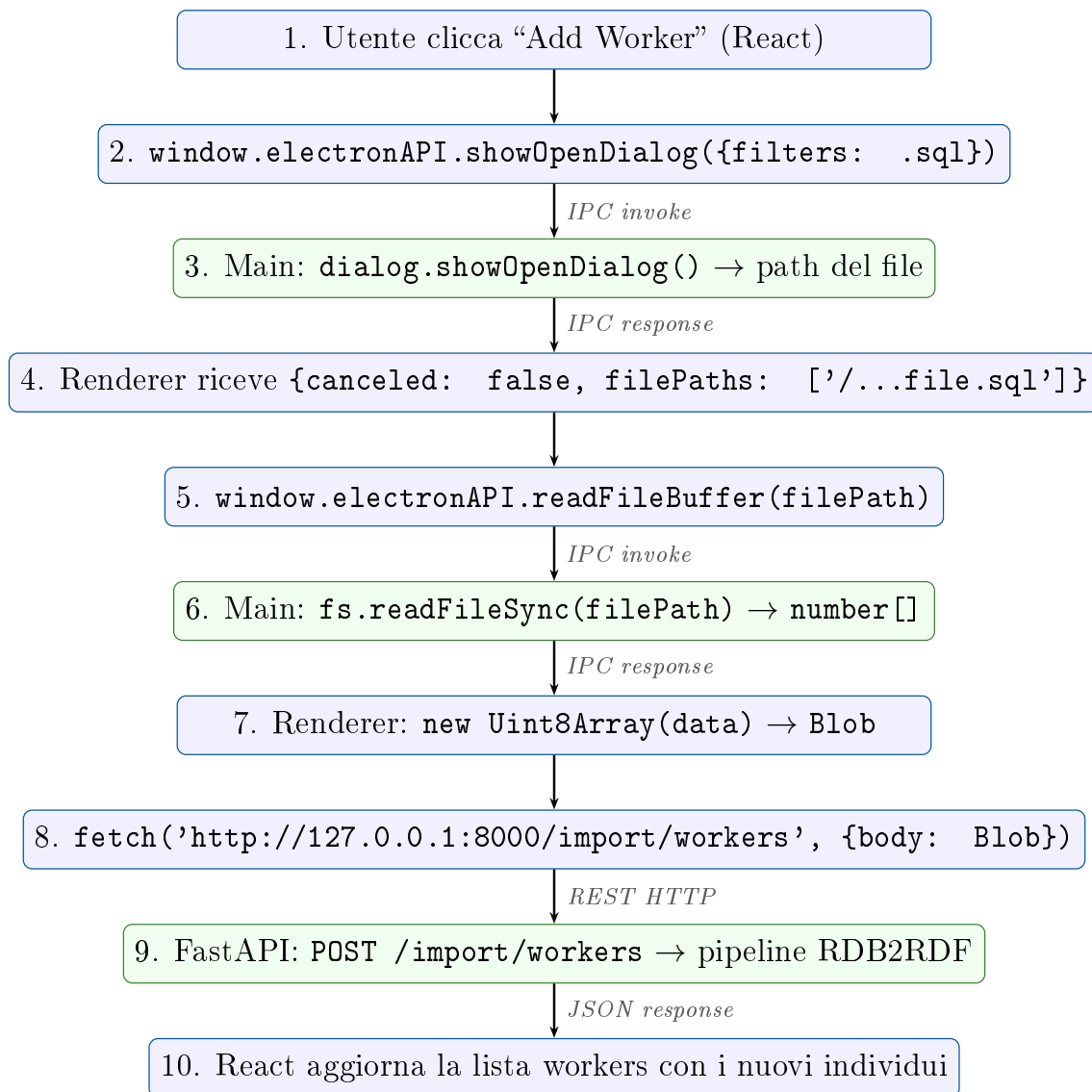


Figure 11: Flusso end-to-end dell'import dataset SQL da "Add Worker".

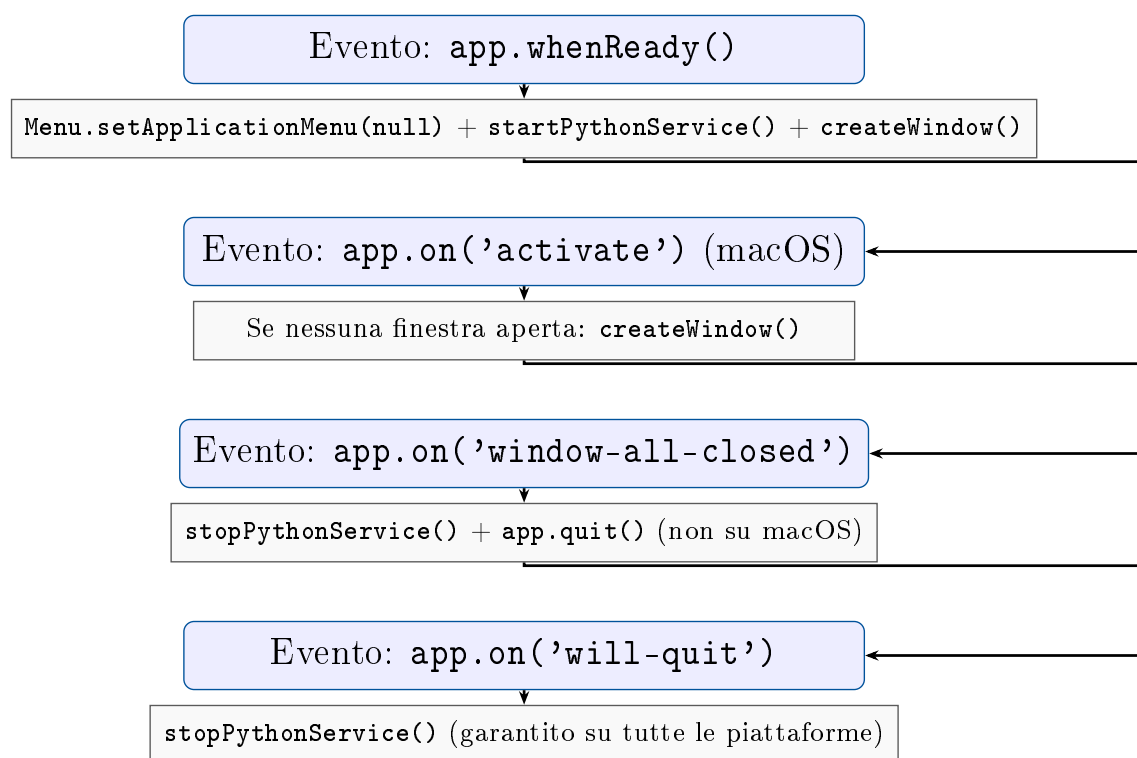


Figure 12: Ciclo di vita dell'applicazione Electron con gestione del processo Python.

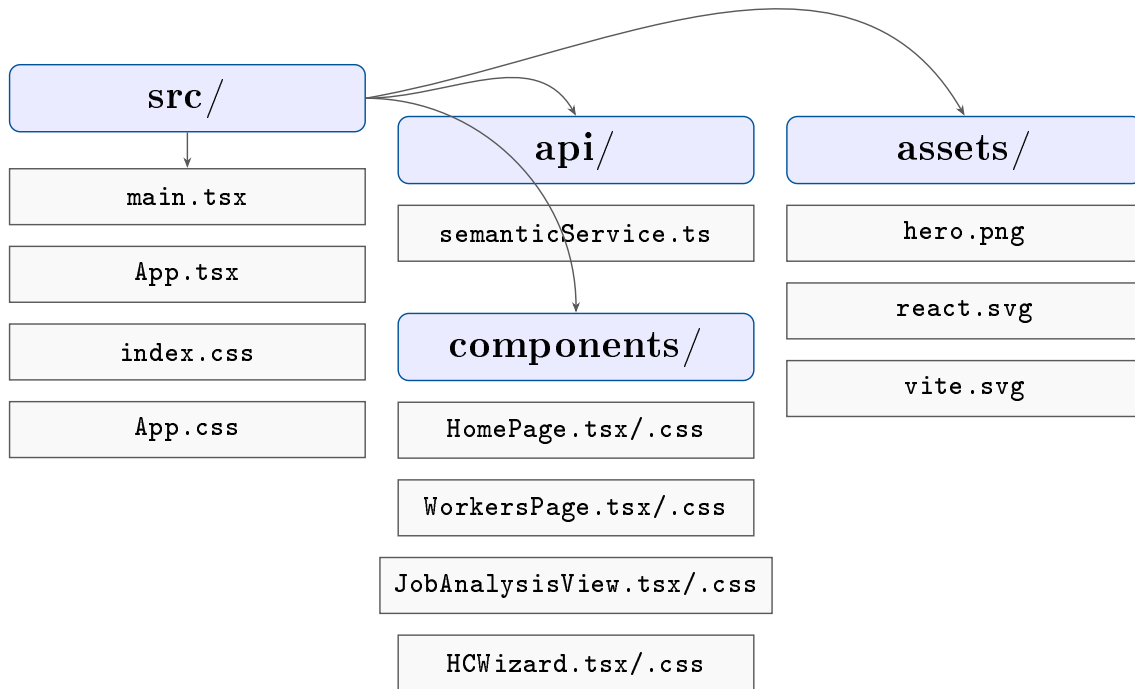
15 Layer React — Sorgenti src/

15.1 Struttura della directory src/

La directory `src/` contiene l'intera logica del frontend React/TypeScript dell'applicazione Rientr@. La struttura, visibile dall'albero del progetto, è organizzata in quattro livelli:

Table 36: Struttura della directory `src/` con contenuto.

Path	Contenuto
<code>src/main.tsx</code>	Entry point React: monta <code><App/></code> in <code>#root</code> .
<code>src/App.tsx</code>	Root component: routing client-side tra <code>HomePage</code> e <code>WorkersPage</code> , gestione dello stato di navigazione.
<code>src/index.css</code>	Reset CSS globale, font, user-select, drag prevention.
<code>src/App.css</code>	Stili legacy del template Vite (contatore, hero, next-steps). Presente ma non attivamente usato dall'UI di produzione.
<code>src/api/</code>	Layer di accesso al microservizio Python.
<code>src/api/semanticService.ts</code>	Tutte le chiamate REST al microservizio FastAPI (porta 8000).
<code>src/assets/</code>	Asset statici importati da Vite: <code>hero.png</code> , <code>react.svg</code> , <code>vite.svg</code> .
<code>src/components/</code>	Componenti React principali dell'applicazione.
<code>src/components/HomePage.tsx / .css</code>	Schermata di benvenuto con navigazione ai moduli.
<code>src/components/WorkersPage.tsx / .css</code>	Dashboard principale: lista workers, matching, dettaglio.
<code>src/components/JobAnalysisView.tsx / .css</code>	Vista analisi job: scatter plot, importanza skill, profilo.
<code>src/components/HealthConditionWizard.tsx / .css</code>	Procedura guidata per la modifica delle condizioni ICF.

Figure 13: Struttura ad albero della directory `src/`.

15.2 `src/main.tsx` — Entry point React

```

1 import { StrictMode } from 'react'
2 import { createRoot } from 'react-dom/client'
3 import './index.css'
4 import App from './App.tsx'
5
6 createRoot(document.getElementById('root')).render(
7   <StrictMode>
8     <App />
9   </StrictMode>,
10 )

```

Listing 87: `main.tsx` — montaggio dell'applicazione React.

`main.tsx` è il punto di ingresso del renderer React. Vite lo individua tramite la dichiarazione `<script type="module" src="/src/main.tsx">` in `index.html`.

15.2.1 Scelte implementative

StrictMode

Il wrapping in `React.StrictMode` attiva controlli aggiuntivi di React in fase di sviluppo:

- Rileva side effect impuri nei componenti (doppia invocazione intenzionale dei render in dev).
- Segnala l'uso di API deprecate.
- Evidenzia componenti che non gestiscono correttamente il cleanup degli effect.

In produzione (`vite build`), `StrictMode` non ha impatto sulle prestazioni: i comportamenti extra vengono rimossi dal tree-shaking.

`createRoot` e **React 18**

L'uso di `createRoot` (anziché il legacy `ReactDOM.render`) abilita il **Concurrent Mode** di React 18, che permette al motore di rendering di interrompere e riprendere i render per mantenere l'interfaccia reattiva durante operazioni costose (es. aggiornamenti lista workers, rendering scatter plot).

Asserzione non-null !

`document.getElementById('root')!` usa l'operatore TypeScript di asserzione non-null. L'elemento `#root` è garantito presente dall'`index.html` generato da Vite; il `!` evita un controllo `null` ridondante che TypeScript richiederebbe altrimenti.

Import di `index.css`

L'import del CSS globale in `main.tsx` (anziché in `App.tsx`) garantisce che il reset e il font siano applicati **prima** del render del primo componente, eliminando il flash di stile non stilizzato (FOUC).

15.3 `src/App.tsx` — Root component e routing

```
1 import { useState } from 'react'
2 import './index.css'
3 import HomePage from './components/HomePage'
4 import WorkersPage from './components/WorkersPage'
5
6 type Page = 'home' | 'workers'
7 export type NavTarget = 'workers' | 'jobs-analysis' | 'jobs-
  ↳ positions'
8
9 function App() {
10   const [page, setPage] = useState<Page>('home')
11   const [initialNav, setInitialNav] = useState<NavTarget>('workers'
  ↳ )
12
13   if (page === 'workers') {
14     return <WorkersPage
15       onNavigateHome={() => setPage('home')}
16       initialNav={initialNav}
17     />
```

```
18   }
19
20   return <HomePage onNavigate={({id}) => {
21     if (id === 'worker-information') {
22       setInitialNav('workers'); setPage('workers');
23     } else if (id === 'job-analysis') {
24       setInitialNav('jobs-analysis'); setPage('workers');
25     } else if (id === 'job-positions') {
26       setInitialNav('jobs-positions'); setPage('workers');
27     }
28   }} />
29 }
30
31 export default App
```

Listing 88: App.tsx — routing client-side e gestione navigazione.

15.3.1 Pattern di routing senza React Router

App.tsx implementa un routing client-side **custom** basato su `useState` anziché una libreria dedicata come React Router. Questa scelta è motivata da:

- **Semplicità strutturale:** l'applicazione ha solo due pagine principali (HomePage e WorkersPage). React Router introdurrebbe overhead di configurazione e dipendenze per un caso d'uso così lineare.
- **Applicazione desktop Electron:** non essendo un'app web con URL navigabili, non c'è bisogno di sincronizzare lo stato della navigazione con la URL del browser. Il concetto di “deep link” e “back button del browser” non si applica.
- **Stato di navigazione trasparente:** il parametro `initialNav` permette di aprire WorkersPage direttamente in una delle sue tre sotto-viste (`workers`, `jobs-analysis`, `jobs-positions`) senza ulteriori livelli di routing.

15.3.2 Tipi esportati

```
1 type Page = 'home' | 'workers' // privato ad App
2 export type NavTarget =
3   | 'workers'
4   | 'jobs-analysis'
5   | 'jobs-positions'
```

Listing 89: Tipi TypeScript esportati da App.tsx.

`NavTarget` è esportato (`export type`) perché viene usato come tipo del prop `initialNav` di `WorkersPage`. Definirlo in `App.tsx` (l'unico posto che lo istanzia) anziché in un file `types.ts` separato mantiene la dipendenza vicina al suo utilizzo principale, riducendo l'accoppiamento tra file.

15.3.3 Flusso di navigazione

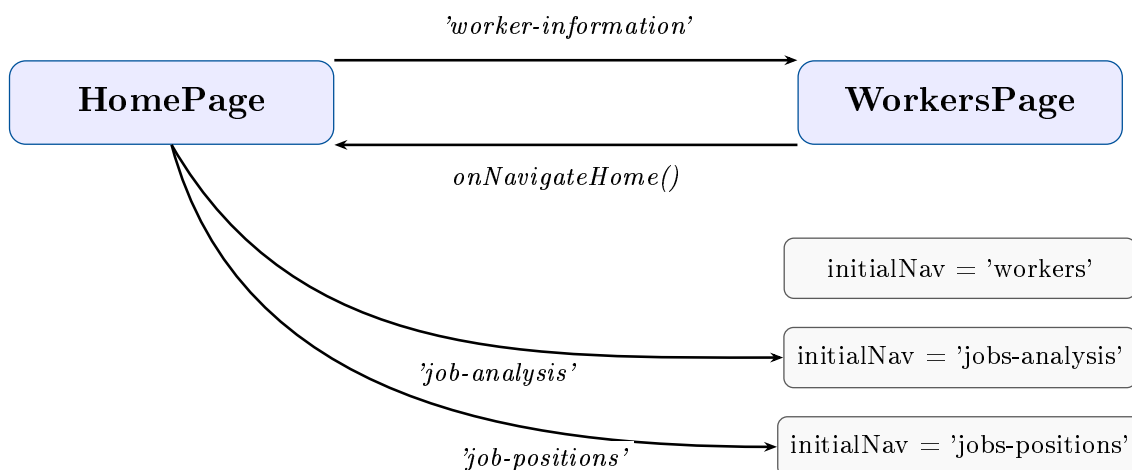


Figure 14: Flusso di navigazione gestito da `App.tsx`.

`WorkersPage` riceve `initialNav` come prop e lo usa per impostare la sotto-vista attiva al momento del montaggio. La navigazione inversa (`onNavigateHome`) imposta `page` a `'home'`, smontando `WorkersPage` e rimontando `HomePage`, con reset automatico di tutto lo stato locale (lista workers caricata, worker selezionato, cache delle query).

15.3.4 Tre punti di ingresso alla WorkersPage

I tre ID di navigazione corrispondono alle sezioni della `WorkersPage`:

Table 37: Mapping ID navigazione → vista iniziale in `WorkersPage`.

ID (da HomePage)	NavTarget	Vista aperta
'worker-information'	'workers'	Tab workers: lista pazienti, selezione, condizioni ICF.
'job-analysis'	'jobs-analysis'	Vista analisi job: scatter plot GCS/AISA, importanza skill.
'job-positions'	'jobs-positions'	Vista posizioni job: profilo O*NET, radar chart.

15.4 src/index.css — Reset e stili globali

```
1 @import url('https://fonts.googleapis.com/css2?
2   family=Inter:wght@300;400;500;600;700;800&display=swap');
3
4 *, *::before, *::after {
5   box-sizing: border-box;
6   margin: 0;
7   padding: 0;
8 }
9 html, body, #root { height: 100%; width: 100%; }
10 body {
11   font-family: 'Inter', sans-serif;
12   -webkit-font-smoothing: antialiased;
13   -webkit-user-select: none;
14   user-select: none;
15 }
16 input, textarea { -webkit-user-select: auto; user-select: auto; }
17 img { -webkit-user-drag: none; }
```

Listing 90: index.css — CSS globale dell'applicazione.

15.4.1 Scelte implementative

Font Inter via Google Fonts

Il font **Inter** (pesi 300–800) è importato via CDN Google Fonts. Inter è un typeface sans-serif ottimizzato per la leggibilità su schermo a basse risoluzioni, ideale per interfacce desktop con dense tabelle di dati come quelle di Rientr@. L'import via CDN è accettabile in un'app desktop con accesso internet garantito; per deploy completamente offline andrebbe bundlato localmente.

Box model universale

`box-sizing: border-box` su tutti gli elementi (`*`, `::before`, `::after`) garantisce che `padding` e `border` siano inclusi nelle dimensioni dichiarate, eliminando la fonte più comune di layout inconsistenti in CSS.

Full-height layout

`height: 100%` e `width: 100%` su `html`, `body` e `#root` permettono ai componenti React di usare `height: 100%` o `min-height: 100vh` per occupare l'intera area della finestra Electron senza scrollbar verticali indesiderate.

User-select disabilitato globalmente

`-webkit-user-select: none` e `user-select: none` sul `body` impediscono la selezione accidentale del testo nell'interfaccia, comportamento atteso nelle applicazioni

desktop (vs. web). Viene ri-abilitato selettivamente su `input` e `textarea`, dove la selezione del testo è funzionale.

Drag prevention sulle immagini

`-webkit-user-drag: none` su `img` impedisce il trascinamento nativo delle immagini fuori dall'applicazione, che in un contesto Electron causerebbe comportamenti imprevedibili (apertura del file nel sistema operativo).

Font smoothing

`-webkit-font-smoothing: antialiased` e `-moz-osx-font-smoothing: grayscale` attivano il rendering subpixel del font su macOS e Firefox, producendo testo più nitido. Particolarmente rilevante per Inter che è progettato per funzionare bene con l'antialiasing.

15.5 `src/App.css` — Stili legacy del template Vite

`App.css` contiene gli stili del template di default generato da `npm create vite@latest` con il preset `react-ts`. Include classi per il contatore interattivo (`.counter`), l'hero image animata (`.hero`), la sezione “next steps” (`#next-steps`) e il separatore (`#spacer`).

Questi stili **non sono attivamente usati** dall'interfaccia di produzione di Rientr@: il file è presente come eredità del bootstrap del progetto con Vite. L'import di `App.css` in `App.tsx` è stato rimosso (si vede che `App.tsx` importa solo `index.css`), rendendo di fatto le classi inutilizzate. Il file può essere eliminato in una futura pulizia del progetto senza impatto funzionale.

Nota sulle CSS Nesting Rules in `App.css`

`App.css` usa la sintassi di **CSS Nesting** nativa (es. `&:hover { ... }` dentro `.counter`) che è supportata nativamente da Vite/PostCSS senza plugin aggiuntivi nei browser moderni (Chrome 112+, Firefox 117+, Safari 16.5+). In Electron (basato su Chromium), questa sintassi è garantita funzionante dalla versione di Chromium bundlata.

15.6 `src/components/HomePage.tsx` e `HomePage.css`

15.6.1 Struttura del file e responsabilità

`HomePage.tsx` (133 righe) è il componente di landing dell'applicazione. Importa esclusivamente il proprio CSS (`HomePage.css`) e non dipende da nessun hook React di stato, né da API esterne: è un componente **puramente presentazionale**.

La sua interfaccia TypeScript è minimale per design:

```
1 interface HomePageProps {
2   onNavigate?: (cardId: string) => void;
3 }
```

`onNavigate` è opzionale (?): il componente può renderizzarsi in isolamento per test o Storybook senza prop obbligatorie. Il callback riceve l'id della card cliccata come stringa e lo propaga a `App.tsx` che esegue il routing.

15.6.2 Il tipo NavCard e l'array cards

```
1 interface NavCard {
2   id:          string;
3   icon:        React.ReactNode;
4   title:       string;
5   description: string[];
6 }
7
8 const cards: NavCard[] = [
9   { id: 'worker-information', icon: <WorkerIcon />, title: 'Worker
10    ↳ Information',
11     description: ['Add new worker,', 'Check current health
12    ↳ conditions,',
13                  'Modify health conditions'] },
14   { id: 'job-analysis', icon: <JobAnalysisIcon />, title: 'Job
15    ↳ Analysis',
16     description: ['Check and compare job opportunities for workers,
17    ↳ ',
18                  'List possible work improving aids'] },
19   { id: 'job-positions', icon: <JobPositionsIcon />, title: 'Job
20    ↳ Positions',
21     description: ['Check jobs, Import new jobs'] },
22 ];
```

L'array `cards` è dichiarato **a livello di modulo**, fuori dal corpo del componente. Questa scelta è intenzionale: in React ogni variabile dichiarata dentro un componente viene ricreata ad ogni render. Poiché ogni elemento `icon` è un `React.ReactNode` (JSX), il runtime React creerebbe nuovi oggetti a ogni render senza alcuna utilità. Dichiararli fuori dalla funzione li rende costanti condivise nell'intera vita del modulo.

Il tipo `description: string[]` anziché una singola stringa permette di spezzare ogni card in più righe di testo senza markup HTML inline, rendendo il contenuto modificabile direttamente nell'array senza toccare il template JSX.

15.6.3 Icone SVG inline — componenti funzionali

Le tre icone (`WorkerIcon`, `JobAnalysisIcon`, `JobPositionsIcon`) sono componenti funzionali React che restituiscono SVG puro. La scelta di SVG inline rispetto a immagini `` è motivata da quattro ragioni:

1. **Nessuna richiesta HTTP:** in un contesto Electron con possibile modalità offline, eliminare fetch di asset è preferibile. Vite bundla le importazioni statiche, ma il pattern `mask-image: url(/file.png)` usato altrove nel progetto richiede comunque che il file esista in `/public`. SVG inline non ha questa dipendenza.
2. **Renderizzazione nitida a qualsiasi DPI:** il `viewBox` dell'SVG garantisce scaling vettoriale perfetto su display Retina e HiDPI senza pixelazione, a differenza dei PNG.
3. **Colorabilità via props:** le forme usano colori hardcoded semanticamente significativi (`#4DD9C0` teal principale, `white`, `#60C8B0` variante) che possono essere parametrizzati tramite prop in futuro senza modificare file esterni.
4. **Composizione semantica:** ogni icona è costruita da primitive SVG (`circle`, `path`, `rect`, `line`) che documentano visivamente la struttura (es. `WorkerIcon`: due cerchi (persone) + `rect` (tessera ID); `JobAnalysisIcon`: persona + lavagna con chart; `JobPositionsIcon`: persona + valigetta con badge "+").

15.6.4 Layout e struttura JSX

Il componente è strutturato in tre aree semantiche HTML:

```

1 <div className="home-page" >                {/* wrapper radice */}
2   {/* 1. Background animato */}
3   <div className="bg-blob bg-blob-1" />
4   <div className="bg-blob bg-blob-2" />
5   <div className="bg-blob bg-blob-3" />
6
7   {/* 2. Header brand */}
8   <header className="home-header">
9     <div className="brand">
10      
11      <div className="brand-text">
12        <span className="brand-title">RIENTR@ returns</span>
13        <span className="brand-subtitle">Decision Support System</
14        ↪ span>
15      </div>
16    </div>

```

```

16   </header>
17
18   {/* 3. Grid delle card */}
19   <main className="home-main">
20     <div className="cards-grid">
21       {cards.map((card) => (
22         <button key={card.id} className="nav-card"
23           id={card.id} aria-label={card.title}
24           onClick={() => onNavigate?.(card.id)}>
25         <div className="card-icon-wrapper">{card.icon}</div>
26         <div className="card-body">
27           <h2 className="card-title">{card.title}</h2>
28           <div className="card-divider" />
29           <ul className="card-description">
30             {card.description.map((line, i) => <li key={i}>{line
31               ↪ }</li>)}
32           </ul>
33         </div>
34         <div className="card-hover-shine" />
35       </button>
36     )]}
37   </div>
38
39   {/* 4. Footer istituzionale */}
40   <footer className="home-footer">
41     <div className="footer-logos">
42       
44       
46     </div>
47   </footer>
48 </div>

```

Card come <button> HTML

Le card sono <button> e non <div> cliccabili. Questo garantisce accessibilità nativa senza attributi aggiuntivi: focalizzazione con **Tab**, attivazione con **Enter/Space**, **role="button"** implicito letto dagli screen reader. L'**aria-label** trasmette il nome della destinazione (es. “*Worker Information*”) che è più informativo del contenuto testuale della card.

Optional chaining su onNavigate?.()

L’invocazione `onNavigate?.(card.id)` usa l’optional chaining di TypeScript: se la prop non viene passata (rendering isolato), la chiamata viene saltata silenziosamente

senza errore runtime.

Footer: filtro CSS per loghi monocromatici

I loghi STIMA-CNR e INAIL vengono resi bianchi tramite `filter: brightness(0) invert(1)` CSS, indipendentemente dal colore originale del file PNG. Questo evita di dover mantenere versioni bianche separate degli asset.

15.6.5 HomePage.css — sistema visivo

Sfondo gradient fisso

```
1 .home-page {  
2   background: linear-gradient(135deg,  
3     #1a2a4a 0%, #1e3a5f 25%, #1a4a5a 50%,  
4     #163860 75%, #102840 100%);  
5 }
```

Il gradiente diagonale (135deg) usa quattro stop di blu-petrolio che definiscono la palette cromatica dell'intera applicazione. Lo stesso sfondo di base (#1a2a4a) è usato come `backgroundColor` in `BrowserWindow` di Electron, evitando il “flash bianco” durante il caricamento iniziale del renderer.

Blob animati

```
1 .bg-blob { filter: blur(80px); opacity: 0.35;  
2   animation: blobDrift 12s ease-in-out infinite alternate; }  
3  
4 @keyframes blobDrift {  
5   0% { transform: translate(0,0) scale(1); }  
6   33% { transform: translate(30px,-20px) scale(1.05); }  
7   66% { transform: translate(-20px,30px) scale(0.95); }  
8   100% { transform: translate(20px,20px) scale(1.02); }  
9 }
```

Tre ellissi radiali con colori diversi (teal #4a90b8, viola #6a5acd, verde-acqua #20b2aa) vengono sfocate a 80px e animate con `blobDrift`. Le tre istanze hanno durate diverse (14s, 10s, 16s) e `animation-delay` sfasati (-4s, -8s), rendendo il movimento non ciclico e “organico”. `infinite alternate` inverte la direzione a ogni ciclo evitando i salti visivi che si avrebbero con `infinite normal`.

Effetto frosted glass sulle card

```
1 .nav-card {  
2   background: rgba(255,255,255,0.08);
```

```

3   border: 1px solid rgba(255,255,255,0.2);
4   backdrop-filter: blur(10px);
5   -webkit-backdrop-filter: blur(10px);
6   min-height: 420px;
7 }

```

`backdrop-filter: blur(10px)` richiede il prefisso `-webkit-` per le versioni di Chromium bundlate in Electron precedenti alla 76. L'effetto è visibile dove le card si sovrappongono ai blob colorati retrostanti: il vetro smerigliato sfuma il colore del blob mantenendo leggibilità del testo sovrapposto.

Hover multi-layer con spring curve

```

1 .nav-card {
2   transition: transform 0.25s cubic-bezier(0.34,1.56,0.64,1),
3               box-shadow 0.25s ease, background 0.25s ease,
4               border-color 0.25s ease;
5 }
6 .nav-card:hover {
7   transform: translateY(-6px) scale(1.02);
8   background: rgba(255,255,255,0.14);
9   border-color: rgba(255,255,255,0.4);
10  box-shadow:
11    0 12px 40px rgba(0,0,0,0.35),
12    0 0 0 1px rgba(255,255,255,0.15),
13    inset 0 1px 0 rgba(255,255,255,0.2);
14 }
15 .card-hover-shine { opacity: 0; transition: opacity 0.3s; }
16 .nav-card:hover .card-hover-shine { opacity: 1; }

```

La curva `cubic-bezier(0.34, 1.56, 0.64, 1)` è una *spring curve* con overshooting (il secondo valore > 1): la card sale leggermente oltre la posizione finale prima di assestarsi, simulando un molla. Il `box-shadow` usa tre livelli: ombra principale sotto la card, outline esterno sottile, evidenziazione interna sul bordo superiore (luce simulata dall'alto). Il `card-hover-shine` è un div `absolute` con gradiente diagonale che simula un riflesso di luce superficiale.

15.7 src/components/WorkersPage.tsx e WorkersPage.css

15.7.1 Ruolo, dimensioni e dipendenze

`WorkersPage.tsx` (1243 righe) è il componente più complesso del frontend. Dipende da:

- Hook React: `useState` (15 istanze), `useEffect` (6), `useCallback` (1), `useRef` (2).
- API layer: 6 funzioni da `semanticService.ts` (`fetchStatus`, `fetchWorkers`, `fetchHealthConditions`, `fetchCoreSets`, `selectWorker`, `importWorkers`).
- Componenti figlio: `JobAnalysisView` e `HealthConditionWizard`.
- API nativa Electron: `window.electronAPI` (`showOpenDialog`, `readFileBuffer`).

15.7.2 Icone SVG inline del componente

`WorkersPage` definisce 10 componenti icona inline: `SearchIcon`, `ListIcon`, `PlusIcon`, `EditIcon`, `DotsIcon`, `PdfIcon`, `RefreshIcon`, `FilterIcon`, `ArrowRightIcon`, `XIcon`. Le icone SVG seguono lo stesso pattern di `HomePage`: stroke-based su `currentColor`, dimensioni fisse in pixel, zero dipendenze da librerie esterne.

`ArchiveIcon` adotta invece il pattern CSS `maskImage`:

```

1 const ArchiveIcon = () => (
2   <span style={{
3     backgroundColor: 'currentColor',
4     maskImage: 'url(/Archive.png)',
5     WebkitMaskImage: 'url(/Archive.png)',
6     maskSize: 'contain', maskRepeat: 'no-repeat',
7     maskPosition: 'center', width: 16, height: 16,
8   }} />
9 );

```

Il quadrato colorato con `backgroundColor: 'currentColor'` viene ritagliato dalla forma del PNG di archivio tramite CSS `mask`. Questo permette di ricolorare l'icona via CSS `color` ereditato dal genitore. Il pattern è usato anche per le icone `empty-state` (`Vector.png`, `User_scan_fill.png`) con dimensioni maggiori (64px).

15.7.3 Parser `parseConditionDescription`

```

1 function parseConditionDescription(description: string):
2   ↳ ParsedDescription {
3     const normalized = (description || '').replace(/\r\n/g, '\n').
4     ↳ trim();
5     const inclusionMatch = normalized.match(/\bInclusions?:/i);
6     const exclusionMatch = normalized.match(/\bExclusions?:/i);
7
8     const firstMarkerIndex = [inclusionMatch?.index, exclusionMatch?.
9     ↳ index]
10    .filter((index): index is number => index != null)

```

```

8      .sort((a, b) => a - b)[0];
9
10     const main = (firstMarkerIndex == null
11       ? normalized
12       : normalized.slice(0, firstMarkerIndex)).trim();
13
14     let inclusion = '';
15     if (inclusionMatch?.index != null) {
16       const start = inclusionMatch.index + inclusionMatch[0].length;
17       const end = exclusionMatch?.index != null && exclusionMatch.
18         ↪ index > inclusionMatch.index
19         ? exclusionMatch.index : normalized.length;
20       inclusion = normalized.slice(start, end).trim();
21     }
22     // ...
  
```

Le descrizioni ICF nell'ontologia seguono un formato testuale con sezioni opzionali. La normalizzazione `replace(/\\r\\n/g, '\\n')` è necessaria perché il testo proveniente dall'ontologia RDF può usare terminatori di riga Windows (CR+LF) a seconda del sistema che ha generato il file. La regex `/\\bInclusions?:/i` usa:

- `\\b`: word boundary, evita match parziali dentro altre parole.
- `s?`: gestisce sia “Inclusion” che “Inclusions”.
- `i`: case-insensitive.

La logica di slicing gestisce tutti i casi limite: marker assenti, marker in ordine inverso (exclusion prima di inclusion), marker consecutivi senza testo intermedio. Il tipo `ParsedDescription` ha campi stringa sempre definiti (mai `undefined`), semplificando il rendering JSX a valle.

Duplicazione in HealthConditionWizard

`parseConditionDescription` è definita identicamente sia in `WorkersPage.tsx` che in `HealthConditionWizard.tsx`. Questa duplicazione è un refactoring pendente: la funzione andrebbe estratta in un modulo utility (es. `src/utills/icfParser.ts`) e importata da entrambi i componenti.

15.7.4 Componente LoadingScreen — schermata di warm-up

`LoadingScreen` è un sottocomponente interno definito nel file ma fuori dal corpo di `WorkersPage`. Ha il proprio stato locale (`dots`, `elapsed`) e due `useEffect` con `interval`:

```

1 // Animazione puntini
  
```

```

2 useEffect(() => {
3   const t = setInterval(
4     () => setDots(d => d.length >= 3 ? '' : d + ' '), 500);
5   return () => clearInterval(t);
6 }, []);
7 // Timer elapsed in secondi
8 useEffect(() => {
9   const t = setInterval(() => setElapsed(e => e + 1), 1000);
10  return () => clearInterval(t);
11 }, []);

```

L'animazione dei puntini usa l'updater funzionale (`setDots(d => ...)`) per evitare la cattura stale nella closure di `setInterval`. Il componente renderizza tre viste condizionali:

1. **Error**: icona  + messaggio + pulsante Retry.
2. **Loading** (nessuno status): "Connecting to semantic service..."
3. **Loading** (status presente): statistiche ontologia + progress bar + timer.

Separare `LoadingScreen` in un sottocomponente garantisce che i `setInterval` vengano cancellati automaticamente quando `WorkersPage` imposta `isReady = true`.

15.7.5 Inventario dello stato di `WorkersPage`

15.7.6 Strategia di polling adattivo

```

1 const poll = useCallback(async () => {
2   try {
3     const s = await fetchStatus();
4     setServiceStatus(s);
5     setServiceError(null);
6     if (s.status === 'ready') { setIsReady(true); return; }
7     if (s.status === 'error') { setServiceError(s.message); return; }
8     ↪ }
9     pollRef.current = setTimeout(poll, 2500); // Pellet in corso
10  } catch {
11    pollRef.current = setTimeout(poll, 1500); // servizio non
12    ↪ raggiungibile
13  }
14 }, []);
15
16 useEffect(() => {
17   poll();
18   return () => { if (pollRef.current) clearTimeout(pollRef.current)
19     ↪ ; };

```

Table 38: Tutte le variabili di stato di `WorkersPage` con tipo e scopo.

Variabile	Tipo	Scopo
<code>serviceStatus</code>	<code>ServiceStatus null</code>	Risposta corrente di <code>GET /status</code>
<code>serviceError</code>	<code>string null</code>	Errore di connessione al microservizio
<code>isReady</code>	<code>boolean</code>	Pellet completato, UI operativa
<code>pollRef</code>	<code>useRef<Timeout></code>	Handle del timeout del polling
<code>workers</code>	<code>Worker[]</code>	Lista completa workers dall'ontologia
<code>loadingWorkers</code>	<code>boolean</code>	Spinner sidebar durante fetch
<code>selectedWorker</code>	<code>Worker null</code>	Worker correntemente visualizzato
<code>archivedWorkerIds</code>	<code>Set<string></code>	IDs archiviati (localStorage)
<code>conditions</code>	<code>HealthCondition[]</code>	Condizioni ICF del worker selezionato
<code>loadingConditions</code>	<code>boolean</code>	Spinner tabella condizioni
<code>searchQuery</code>	<code>string</code>	Testo ricerca nella sidebar workers
<code>conditionSearchQuery</code>	<code>string</code>	Testo ricerca nella tabella ICF
<code>dotsMenuOpen</code>	<code>boolean</code>	Menu overflow aperto/chiuso
<code>coreSetFilterOpen</code>	<code>boolean</code>	Dropdown filtro core set
<code>selectedCoreSets</code>	<code>string[]</code>	Core set selezionati per filtro
<code>allCoreSets</code>	<code>string[]</code>	Tutti i core set dall'API
<code>expandedConditionCode</code>	<code>string null</code>	Riga ICF espansa nella tabella
<code>activeTab</code>	<code>'active' 'archived'</code>	Tab attiva nella sidebar
<code>isSearchFocused</code>	<code>boolean</code>	Focus sulla search box (mostra freccia)
<code>switchingWorkerId</code>	<code>string null</code>	Worker in attesa di selezione API
<code>isWizardOpen</code>	<code>boolean</code>	Wizard HC visibile
<code>healthWizardStep</code>	<code>Step</code>	Step corrente della wizard
<code>isSidebarOpen</code>	<code>boolean</code>	Sidebar collassata o visibile
<code>importPhase</code>	<code>ImportPhase</code>	Fase del flusso di import
<code>importResult</code>	<code>ImportWorkersResult null</code>	Risultato ultimo import
<code>importError</code>	<code>string null</code>	Errore ultimo import
<code>importModalOpen</code>	<code>boolean</code>	Modale import visibile
<code>sortRules</code>	<code>SortRule[]</code>	Regole di sort attive sulla tabella

```
17 }, [poll]);
```

Il polling usa `setTimeout` ricorsivo invece di `setInterval` per due ragioni fondamentali:

1. **Intervallo adattivo:** 1500ms quando il servizio non risponde ancora (avvio Electron), 2500ms quando il servizio risponde ma Pellet è in corso. Riduce l'attesa percepita nella fase critica di avvio.
2. **Nessun overlap:** il tick successivo viene schedulato solo dopo la risposta del precedente. Con `setInterval`, se la risposta tardasse più dell'intervallo, le chiamate si accumulerebbero. Con `setTimeout` ricorsivo ci è sempre al massimo una richiesta in volo.

`useCallback` con array vuoto garantisce che `poll` abbia un'identità stabile tra render (necessario perché è nella dependency array del `useEffect`). Il `useRef` (`pollRef`) conserva il riferimento al timeout per il cleanup.

15.7.7 Selezione del worker — sequenza critica

```
1 const handleSelectWorker = async (w: Worker) => {
2   if (w.id === selectedWorker?.id || switchingWorkerId) return;
3   setSwitchingWorkerId(w.id);      // mostra "switching..." nella
   ↪ sidebar
4   try {
5     await selectWorker(w.id);      // POST /workers/select
6     // Aggiornamento ottimistico: flip is_selected nella lista
   ↪ locale
7     setWorkers(prev =>
8       prev.map(x => ({ ...x, is_selected: x.id === w.id })));
9     setSelectedWorker({ ...w, is_selected: true });
10  } catch {
11    // Fallback: mostra il worker anche se l'API ha fallito
12    // (tabella ICF funziona; Job Analysis mostrerà il proprio
   ↪ errore)
13    setSelectedWorker(w);
14  } finally {
15    setSwitchingWorkerId(null);
16  }
17 };
```

La guardia `if (w.id === selectedWorker?.id || switchingWorkerId)` previene doppi clic e click paralleli su worker diversi mentre la chiamata è in volo. `switchingWorkerId` viene impostato prima della `await`: questo garantisce che il render intermedio mostri l'indicatore di switching nella lista prima della risposta.

Il **perché attendere l'API** prima di impostare `selectedWorker`: `JobAnalysisView` viene montato subito dopo e lancia `GET /match/{workerId}` al mount. Se il flag `isSelected` nell'ontologia non fosse ancora aggiornato, la query `SPARQL FILTER(?selected = true)` restituirebbe i risultati del worker precedente.

15.7.8 Sistema di ordinamento multi-colonna

```

1 type SortCol = 'icf_code' | 'name' | 'eff_qualifier';
2 type SortDir = 'asc' | 'desc' | 'b-first' | 'd-first';
3 type SortRule = { col: SortCol; dir: SortDir };
4 const [sortRules, setSortRules] = useState<SortRule[]>([]);
5
6 const handleSort = (col: SortCol) => {
7   setSortRules(prev => {
8     const cycle = col === 'icf_code'
9       ? ['b-first', 'd-first', null] as const
10       : ['asc', 'desc', null] as const;
11     const existing = prev.find(rule => rule.col === col);
12     const currentIndex = existing ? cycle.indexOf(existing.dir as
13     ↳ any) : -1;
14     const next = cycle[(currentIndex + 1) % cycle.length];
15     if (next === null) return prev.filter(rule => rule.col !== col)
16     ↳ ;
17     return [{ col, dir: next },
18       ...prev.filter(rule => rule.col !== col)];
19   });
20 };

```

L'ordinamento implementa un sistema tri-state per colonna (ciclo di tre stati con rimozione automatica) e multi-colonna stabile (array di regole ordinate per priorità):

- **icf_code**: ciclo `b-first` → `d-first` → rimosso. Porta in cima i codici con prefisso `b` (Body Functions) o `d` (Activities). L'ordinamento su `icf_code` attiva anche un **filtro**: `filteredConditions` esclude le righe che non corrispondono al prefisso attivo, non solo le riordina.
- **name** e **eff_qualifier**: ciclo `asc` → `desc` → rimosso. Il qualifier effettivo (`eff`) è `Math.max(bf_qualifier, ap1_qualifier)`: prende il massimo tra i due qualificatori ICF disponibili.

Quando una colonna viene (ri)cliccata, viene portata in testa all'array (`[ col, dir: next , ...prev.filter(...)]`), scalando le colonne precedenti di priorità. Il badge numerico nel `sortIcon` (`b1, d2, asc3`) mostra sia la direzione che la posizione gerarchica.

15.7.9 Tabella ICF — rendering a doppia riga

La tabella delle condizioni ICF usa un pattern non standard di React: ogni riga espandibile genera **un array di due <tr>**:

```

1 {filteredConditions.map((c, i) => {
2   const isExpanded = expandedConditionCode === c.icf_code;
3   return [
4     <tr key={` ${c.icf_code}-${i}`}
5       className={`wp-condition-row${isExpanded ? ' is-expanded' : ' '
6         ↪ '}`}
7       onClick={() => setExpandedConditionCode(
8         prev => prev === c.icf_code ? null : c.icf_code)}>
9       /* Colonne: ICF Code, Name, Core Set, Qualifier */
10    </tr>,
11    isExpanded ? (
12      <tr key={` ${c.icf_code}-${i}-detail`}
13        className="wp-condition-detail-row">
14        <td />{/* cella vuota per allineamento */}
15        <td colSpan={1}>
16          /* Descrizione parsata: main, inclusion, exclusion */
17        </td>
18        <td /><td />
19      </tr>
20    ) : null,
21  ];
22 })}

```

`Array.map` può restituire array annidati che React appiattisce automaticamente nel DOM (da React 16). Il toggle usa l’updater funzionale del toggle-accordion: `prev === c.icf_code ? null : c.icf_code` implementa il collasso della riga precedentemente espansa quando se ne apre una nuova.

La cella `<td />` vuota nella prima colonna della detail-row è intenzionale per mantenere l’allineamento visivo dell’indicatore di espansione. Il `colSpan=1` sulla seconda cella limita la descrizione alla colonna “Code Name”, lasciando Core Set e Qualifier vuoti nella riga di dettaglio.

15.7.10 Il qualificatore effettivo

```

1 const eff = Math.max(c.bf_qualifier ?? 0, c.ap1_qualifier ?? 0);

```

Ogni codice ICF può avere `BFqual` (Body Function qualifier) o `AP1qual` (Activities qualifier) o entrambi. La colonna “Qualifier” mostra il *massimo* dei due: la gravità effettiva. Il `?? 0` sostituisce `null/undefined` con 0 prima di `Math.max`, evitando che NaN si propaghi. Questo valore è anche usato per applicare la classe

CSS `wp-qualifier-badge-eff wp-eff-{0..4}` che colora il badge.

15.7.11 Gestione import — flusso completo con stati

```

1 type ImportPhase = 'idle' | 'picking' | 'uploading' | 'done' | '
  ↳ error';
2
3 const handleAddWorker = async () => {
4   // 1. Verifica disponibilit  Electron API
5   const electronAPI = (window as any).electronAPI;
6   if (!electronAPI?.showOpenDialog) {
7     setImportError('File dialog not available (Electron API missing
  ↳ ).');
8     setImportModalOpen(true); return;
9   }
10  // 2. Apre file dialog nativo
11  setImportPhase('picking'); setImportModalOpen(true);
12  const dialogResult = await electronAPI.showOpenDialog({
13    title: 'Select SQL Dataset',
14    filters: [{ name: 'SQL Files', extensions: ['sql'] }],
15    properties: ['openFile'],
16  });
17  if (dialogResult.canceled) {
18    setImportPhase('idle'); setImportModalOpen(false); return;
19  }
20  // 3. Legge il file via IPC
21  setImportPhase('uploading');
22  const readResult = await electronAPI.readFileBuffer(filePath);
23  if (!readResult.ok) {
24    setImportError('Cannot read file: ${readResult.error}');
25    setImportPhase('error'); return;
26  }
27  // 4. Invia al microservizio
28  const result = await importWorkers(readResult.data as number[],
  ↳ fileName);
29  setImportResult(result);
30  setImportPhase('done');
31  // 5. Ricarica la lista workers se sono stati aggiunti nuovi
32  if (result.persons_added > 0)
33    fetchWorkers().then(setWorkers).catch(console.error);
34 };

```

Il type `ImportPhase` modella la macchina a stati del flusso di import. Ogni fase corrisponde a una vista diversa della modale: `picking` mostra “Opening file dialog...”, `uploading` mostra uno spinner con il messaggio della pipeline R2RML, `done` mostra le statistiche dettagliate (persone aggiunte, skipped, ICF validi, job validi), `error`

mostra il messaggio di errore in rosso.

`(window as any).electronAPI` usa il cast TypeScript esplicito perché `electronAPI` non è dichiarato nel tipo globale `Window`. Il `guard !electronAPI?.showOpenDialog` gestisce il caso in cui il componente venga eseguito fuori da Electron (es. browser web durante sviluppo): in tal caso mostra un errore descrittivo anziché crashare.

15.7.12 Navigazione a tre modalità e sidebar collassabile

La `WorkersPage` renderizza condizionalmente in base a `activeNav` (fissato al mount da `initialNav`):

```

1 {activeNav === 'jobs-analysis' ? (
2   <div className="wp-section">
3     <JobAnalysisView
4       workerId={selectedWorker.id}
5       workerDisplayName={...}
6     />
7   </div>
8 ) : (
9   /* Worker nav: header worker + tabella ICF o Wizard */
10  <>
11    {!isWizardOpen && <div className="wp-worker-header">...</div>}
12    {isWizardOpen
13      ? <HealthConditionWizard ... />
14      : <div className="wp-section">...</div>
15    }
16  </>
17 )}

```

`activeNav` viene letto con `useState(initialNav)` ma **mai aggiornato**: funge da costante di configurazione al mount. La scelta di non renderizzare tab di navigazione tra le modalità è intenzionale: la navigazione avviene tornando alla `HomePage` e rifelezionando una card diversa.

La sidebar collassabile (`isSidebarOpen`) viene chiusa tramite il button con `ListIcon` nell'header della sidebar e riaperta tramite un button in navbar che appare solo quando la sidebar è chiusa. La classe CSS `wp-sidebar-closed` nasconde la sidebar con una transizione CSS.

15.8 src/components/JobAnalysisView.tsx

e `JobAnalysisView.css`

15.8.1 Ruolo, dimensioni e dipendenze

`JobAnalysisView.tsx` (924 righe) implementa la dashboard di analisi del matching semantico. Dipende da:

- Hook React: `useState` (15), `useEffect` (6), `useMemo` (5), `useRef` (3), `useCallback` (2).
- API layer: `fetchMatchResults`, `fetchSkillDetail`, `fetchAllJobs`, `updateWorkerJobs`.
- Browser API: `ResizeObserver`.

Il componente è ricevuto da `WorkersPage` con due soli props:

```
1 interface JobAnalysisViewProps {
2   workerId: string;
3   workerDisplayName: string;
4 }
```

Tutta la gestione dello stato è interna, inclusa la cache di sessione per-worker.

15.8.2 Costanti e mappe di critica

```
1 const CRIT_COLOR: Record<string, string> = {
2   'not critical': 'rgba(255,255,255,0.4)',
3   'SLIGHTLY CRITICAL': '#9ab87a',
4   'MODERATELY CRITICAL': '#c4a83a',
5   'RELEVANTLY CRITICAL': '#c47e45',
6   'EXTREMELY CRITICAL': '#c04040',
7 };
8 const CRIT_RANK: Record<string, number> = {
9   'not critical': 0, 'SLIGHTLY CRITICAL': 1,
10  'MODERATELY CRITICAL': 2, 'RELEVANTLY CRITICAL': 3,
11  'EXTREMELY CRITICAL': 4,
12 };
13 const ANCHOR_ICON = ['---', 'up', 'upup', 'upupup'];
```

`CRIT_COLOR` mappa ogni etichetta di criticità a un colore hex/rgba. I colori seguono una scala dal bianco (non critico) al rosso (estremamente critico), passando per verde-giallo-arancione. Sono usati per colorare sia il testo delle celle (`color: col`) che le barre progress (`background: col`), mantenendo la coerenza visiva tra le due rappresentazioni.

`CRIT_RANK` assegna un ordine numerico alle etichette, necessario per l'ordinamento della colonna "Criticality" che non è ordinabile alfabeticamente (le etichette non seguono l'ordine lessicale). Usato in `doSort` quando `col === 'criticality_label'`.

ANCHOR_ICON mappa gli anchor [0–3] a simboli Unicode di frecce (anziché valori numerici) per rendere immediatamente visivo il livello di importanza della skill nella tabella.

15.8.3 Componente ScatterPlot — SVG responsive con ResizeObserver

```

1  const PAD = { l: 62, r: 24, t: 20, b: 50 };
2
3  function ScatterPlot({ matchResults, jobA, jobB, onSelect }) {
4    const wrapRef = useRef<HTMLDivElement>(null);
5    const [size, setSize] = useState({ w: 600, h: 300 });
6    const [tooltip, setTooltip] = useState<{
7      dataX: number; dataY: number; result: MatchResult
8    } | null>(null);
9
10   useEffect(() => {
11     const el = wrapRef.current;
12     const ro = new ResizeObserver(entries => {
13       const { width, height } = entries[0].contentRect;
14       if (width > 10 && height > 10)
15         setSize({ w: Math.round(width), h: Math.round(height) });
16     });
17     ro.observe(el!);
18     return () => ro.disconnect();
19   }, []);
20
21   const toX = (a: number) => PAD.l + (a / X_MAX) * (w - PAD.l - PAD
22     ↪ .r);
23   const toY = (g: number) => PAD.t + PH - (g / Y_MAX) * PH;

```

PAD è dichiarato fuori dal componente come costante di modulo: i valori di padding in SVG user-units sono fissi e non dipendono dal tema o da props. Il padding sinistro (62) è maggiore per ospitare le etichette dell'asse Y.

Le funzioni `toX` e `toY` convertono valori nei domain dei dati (AISA% e GCS%) in coordinate SVG. La Y è invertita ($PH - (g / Y_MAX) * PH$) perché in SVG l'asse Y cresce verso il basso, mentre sul grafico GCS% cresce verso l'alto.

Il tooltip viene mostrato sugli eventi `onMouseEnter` dei dot SVG e contiene le coordinate dati (`dataX`, `dataY`) per poter calcolare la posizione del tooltip SVG indipendentemente dalla posizione del DOM.

15.8.4 Assi dinamici: `axisMax` e `makeTicks`

```

1  function axisMax(v: number, step: number, min: number): number {
2    return Math.max(min, Math.ceil(v / step) * step);
3  }

```

```

4 const X_MAX = useMemo(() => {
5   const dataMax = matchResults.length
6     ? Math.max(...matchResults.map(r => r.aisa_pct)) : 0;
7   return axisMax(dataMax + 2, 10, 55);
8 }, [matchResults]);
9 const Y_MAX = useMemo(() => {
10  const dataMax = matchResults.length
11    ? Math.max(...matchResults.map(r => r.gcs_pct)) : 0;
12  return axisMax(dataMax + 1, 5, 25);
13 }, [matchResults]);

```

`axisMax` arrotonda v al multiplo di `step` superiore, con un minimo garantito. Il margine (+2 per X, +1 per Y) evita che i punti più alti tocchino il bordo del grafico. I minimi (55 per X, 25 per Y) garantiscono che le rette di soglia siano sempre completamente visibili: la soglia “NOT SUITABLE” tocca l’asse X a AISA=42, quindi `X_MAX` deve essere almeno 55 per mostrare l’intera retta.

15.8.5 Rette di soglia SVG clippate

```

1 <defs>
2   <clipPath id="ja-clip">
3     <rect x={PAD.l} y={PAD.t} width={PW} height={PH} />
4   </clipPath>
5 </defs>
6 <g clipPath="url(#ja-clip)">
7   /* NOT SUITABLE: GCS = -0.5*AISA + 21, x-intercetta AISA=42 */
8   <line x1={toX(0)} y1={toY(21)}
9     x2={toX(Math.min(42, X_MAX))}
10    y2={toY(Math.max(0, 21 - 0.5 * Math.min(42, X_MAX))}
11    stroke="rgba(210,80,80,0.30)" strokeDasharray="6,4" />
12   /* WITH PRECAUTIONS: GCS = -0.5*AISA + 15.5, x-intercetta AISA
13    ↪ =31 */
14   <line x1={toX(0)} y1={toY(15.5)}
15     x2={toX(Math.min(31, X_MAX))}
16     y2={toY(Math.max(0, 15.5 - 0.5 * Math.min(31, X_MAX))}
17     stroke="rgba(190,145,55,0.30)" strokeDasharray="6,4" />
18 </g>

```

Le rette sono calcolate da (AISA=0, GCS=intercetta) a (AISA=x-intercetta). Il clamp `Math.min(42, X_MAX)` evita che le rette si estendano oltre i dati visibili. Il `<clipPath>` taglia la retta al plot area, evitando sconfinamenti nei padding degli assi. L’opacità 0.30 rende le soglie visibili ma non dominanti rispetto ai punti dati.

15.8.6 Sidebar dei job ordinata per idoneità

```

1 [...matchResults].sort((a, b) => {
2   const rank: Record<string, number> = {
3     'NOT SUITABLE': 0, 'SUITABLE WITH PRECAUTIONS': 1, 'SUITABLE':
      ↪ 2
4   };
5   if (rank[a.suitability] !== rank[b.suitability])
6     return rank[a.suitability] - rank[b.suitability];
7   return a.job_id.localeCompare(b.job_id);
8 })

```

I job nella sidebar del tab “Suitability Map” sono ordinati per idoneità crescente (NOT SUITABLE prima, SUITABLE alla fine) e poi alfabeticamente per parità di idoneità. L’operatore spread `[...matchResults]` crea una copia prima del sort per non mutare lo stato originale (`Array.sort` è in-place).

15.8.7 Funzione `renderSkillsTable` — render helper con parametri completi

```

1 const renderSkillsTable = (
2   panelId: 'A' | 'B',
3   jobId, setJobId, skillData, loading,
4   open, setOpen, sorts, setSorts,
5   search, setSearch, showSearch, setShowSearch,
6   expandedSkill, setExpandedSkill,
7 ) => { ... };

```

Questa funzione (non un componente React) renderizza la tabella dettaglio skill per un pannello (A o B in split view). Riceve stato e setter come parametri anziché usare state interno, permettendo al componente padre di possedere tutto lo stato per la persistenza nella session cache.

La funzione controlla `!splitMode` per mostrare/nascondere le colonne “Importance”, “CS” e “CS bar”: in split view queste colonne vengono omesse per risparmiare spazio orizzontale, mantenendo solo Score, Qualifier e Criticality.

La colonna “Skill / Ability” usa un’header interattivo: cliccando sull’icona lente si trasforma in un input di ricerca `autoFocus`. Quando l’input perde il focus senza testo, torna al label originale.

Ogni riga è espandibile se la skill ha una definizione O*NET (`s.description.trim().length > 0`): il click apre una riga di dettaglio con la definizione testuale. La barra di criticità (`ja-cs-bar-track`) ha larghezza proporzionale a $(s.cs / 12) * 100\%$, riflettendo il CS normalizzato.

15.8.8 Cache di sessione anti-stale-closure

```

1 type WorkerSession = {
2   jobA: string|null; jobB: string|null; splitMode: boolean;
3   sortsA: SortState; sortsB: SortState;
4   skillSearchA: string; skillSearchB: string;
5 };
6 const sessionCache = useRef<Record<string, WorkerSession>>({});
7 const liveState     = useRef<WorkerSession>({
8   jobA: null, jobB: null, splitMode: false,
9   sortsA: [], sortsB: [], skillSearchA: '', skillSearchB: '',
10 });
11 liveState.current = { jobA, jobB, splitMode, sortsA,
12                       sortsB, skillSearchA, skillSearchB };
13
14 useEffect(() => {
15   return () => {
16     sessionCache.current[workerId] = { ...liveState.current };
17   };
18 }, [workerId]);

```

Il problema dello *stale closure* in React: quando il cleanup di un `useEffect` viene eseguito (al cambio di `workerId`), le variabili catturate nella closure hanno i valori del render al momento del **mount**, non del render corrente. Se si scrivesse direttamente `sessionCache.current[workerId] = { jobA, jobB, ... }` nel cleanup, si salverebbero sempre i valori iniziali (`null`).

La soluzione `liveState.current = { jobA, ... }` aggiorna il ref **in modo sincrono ad ogni render** (al di fuori di un `useEffect`). Il cleanup legge da `liveState.current` che è sempre aggiornato. Questo pattern è raccomandato dalla documentazione React ufficiale per accedere a valori correnti in effect cleanup.

Il restore avviene in un `useEffect` separato che dipende da `workerId`:

```

1 useEffect(() => {
2   const saved = sessionCache.current[workerId];
3   if (saved) {
4     skipAutoSelectRef.current = true; // non auto-selezionare il
    ↪ primo job
5     setJobA(saved.jobA); setJobB(saved.jobB);
6     setSplitMode(saved.splitMode); ...
7   } else {
8     skipAutoSelectRef.current = false; // prima visita: auto-
    ↪ selezione
9     setJobA(null); setJobB(null); ...
10  }
11  // Sempre: cancella i dati fetchati (verranno ri-fetchati)
12  setMatchResults([]); setSkillDataA(null); setSkillDataB(null);
13 }, [workerId]);

```


`skipAutoSelectRef` segnala al successivo effect di fetch di non auto-selezionare il primo job (perché la sessione salvata già contiene la selezione corretta). È un `useRef` (non `useState`) perché il suo cambio non deve triggerare un re-render.

15.8.9 Salvataggio job e re-reasoning

```

1 const saveJobAssignment = useCallback(async () => {
2   setSavingJobs(true);
3   await updateWorkerJobs(workerId, Array.from(draftJobIds));
4   // Ricarica i match results (Pellet e' rieseguito lato server)
5   setLoadingMatch(true);
6   const results = await fetchMatchResults(workerId);
7   setMatchResults(results);
8   if (results.length) setJobA(results[0].job_id);
9   // Reset split view
10  setJobB(null); setSplitMode(false);
11  setSkillDataA(null); setSkillDataB(null);
12  setEditJobsOpen(false);
13 }, [workerId, draftJobIds]);

```

Dopo `updateWorkerJobs`, il backend (`reasoner.py`) salva il file RDF, riesegue Pellet e aggiorna la cache JSON. La sequenza di chiamate frontend attende il completamento prima di ricaricare i match results, garantendo che il nuovo reasoning sia già disponibile. Il label del pulsante “Saving & Re-running Pellet...” comunica all’utente che l’operazione include un’esecuzione del reasoner.

15.8.10 Modale “Edit Jobs” — checkbox accessibili

```

1 <label key={job.id} className={`ja-modal-job-row${checked ? '
  ↳ checked' : ''}`}
2   htmlFor={`job-cb-${job.id}`}>
3   <span className="ja-modal-checkbox" aria-hidden="true">
4     {checked && <svg>...</svg>} { /* checkmark SVG custom */}
5   </span>
6   <input id={`job-cb-${job.id}`} type="checkbox"
7     checked={checked} onChange={toggle}
8     style={{ position: 'absolute', opacity: 0, pointerEvents: 'none
9       ↳ , }} />
10  <span className="ja-modal-job-label">{job.label}</span>
11  <span className="ja-modal-job-id">{job.id}</span>
12 </label>

```

Il pattern `<label htmlFor> + <input hidden> + span custom` è il modo standard per creare checkbox custom visivamente senza perdere l’accessibilità: l’input nativo è nascosto visivamente ma rimane nel DOM, permettendo l’attivazione da

tastiera (Space) e la corretta lettura da screen reader. `aria-hidden="true"` sul `span custom` evita che il SVG venga letto due volte.

15.9 src/components/HealthConditionWizard.tsx e HealthConditionWizard.css

15.9.1 Ruolo, dimensioni e props

`HealthConditionWizard.tsx` (1126 righe) implementa la procedura guidata per la modifica delle condizioni ICF. Viene montato dentro `WorkersPage` quando l'utente clicca "Modify Health Conditions" e smontato alla chiusura.

```

1 interface HealthConditionWizardProps {
2   workerId:          string;
3   currentConditions: HealthCondition[];
4   allCoreSets:       string[];    // passato da WorkersPage (già
    ↳ fetchato)
5   onClose:           () => void;
6   onSave:             () => void;
7   onStepChange?:     (step: Step) => void;
8 }

```

`allCoreSets` viene passato come prop da `WorkersPage` (anziché `fetchato` internamente) perché `WorkersPage` lo ha già caricato per il proprio filtro. Passarlo come prop evita una richiesta API duplicata al momento dell'apertura della wizard.

`onStepChange` è opzionale (?): permette a `WorkersPage` di aggiornare il breadcrumb di navigazione senza dover gestire lo stato dello step internamente.

15.9.2 La macchina a stati della wizard

```

1 type Step = 'select' | 'review' | 'saving' | 'done';
2 const [step, setStep] = useState<Step>('select');
3
4 useEffect(() => { onStepChange?.(step); }, [onStepChange, step]);

```

La wizard è una macchina a stati esplicita con quattro stati e transizioni unidirezionali: `select` → `review` → `saving` → `done` (con possibilità di tornare a `review` in caso di errore di salvataggio).

Le transizioni sono gestite in `handleConfirm`:

```

1 const handleConfirm = async () => {
2   if (step === 'select') { setStep('review'); return; }
3   if (step === 'review') {
4     // Validazione: tutti i codici selezionati hanno un qualifier
5     const hasMissing = Array.from(selectedCodes)

```

```

6      .some(code => assignedQualifiers[code] == null);
7      if (hasMissing) {
8          // Scroll automatico all'errore + toast
9          const firstErrorEl = document.querySelector('.hc-row-error');
10         container.scrollTo({ top: offsetTop, behavior: 'smooth' });
11         return;
12     }
13     setStep('saving');
14     // ... build changes, POST API, setStep('done')
15 }
16 };

```

15.9.3 Inventario completo dello stato interno

Table 39: Variabili di stato di HealthConditionWizard.

Variabile	Tipo	Scopo
step	Step	Stato corrente della macchina
allIcfCodes	IcfCodeEntry[]	Catalogo ICF dall'API
loading	boolean	Spinner durante fetch catalogo
error	string null	Errore fetch o salvataggio
search	string	Ricerca nel catalogo (step select)
reviewSearch	string	Ricerca nello step review
isSearchFocused	boolean	Freccia nella search box
isReviewSearchFocused	boolean	idem nello step review
coreSetFilterOpen	boolean	Dropdown Core Set aperto
wizardSelectedCoreSets	string[]	Filtri core set attivi
categoryFilterOpen	boolean	Dropdown categoria ICF
selectedCategories	string[]	Prefissi ICF filtrati (b, d)
expandedReviewCode	string null	Riga espansa in review
showScrollToast	boolean	Toast "scroll down"
showMissingValueToast	boolean	Toast qualifier mancante
showSavedToast	boolean	Toast salvataggio riuscito
showDiscardPopup	boolean	Popup conferma abbandono
scrollToastDismissedStep	Step null	Step in cui il toast è stato chiuso
selectedCodes	Set<string>	Codici ICF selezionati
assignedQualifiers	Record<string, int null>	Qualifier assegnato per codice

15.9.4 Inizializzazione dallo stato attuale del worker

```

1 useEffect(() => {
2   // Preseleziona i codici ICF gia' presenti nel profilo del worker
3   const current = new Set(currentConditions.map(c => c.icf_code));
4   setSelectedCodes(current);
5
6   // Pre-popola i qualifier dai valori correnti
7   const initialQuals: Record<string, number | null> = {};
8   currentConditions.forEach(c => {
9     const q = c.bf_qualifier ?? c.ap1_qualifier ?? null;
10    if (q !== null) initialQuals[c.icf_code] = q;
11  });
12  setAssignedQualifiers(initialQuals);
13
14  // Carica il catalogo ICF completo
15  fetchAllIcfCodes()
16    .then(codes => { setAllIcfCodes(codes); setError(null); })
17    .catch(e => setError(e.message))
18    .finally(() => setLoading(false));
19 }, [currentConditions]);

```

Questo effetto gira al mount (e ogni volta che `currentConditions` cambia). La pre-selezione dei codici correnti garantisce che la wizard parta con lo stato del worker già riflesso nel catalogo, rendendo immediatamente visibile cosa è selezionato. Il qualifier pre-popolato usa `bf_qualifier ?? ap1_qualifier`: la stessa logica di fallback usata nel display della tabella ICF.

15.9.5 Normalizzazione del catalogo con useMemo

```

1 const normalizedIcfCodes = useMemo(() => {
2   const byCode = new Map<string, IcfCodeEntry>();
3   allIcfCodes.forEach(code => {
4     const existing = byCode.get(code.icf_code);
5     if (!existing) {
6       byCode.set(code.icf_code, {
7         ...code, description: code.description || '',
8         core_sets: [...new Set(code.core_sets || [])],
9       });
10      return;
11    }
12    // Fusione: first-non-empty wins per nome e descrizione;
13    // union per core_sets
14    byCode.set(code.icf_code, {
15      ...existing,
16      icf_name: existing.icf_name || code.icf_name,

```

```

17     description: existing.description || code.description || '',
18     category:    existing.category !== 'Other' ? existing.
    ↪ category
19                                     : code.category,
20     core_sets: [...new Set([...existing.core_sets, ...code.
    ↪ core_sets])],
21   });
22 });
23 return Array.from(byCode.values())
24   .sort((a, b) => a.icf_code.localeCompare(b.icf_code));
25 }, [allIcfCodes]);

```

Il catalogo ICF dall'API può contenere più entry con lo stesso codice ICF ma IRI diversi (varianti ontologiche dello stesso codice). La normalizzazione produce una entry unica per codice:

- **icf_name e description:** prima stringa non vuota vince (*first-non-empty wins*).
- **category:** preferisce qualsiasi valore diverso da 'Other'.
- **core_sets:** unione deduplicata con `new Set([...a, ...b])`.

Il sort finale per `icf_code` usa `localeCompare` anziché il sort lessicale standard (`a < b`) per una ragione specifica dei codici ICF: i codici usano ordinamento alfanumerico misto (b1, b110, b1100, b114) che un sort lessicale disporrebbe correttamente solo per caso. `localeCompare` con la locale corrente tratta le cifre come numeri nelle sequenze alfanumeriche, garantendo `b1 < b110 < b1100` e `d175 < d330`, che corrisponde all'ordine gerarchico della classificazione ICF.

15.9.6 Filtro multi-criterio con useMemo

```

1 const filteredCodes = useMemo(() => {
2   const q = search.toLowerCase();
3   const matchesText = (c: IcfCodeEntry) =>
4     c.icf_code.toLowerCase().includes(q) ||
5     c.icf_name.toLowerCase().includes(q) ||
6     (c.core_sets || []).join(' ').toLowerCase().includes(q);
7
8   const matchesFilter = (c: IcfCodeEntry) =>
9     wizardSelectedCoreSets.length === 0 ||
10    wizardSelectedCoreSets.every(cs => (c.core_sets || []).includes
    ↪ (cs));
11
12   // selectedCategories contiene prefissi ICF ('b', 'd'), non label
13   const matchesCategory = (c: IcfCodeEntry) =>
14     selectedCategories.length === 0 ||

```

```

15     selectedCategories.includes(c.icf_code.toLowerCase().charAt(0))
    ↪ ;
16
17     return normalizedIcfCodes.filter(
18       c => matchesText(c) && matchesFilter(c) && matchesCategory(c));
19 }, [normalizedIcfCodes, search, wizardSelectedCoreSets,
    ↪ selectedCategories]);

```

Il filtro combina tre predicati con **AND** — non OR. La scelta della congiunzione è deliberata: un OR produrrebbe un insieme troppo ampio (tutti i codici che corrispondono a *qualsiasi* criterio), rendendo difficile trovare i codici rilevanti per il caso clinico specifico. L'AND implementa una logica di *raffinamento progressivo*: ogni filtro aggiuntivo restringe l'insieme, permettendo al clinico di arrivare rapidamente ai codici pertinenti (es. solo codici b* del core set Stroke con testo “attention”). Per il filtro core set in particolare, l'AND su più core set selezionati simultaneamente trova i codici che coprono condizioni multi-patologiche — caso frequente nei pazienti di Rientr@.

1. **Text search**: cerca nel codice, nel nome, e nei core set (es. cercare “stroke” trova tutti i codici del core set Stroke).
2. **Core set filter**: AND su tutti i core set selezionati (**every**): un codice deve appartenere a *tutti* i core set selezionati simultaneamente.
3. **Category filter**: filtra per prefisso ICF (b o d), non per label. La const CATEGORIES mappa label → prefisso per la UI, ma il confronto avviene sul carattere iniziale.

15.9.7 Sistema di diff per il batch di modifiche

```

1 // Costruisce il payload minimo di modifiche
2 const currentMap = new Map(currentConditions.map(c => [c.icf_code,
    ↪ c]));
3 const changes: HcChangeItem[] = [];
4
5 // REMOVE: era presente, non e' piu' selezionato
6 for (const c of currentConditions)
7   if (!selectedCodes.has(c.icf_code))
8     changes.push({ icf_code: c.icf_code, action: 'remove',
    ↪   qualifier: null });
9
10 // ADD / MODIFY: e' selezionato
11 for (const code of Array.from(selectedCodes)) {
12   const isNew = !currentMap.has(code);
13   const oldQual = currentMap.get(code)?.bf_qualifier
14     ?? currentMap.get(code)?.ap1_qualifier ?? null;

```

```

15   const newQual = assignedQualifiers[code] ?? null;
16
17   if (isNew && newQual !== null)
18     changes.push({ icf_code: code, action: 'add', qualifier:
19       ↪ newQual });
20   else if (!isNew && oldQual !== newQual && newQual !== null)
21     changes.push({ icf_code: code, action: 'modify', qualifier:
22       ↪ newQual });
23   }
24
25   if (changes.length === 0) { onSave(); onClose(); return; } //
26   ↪ nulla da fare

```

Il diff calcola il minimo numero di operazioni necessarie: solo i codici *effettivamente* modificati vengono inclusi nel batch. Un codice presente con lo stesso qualifier non genera nessuna modifica. Il caso `changes.length === 0` viene cortocircuitato: se non c'è nulla da salvare, la wizard si chiude immediatamente senza chiamata API.

15.9.8 Toast system: visibilità overflow-safe

```

1  import { createPortal } from 'react-dom';
2  // ...
3  {showScrollToast && createPortal(
4    <div className="hc-toast hc-toast--info">
5      Scroll down to see more ICF codes
6      <ArrowDownIcon />
7    </div>,
8    document.body
9  )}

```

I toast vengono renderizzati nel `document.body` tramite `createPortal`, uscendo dal flusso del DOM della wizard. Questo è necessario per due ragioni:

1. La wizard ha `overflow: hidden` per contenere gli scroll interni: i toast resi dentro la wizard verrebbero ritagliati.
2. Il `z-index` della wizard potrebbe non essere sufficiente per superare altri layer globali dell'applicazione.

15.9.9 Toast di scroll: rilevamento overflow dinamico

```

1  useEffect(() => {
2    const activeContainer = step === 'select'
3      ? selectScrollRef.current
4      : step === 'review' ? reviewScrollRef.current : null;

```

```

5   if (!activeContainer || scrollToastDismissedStep === step) return
    ↪ ;
6
7   const updateScrollToast = () => {
8     const hasOverflow =
9       activeContainer.scrollHeight - activeContainer.clientHeight >
    ↪ 8;
10    const hasMoreBelow =
11      activeContainer.scrollTop + activeContainer.clientHeight
12      < activeContainer.scrollHeight - 8;
13    setShowScrollToast(hasOverflow && hasMoreBelow);
14  };
15
16  updateScrollToast();
17  activeContainer.addEventListener('scroll', updateScrollToast);
18  window.addEventListener('resize', updateScrollToast);
19  return () => {
20    activeContainer.removeEventListener('scroll', updateScrollToast
    ↪ );
21    window.removeEventListener('resize', updateScrollToast);
22  };
23 }, [step, loading, filteredCodes.length, toModify.length,
24    toAdd.length, toRemove.length, expandedReviewCode,
25    scrollToastDismissedStep]);

```

Il toast “Scroll down” appare quando il contenitore ha overflow verticale e ci sono ancora contenuti sotto la viewport. La soglia di 8px evita falsi positivi da pixel di rounding. L'effetto dipende da molte variabili (`filteredCodes.length`, `expandedReviewCode`, ecc.) perché qualsiasi di questi cambi può alterare l'altezza del contenuto e quindi la visibilità del toast. `scrollToastDismissedStep` registra in quale step l'utente ha chiuso il toast, evitando che riappaia immediatamente al primo render successivo dello stesso step.

15.9.10 Rilevamento modifiche non salvate (`hasUnsavedChanges`)

```

1   const hasUnsavedChanges = () => {
2     const currentMap = new Map(currentConditions.map(c => [c.icf_code
    ↪ , c]));
3     // Rimossi?
4     for (const c of currentConditions)
5       if (!selectedCodes.has(c.icf_code)) return true;
6     // Aggiunti o modificati?
7     for (const code of Array.from(selectedCodes)) {
8       if (!currentMap.has(code)) return true;
9       const oldQual = currentMap.get(code)?.bf_qualifier
10        ?? currentMap.get(code)?.api_qualifier ?? null;

```



```

11     if (oldQual !== (assignedQualifiers[code] ?? null)) return true
12     ↪ ;
13   }
14   return false;
15 };
16
17 const handleCloseClick = () => {
18   if (step !== 'saving' && step !== 'done' && hasUnsavedChanges())
19     setShowDiscardPopup(true); // popup conferma
20   else
21     onClose();
22 };

```

Prima di chiudere la wizard, viene verificato se ci sono modifiche non salvate. Il popup di conferma (`showDiscardPopup`) appare solo se `hasUnsavedChanges()` ritorna `true` e il salvataggio non è già in corso o completato. Questo previene la perdita accidentale di lavoro durante step lunghi come la revisione dei qualifier.

15.9.11 Chiusura automatica post-salvataggio

```

1 try {
2   await updateHealthConditions(workerId, changes);
3   setShowSavedToast(true);
4   setStep('done');
5   setTimeout(() => {
6     onSave(); // ricarica le condizioni in WorkersPage
7     onClose(); // smonta la wizard
8   }, 1800); // lascia il tempo di leggere il toast
9 } catch (err: any) {
10   setError(err.message || 'Failed to save changes. ');
11   setStep('review'); // torna allo step precedente
12 }

```

Dopo il salvataggio riuscito, la wizard mostra il toast di conferma per 1800ms prima di chiamare `onSaved()` e `onClose()`. Questo intervallo è sufficiente per leggere il messaggio senza essere eccessivo. In caso di errore, lo step torna a `'review'` (non a `'select'`) perché le selezioni sono ancora valide; solo il tentativo di salvataggio è fallito.

15.10 Sistema di design CSS — Token, convenzioni e componenti

15.10.1 Paletta cromatica e token impliciti

I tre file CSS (`WorkersPage.css`, `JobAnalysisView.css`, `HealthConditionWizard.css`) condividono una paletta cromatica coerente definita da token impliciti (valori ricorrenti) piuttosto che da variabili CSS globali, ad eccezione di `HealthConditionWizard.css` che usa `var()` per alcuni colori semantici.

Table 40: Token cromatici impliciti condivisi tra tutti i CSS.

Ruolo	Valore	Uso
Brand teal primario	#4DD9C0	Selezionato attivo, accenti interattivi, border selected, bar fill.
Brand teal hover	#5de8d0	Hover sui pulsanti primari (CTA).
Sfondo applicazione	#1a2a4a	Gradiente base di HomePage e WorkersPage.
Sfondo tabelle sticky	#1a2e4a	Header sticky di tabelle (leggermente più chiaro del fondo).
Sfondo sidebar	#234365	Sidebar di WorkersPage.
Sfondo modale	#162540 / #0f1e35	Gradient modale Edit Jobs.
Critico verde	#4ade80	.wp-eff-1 qualifier; .ja-anchor-3 anchor.
Critico ambra	#fbbf24	.wp-eff-2 qualifier.
Critico arancio	#fb923c	.wp-eff-3 qualifier.
Critico rosso	#f87171	.wp-eff-4 qualifier; errori.
Testo su teal	#0f2233 / #111e30	Testo scuro sui pulsanti CTA teal (contrasto WCAG).
Discard rosso	#A33F39	Titolo e bordi del popup discard (unico elemento light-mode).

Coerenza cromatica tra sfondo e Electron

Il valore #1a2a4a usato in `.home-page` e `.wp-page` come stop iniziale del gradiente è lo stesso impostato in `backgroundColor` della `BrowserWindow` di Electron. Questa coerenza è intenzionale: durante il caricamento del renderer, Electron mostra prima il colore nativo della finestra e poi il DOM React. Con colori corrispondenti, la transizione è impercettibile.

15.10.2 Convenzioni di naming CSS

I tre file usano prefissi BEM-like per namespace delle classi:

Table 41: Prefissi CSS e namespace corrispondenti.

Prefisso	File	Motivazione
wp-	WorkersPage.css	Workers Page. Evita collisioni con classi globali o di terze parti.
ja-	JAView.css	Job Analysis. Stessa motivazione.
hc-	HCWizard.css	Health Condition. Coerente con i prefissi API (BFqual, AP1qual).
bg-blob-	entrambi	Condiviso: stesso keyframe blobDrift riutilizzato in WorkersPage senza ridefinire il gradiente.

15.10.3 WorkersPage.css — analisi approfondita

Navbar con backdrop-filter: blur(16px)

```

1 .wp-navbar {
2   z-index: 20;
3   background: rgba(255,255,255,0.06);
4   backdrop-filter: blur(16px);
5   border-bottom: 1px solid rgba(255,255,255,0.12);
6   box-shadow: 0 2px 16px rgba(0,0,0,0.2);
7 }

```

La navbar usa 16px di blur (più del frosted glass delle card che usano 10px) perché deve rimanere leggibile anche quando i blob animati le passano sotto ad alta velocità. Il bordo inferiore e l'ombra creano una separazione visiva netta tra navbar e contenuto, necessaria quando il contenuto scorre sotto di essa. `z-index: 20` la posiziona sopra la sidebar (`z-index: 10`) ma non sopra le modali (`z-index: 200+`).

Breadcrumb — design pill con stati di opacità

```

1 .wp-breadcrumb-text { color: rgba(255,255,255,0.60); }
2 .wp-breadcrumb-text.last-before-current { color: rgba
  ↪ (255,255,255,0.95); font-weight: 500; }
3 .wp-breadcrumb-text.current { color: #4DD9C0; font-weight: 600; }

```

Tre livelli di visibilità comunicano la gerarchia di navigazione senza colori diversi: antenati lontani in grigio, genitore diretto quasi bianco, nodo corrente teal. Questo è il pattern di breadcrumb standard usato in Google, GitHub e Figma.

Sidebar collassabile — transizione multi-proprietà

```

1 .wp-sidebar {

```

```

2   width: 240px; min-width: 240px; max-width: 240px;
3   transition:
4     width 0.35s cubic-bezier(0.2,0.8,0.2,1),
5     min-width 0.35s cubic-bezier(0.2,0.8,0.2,1),
6     max-width 0.35s cubic-bezier(0.2,0.8,0.2,1),
7     padding 0.35s cubic-bezier(0.2,0.8,0.2,1),
8     margin 0.35s cubic-bezier(0.2,0.8,0.2,1),
9     opacity 0.35s ease;
10  }
11  .wp-sidebar--closed {
12    width: 0 !important; min-width: 0 !important; max-width: 0 !
    ↪ important;
13    padding-left: 0 !important; padding-right: 0 !important;
14    margin-left: 0 !important; border: none !important;
15    opacity: 0; pointer-events: none;
16  }

```

Animare solo `width` non è sufficiente: se `min-width` rimane a 240px la sidebar non si contrae mai. Animare anche `min-width` e `max-width` con la stessa curva garantisce che il collapse avvenga uniformemente. Il `padding` deve essere azzerato per evitare che il bordo della sidebar rimanga visibile come spazio vuoto. `pointer-events: none` disabilita i click durante e dopo il collapse, evitando interazioni accidentali con elementi nascosti.

Il `!important` è necessario perché i valori di base (`width: 240px`) sono dichiarati direttamente sul selettore `.wp-sidebar` senza classi di media query: la specificità CSS di `.wp-sidebar-closed` sarebbe uguale e il collasso vincerebbe solo a seconda dell'ordine nel file. Usare `!important` rende lo stato `closed` sempre prevalente.

Spinner CSS puro

```

1  .wp-spinner {
2    width: 20px; height: 20px;
3    border: 2px solid rgba(77,217,192,0.2);
4    border-top-color: #4DD9C0;
5    border-radius: 50%;
6    animation: spin 0.8s linear infinite;
7  }
8  @keyframes spin { to { transform: rotate(360deg); } }

```

Il pattern del cerchio con un solo bordo colorato (`border-top-color`) è il modo standard per creare uno spinner CSS senza SVG o immagini. `border-radius: 50%` rende il rettangolo circolare. La velocità 0.8s è più rapida dello standard (1s) per trasmettere senso di attività intensa durante il caricamento dei dati SPARQL.

Sistema badge qualificatori — scala semantica 5 livelli

```

1 .wp-eff-0 { background: rgba(255,255,255,0.06); color: rgba
  ↳ (255,255,255,0.4); } /* nessun problema */
2 .wp-eff-1 { background: rgba(34,197,94,0.15); color: #4ade80; }
  ↳ /* lieve */
3 .wp-eff-2 { background: rgba(245,158,11,0.15); color: #fbbf24; }
  ↳ /* moderato */
4 .wp-eff-3 { background: rgba(249,115,22,0.15); color: #fb923c; }
  ↳ /* grave */
5 .wp-eff-4 { background: rgba(239,68,68,0.18); color: #f87171; }
  ↳ /* completo */

```

I cinque livelli usano i colori semantici della classificazione ICF (verde=sano, rosso=grave), con opacità ridotta sullo sfondo (0.15/0.18) per non appesantire visivamente le righe della tabella. I colori corrispondono ai valori Tailwind CSS standard (green-400, amber-400, orange-400, red-400), coerenti con i colori di idoneità in MatchResult (CRIT_COLOR in JobAnalysisView).

Skeleton loading — shimmer effect

```

1 .wp-skeleton-line {
2   background: linear-gradient(90deg,
3     rgba(255,255,255,0.07) 25%,
4     rgba(255,255,255,0.14) 50%,
5     rgba(255,255,255,0.07) 75%
6   );
7   background-size: 200% 100%;
8   animation: skeletonShimmer 1.4s linear infinite;
9 }
10 @keyframes skeletonShimmer {
11   0% { background-position: 200% 0; }
12   100% { background-position: -200% 0; }
13 }

```

L'effetto shimmer è implementato animando la posizione di un gradiente lineare più largo del contenitore (background-size: 200%): il gradiente scorre da destra a sinistra simulando un riflesso di luce che attraversa il placeholder. È preferito a un fade semplice perché comunica visivamente “caricamento in corso” con direzione, non solo presenza.

Switching state — animazione API in-flight

```

1 .wp-worker-item--switching {

```

```

2   opacity: 0.6;
3   pointer-events: none;
4   animation: workerSwitchPulse 0.9s ease-in-out infinite alternate;
5 }
6 @keyframes workerSwitchPulse { from { opacity:0.45; } to { opacity
  ↪ :0.75; } }
7
8 .wp-worker-switching-dot {
9   background: #4DD9C0;
10  animation: switchDotBlink 0.7s ease-in-out infinite alternate;
11 }
12 @keyframes switchDotBlink {
13   from { opacity:0.3; transform:scale(0.7); }
14   to   { opacity:1;   transform:scale(1.2); }
15 }

```

Mentre `POST /workers/select` è in volo, il worker in attesa viene reso con due indicatori simultanei: il pulsante intero pulsa in opacità (`workerSwitchPulse`) e un dot teal lampeggia con scala (`switchDotBlink`). La doppia animazione garantisce visibilità anche su display con refresh rate ridotto o con daltonismo. `pointer-events: none` previene un secondo click sullo stesso worker durante l'attesa.

Barra di caricamento indeterminata

```

1 @keyframes barFill {
2   0%   { width: 0%;   opacity: 1; }
3   85%  { width: 100%; opacity: 1; }
4   95%  { width: 100%; opacity: 0; }
5   100% { width: 0%;   opacity: 0; }
6 }

```

La barra di caricamento Pellet non conosce il progresso reale (Pellet non espone una API di avanzamento), quindi simula un avanzamento indeterminato: cresce rapidamente fino al 100% nei primi 85% del tempo, poi svanisce e riparte. Il reset a larghezza 0 con opacità 0 (frame 95–100%) evita il salto visivo da 100% a 0%.

Modale import — animazione spring

```

1 .wp-modal { animation: modalSlideIn 0.25s cubic-bezier
  ↪ (0.2,0.8,0.2,1); }
2 @keyframes modalSlideIn {
3   from { opacity:0; transform:translateY(20px) scale(0.97); }
4   to   { opacity:1; transform:translateY(0)   scale(1); }
5 }

```

La modale entra con una combinazione di fade, slide (20px verso l'alto) e leggero scale-up (da 0.97 a 1). La curva cubic-bezier(0.2, 0.8, 0.2, 1) è una *ease-out* accentuata: l'animazione è rapida all'inizio e si stabilizza dolcemente. Questo è il pattern Material Design 3 per le modali.

15.10.4 JobAnalysisView.css — analisi approfondita

Architettura flex a doppio livello — fill disponibile

```

1 .ja-root      { display:flex; flex-direction:column; flex:1; min-
  ↪ height:0; }
2 .ja-tab-content { flex:1; min-height:0; overflow:hidden; }
3 .ja-skills-wrap { flex:1; overflow:auto; min-height:0; }
```

Il pattern `flex: 1; min-height: 0` è necessario in ogni livello dell'albero flex quando il figlio deve occupare lo spazio rimanente e scorrere internamente. Senza `min-height: 0`, un figlio flex ignora l'altezza del genitore e si espande oltre il viewport. La regola CSS dice che il comportamento di default di un elemento flex è `min-height: auto` (che è la dimensione del contenuto), non 0. Questa è la fonte più comune di bug di layout in applicazioni React con scroll nidificato.

KPI strip — font-variant-numeric: tabular-nums

```

1 .ja-kpi-val { font-variant-numeric: tabular-nums; }
```

`tabular-nums` impone che tutte le cifre abbiano la stessa larghezza, evitando che i valori numerici nelle KPI card “ballino” visivamente quando cambiano (es. da “5” a “12” a “9”). È particolarmente rilevante per GCS% e AISA% che sono float con variabilità a due cifre.

Tab attivo — border-bottom pixel-perfect

```

1 .ja-tab {
2   border-bottom: 2px solid transparent;
3   position: relative;
4   bottom: -1px;    /* copre il bordo del tab-bar */
5 }
6 .ja-tab.active { border-bottom-color: #ffffff; }
```

Il trick `bottom: -1px` sposta il tab di un pixel verso il basso in modo che il suo bordo inferiore bianco si sovrapponga esattamente al bordo inferiore della tab-bar, creando l'effetto “tab selezionato che si fonde con il contenuto”. Senza questo offset il bordo della tab-bar e il bordo del tab sarebbero adiacenti ma non sovrapposti, producendo una doppia linea.

Dot selezionato nel scatter plot — animazione pulse

```

1 .ja-ring-pulse {
2   animation: jaRingPulse 2s ease-in-out infinite;
3   transform-origin: center;
4 }
5 @keyframes jaRingPulse {
6   0%,100% { opacity:0.45; transform:scale(0.88); }
7   50%     { opacity:0.10; transform:scale(1.12); }
8 }

```

Il dot selezionato mostra un anello SVG che pulsa verso l'esterno (`scale: 0.88` → `1.12`). L'`opacity` cala a 0.10 al massimo di scala (quasi trasparente), simulando un'onda che si propaga e si dissolve. `transform-box: fill-box` garantisce che l'origine della trasformazione sia il centro del cerchio SVG, non l'angolo superiore-sinistro del viewport SVG.

Tabella skill — header sticky con sfondo solido

```

1 .ja-skills-tbl th {
2   position: sticky; top: 0; z-index: 2;
3   background: #1a2e4a; /* solido, non rgba */
4 }

```

L'header della tabella usa un colore solido (`#1a2e4a`) anziché `rgba` perché con `position: sticky` il colore semi-trasparente permetterebbe alle righe che scorrono sotto di apparire in trasparenza. Il valore `#1a2e4a` è scelto come approssimazione opaca dello sfondo medio della tabella.

Freccia espansore skill — posizionamento assoluto a sinistra

```

1 .ja-row-expander-arrow {
2   position: absolute;
3   left: -22px; /* fuori dalla cella, nello spazio del padding */
4   transition: transform 0.2s ease, opacity 0.2s ease;
5 }
6 .ja-row-expander-arrow.is-expanded { transform: rotate(180deg);
  ↪ opacity:1; }

```

La freccia di espansione è posizionata a sinistra della prima colonna, nel padding della cella (`padding-left: 36px` per le celle dati, `left: -22px` per la freccia). Questo evita di aggiungere una colonna dedicata alla tabella, preservando lo spazio per i dati. La rotazione di `180deg` al click ruota la freccia da “giù” a “su” senza cambio di icona.

Barra CS — transizione fluida sui dati

```
1 .ja-cs-bar-fill { transition: width 0.45s ease; }
```

La transizione CSS sulla `width` della barra del CS produce un'animazione fluida quando i dati cambiano (es. cambiando il job selezionato). Non richiede gestione JavaScript dell'animazione: il browser interpola automaticamente tra il valore vecchio e il nuovo.

Modale Edit Jobs — spring curve e z-index stratificato

```
1 .ja-modal-backdrop { z-index: 1000; background: rgba(6,15,32,0.72);
  ↳ backdrop-filter: blur(8px); }
2 .ja-modal { animation: jaModalIn 0.2s cubic-bezier
  ↳ (0.34,1.56,0.64,1); }
3 @keyframes jaModalIn {
4   from { opacity:0; transform:scale(0.94) translateY(12px); }
5   to   { opacity:1; transform:scale(1) translateY(0); }
6 }
```

Il backdrop della modale Edit Jobs usa `z-index: 1000` (vs.200 della modale import di `WorkersPage`): questo garantisce che sovrasti qualsiasi altro elemento compreso il dropdown picker del job (`z-index: 100`). La curva `cubic-bezier(0.34, 1.56, 0.64, 1)` è la stessa spring curve con overshooting usata nelle card di `HomePage`, mantenendo coerenza percettiva tra le animazioni dell'applicazione.

15.10.5 HealthConditionWizard.css — analisi approfondita

CSS custom properties (var()) nella wizard

`HealthConditionWizard.css` è l'unico file tra i tre a usare variabili CSS (`var()`) estensivamente. Le variabili usate (`-border-color`, `-text-base`, `-text-muted`, `-brand-teal`, `-bg-surface-1`) non sono dichiarate nel file stesso ma sono ereditate dal DOM genitore (`WorkersPage`).

Perché var() solo nella wizard

La wizard è stata sviluppata in una fase del progetto in cui si prevedeva un sistema di theming globale con variabili CSS dichiarate in `:root`. Gli altri componenti (`WorkersPage`, `JobAnalysisView`) usano valori hardcoded per le stesse ragioni per cui `App.css` è legacy: il design system si è stabilizzato prima che le variabili venissero uniformate. Il misto `var()` + hardcoded è un debito tecnico: le variabili della wizard ricevono i valori di fallback (es. `var(-brand-teal,`

#4DD9C0)) per evitare il caso in cui le variabili non siano definite, ma il sistema non è pienamente coerente.

Wizard montata in-page, non come modale globale

```

1 .hc-wizard-overlay { width:100%; height:100%; display:flex; align-
  ↪ items:center; }
2 .hc-wizard-modal {
3   width:100%; height:100%; max-width:none;
4   /* No background/border/shadow: displayed inside wp-panel */
5 }

```

La wizard non è un `position: fixed` che copre tutta la finestra: occupa il pannello principale di `WorkersPage` al posto della tabella ICF. Questo è visibile nel commento del CSS: “*No background/border/shadow since it’s displayed directly inside the page .wp-panel now*”.

La motivazione di questa scelta è UX: mantenere la sidebar con la lista workers visibile durante la modifica permette di vedere quale worker si sta modificando e il breadcrumb aggiornato in tempo reale. Una modale full-screen avrebbe oscurato queste informazioni contestuali.

Tabella catalogo — `table-layout: fixed`

```

1 .hc-wizard-table { table-layout: fixed; }

```

`table-layout: fixed` fa sì che le larghezze delle colonne siano determinate dalla prima riga (o da larghezze esplicite sul `<col>`) invece di calcolarle adattivamente sul contenuto. Questo è necessario perché: (1) il catalogo ICF ha 400+ righe e il calcolo adattivo richiederebbe di leggere tutte prima del render; (2) con `fixed`, il testo lungo viene troncato con `text-overflow: ellipsis` invece di allargare la colonna; (3) le transizioni di scroll sono più performanti con colonne di larghezza fissa.

Header sticky con sfondo solido

```

1 .hc-wizard-table th {
2   position: sticky; top: 0; z-index: 10;
3   background: #1a2e4a; /* solido per evitare bleed-through */
4 }
5 .hc-review-table-header {
6   position: sticky; top: 0; z-index: 10;
7   background: #1a2e4a;
8 }

```

Stessa motivazione della tabella skill: colore solido obbligatorio per gli header sticky. Il commento nel CSS lo conferma esplicitamente: “*solid to prevent row bleed-through*”. `z-index: 10` è superiore a quello degli elementi della riga (che non hanno `z-index` esplicito) ma inferiore al toast (`z-index: 1200`).

Checkbox custom — outline-only

```

1 .hc-checkbox {
2   border: 2px solid rgba(255,255,255,0.3);
3   border-radius: 6px; color: transparent;
4   pointer-events: none; /* click sulla riga */
5 }
6 .hc-checkbox.checked {
7   background: transparent;
8   border-color: #ffffff;
9   color: #ffffff;
10 }
```

La checkbox della wizard usa un design *outline-only*: quando selezionata, solo il bordo e la spunta (SVG figlio) diventano bianchi, senza riempimento colorato. Questo è diverso dal design `background: #4DD9C0` della modale Edit Jobs.

La differenza di design è motivata dal contesto: nella wizard le righe selezionate hanno già uno sfondo verde (`.selected-row: background: rgba(77, 217, 192, 0.08)`), quindi aggiungere anche un checkbox colorato sarebbe ridondante. Nella modale Edit Jobs lo sfondo è trasparente, quindi il checkbox teal è l'unico indicatore di stato.

Qualifier selector — segmented control

```

1 .hc-qualifier-group {
2   background: rgba(255,255,255,0.04);
3   border-radius: 20px;
4   overflow: hidden;
5   border: 1px solid rgba(255,255,255,0.08);
6 }
7 .hc-qualifier-btn { flex:1; border-right:1px solid rgba
8   ↪ (255,255,255,0.05); }
9 .hc-qualifier-btn:last-child { border-right:none; }
10 .hc-qualifier-btn.active {
11   background: var(--brand-teal, #4DD9C0);
12   color: #111e30; pointer-events:none;
13 }
```

Il selettore del qualifier usa il pattern *segmented control*: un container con

`border-radius` e `overflow: hidden` che ospita pulsanti flex con separatori interni. `pointer-events: none` sul bottone attivo previene un click accidentale sul qualifier già selezionato, che ricreerebbe inutilmente il state.

Sistema toast — posizionamento e animazione

```

1 .hc-toast-stack {
2   position: fixed; bottom: 28px; left: 50%;
3   transform: translateX(-50%);
4   z-index: 1200; pointer-events:none;
5 }
6 .hc-toast {
7   animation: hc-toast-in 0.22s cubic-bezier(0.22,1,0.36,1);
8 }
9 @keyframes hc-toast-in {
10   from { opacity:0; transform:translateY(11px) scale(0.97); }
11   to   { opacity:1; transform:translateY(0) scale(1); }
12 }
```

Il container dei toast è `position: fixed` centrato orizzontalmente (`left:50%; transform: translateX(-50%)`). `z-index: 1200` è il più alto dell'applicazione: supera le modali (`z-index: 1000`) garantendo visibilità assoluta. `pointer-events: none` sul container (ma auto sui singoli toast tramite override) permette al click di passare attraverso lo stack quando nessun toast è visibile.

I tre tipi di toast (`-info`, `-success`, `-error`) usano colori **light-mode** (sfondo bianco, testo scuro):

```

1 .hc-toast--info    { background: #E8F0FE; color: #1967D2; } /*
  ↳ Google Blue */
2 .hc-toast--success { background: #E6F4EA; color: #137333; } /*
  ↳ Google Green */
3 .hc-toast--error   { background: #FCE8E6; color: #C5221F; } /*
  ↳ Google Red   */
```

Questa è l'unica eccezione al tema dark dell'applicazione. La motivazione è il contrasto e la leggibilità: i toast devono essere immediatamente distinguibili dallo sfondo scuro. I colori Material Design (Google) per i toast sono stati scelti perché ben testati su accessibilità (rapporto di contrasto WCAG AA) e immediatamente riconoscibili come messaggi di sistema.

Popup Discard — unico elemento light-mode

```

1 .hc-discard-popup { background: #F8F9FA; border-radius: 12px; }
2 .hc-discard-title { color: #A33F39; }
```

```
3 .hc-discard-btn-discard { background: #A33F39; color: #fff; }
```

Il popup di conferma abbandono usa un design completamente **light** (`background: #F8F9FA`, testo scuro), in netto contrasto con il tema dark dell'applicazione. Questa scelta è intenzionale: la rottura del tema segnala visivamente all'utente che sta per compiere un'azione *distruttiva* e irreversibile (perdere le modifiche). Il rosso `#A33F39` (un bordeaux scuro ad alta leggibilità su bianco) è usato sia per il titolo che per il pulsante "Discard", creando coerenza semantica interna al popup.

Pulsante primario — gradient con filter: brightness

```
1 .hc-btn-primary {  
2   background: linear-gradient(135deg, #2aa8c8 0%, #1a8ba8 100%);  
3 }  
4 .hc-btn-primary:hover:not(:disabled) { filter:brightness(1.1);  
  ↪ transform:translateY(-1px); }  
5 .hc-btn-primary:disabled { filter:grayscale(0.5); }
```

Il pulsante "Confirm / Save" usa `filter: brightness` per l'hover anziché un secondo gradiente: questo permette di calcolare il colore di hover automaticamente dal gradiente base, senza dover dichiarare un secondo gradiente leggermente più chiaro. La stessa tecnica è usata per lo stato disabilitato: `filter: grayscale(0.5)` riduce la saturazione del gradiente teal senza cambiare colore.

16 Configurazione del progetto frontend

16.1 `src/api/semanticService.ts` — Layer di accesso al microservizio

16.1.1 Ruolo e posizione architetturale

`semanticService.ts` è l'unico punto di accesso del frontend al microservizio Python. Tutti i componenti React che hanno bisogno di dati (`WorkersPage`, `JobAnalysisView`, `HealthConditionWizard`) importano le funzioni da questo modulo anziché chiamare `fetch` direttamente. Questa centralizzazione è motivata da tre ragioni:

1. **Single point of change:** se l'URL base, la porta o l'autenticazione cambia, si modifica solo `BASE_URL` in questo file, non ogni componente che fa `fetch`.
2. **Type safety:** ogni funzione è tipizzata con i tipi corrispondenti ai modelli Pydantic del backend, documentati in `models.py`. TypeScript segnala immediatamente se un componente accede a un campo inesistente o con tipo sbagliato.
3. **Gestione errori centralizzata:** la funzione generica `apiFetch` traduce gli errori HTTP in eccezioni JavaScript con `status` e `body` strutturati, senza ripetere questa logica in ogni componente.

16.1.2 Costante `BASE_URL` e protocollo

```
1 const BASE_URL = 'http://127.0.0.1:8000';
```

L'indirizzo usa `127.0.0.1` (loopback IPv4 esplicito) anziché `localhost` per una ragione tecnica precisa: su sistemi con IPv6 abilitato, `localhost` si risolve in `::1` (loopback IPv6), mentre il microservizio Python è avviato con `-host 127.0.0.1` (IPv4). La mancata corrispondenza causerebbe un errore di connessione silenzioso. Usare esplicitamente `127.0.0.1` garantisce che il protocollo di rete corrisponda sempre.

La porta `8000` è la costante `PYTHON_PORT` di `electron/main.js`. Le due costanti non sono sincronizzate automaticamente: se la porta cambia in `main.js`, va aggiornata anche qui. Un miglioramento futuro sarebbe usare il canale IPC `get-python-port` per leggerla dinamicamente.

16.1.3 Tipi TypeScript — specchio dei modelli Pydantic

Il file dichiara un'interfaccia TypeScript per ogni modello Pydantic rilevante del backend. La corrispondenza è esatta per design:

Table 42: Corrispondenza tipi TypeScript ↔ modelli Pydantic.

TypeScript (semanticService.ts)	Pydantic (models.py)
ServiceStatus	StatusResponse
Worker	WorkerSummary
HealthCondition	HealthCondition
HealthConditionsResponse	HealthConditionsResponse
MatchResult	MatchResult
SkillDetail	SkillDetail
SkillDetailResponse	SkillDetailResponse
JobSkillEntry	JobSkillProfileEntry
SelectWorkerResponse	SelectWorkerResponse
IcfCodeEntry	IcfCodeEntry
HcChangeItem	HcChangeItem
UpdateHcResponse	UpdateHealthConditionsResponse
ImportWorkersResult	ImportResult (dataclass importer)
JobEntry	JobSummary
UpdateWorkerJobsResponse	UpdateWorkerJobsResponse

Il tipo `MatchResult.suitability` è dichiarato come union letterale TypeScript:

```
1 suitability: 'SUITABLE' | 'SUITABLE WITH PRECAUTIONS' | 'NOT
  ↳ SUITABLE';
```

Aniché `string`, l'union letterale permette al compilatore TypeScript di verificare staticamente ogni accesso al campo: se il backend aggiungesse un quarto valore, TypeScript segnalerebbe tutti i `switch/if` che non lo gestiscono.

16.1.4 Helper generico `apiFetch<T>`

```
1 async function apiFetch<T>(path: string, options?: RequestInit):
  ↳ Promise<T> {
2   const res = await fetch(`${BASE_URL}${path}`, {
3     headers: { 'Content-Type': 'application/json' },
4     ...options,
5   });
6   if (!res.ok) {
7     const body = await res.json().catch(() => ({ detail: res.
      ↳ statusText }));
```

```

8      throw Object.assign(
9        new Error(body?.detail?.message ?? body?.detail ?? res.
10          ↳ statusText),
11        { status: res.status, body }
12      );
13    }
14    return res.json() as Promise<T>;
15  }

```

Header Content-Type: `application/json`

L'header è impostato di default per tutte le richieste tramite lo spread `...options` che sovrascrive solo le opzioni fornite esplicitamente. L'eccezione è `importWorkers` che bypassa `apiFetch` completamente (usa `fetch` direttamente) perché deve impostare `application/octet-stream` e aggiungere `X-Filename`: l'header `Content-Type: application/json` di default di `apiFetch` sarebbe errato per un upload binario.

Gestione errori strutturata

FastAPI restituisce gli errori HTTP come JSON con campo `detail` (stringa o oggetto). La catena:

```

1 body?.detail?.message ?? body?.detail ?? res.statusText

```

gestisce tre casi: (1) `detail` è un oggetto con campo `message` (errori applicativi di `reasoner.py`); (2) `detail` è una stringa (errori HTTP standard di FastAPI); (3) il body non è JSON valido (fallback a `statusText`). L'`Object.assign` aggiunge `status` e `body` come proprietà dell'`Error`, permettendo ai componenti di distinguere tra un 404 (worker non trovato) e un 503 (Pellet non pronto) senza parsing del messaggio di testo.

16.1.5 Funzioni API — analisi per gruppo

Polling: `fetchStatus()`

```

1 export function fetchStatus(): Promise<ServiceStatus> {
2   return apiFetch<ServiceStatus>('/status');
3 }

```

Funzione di polling senza parametri. Il tipo di ritorno include `elapsed_pellet: number | null` (null prima che Pellet completi) e `stats: {...} | null` (stessa ragione). Il commento JSDoc *"Poll this until status === 'ready' before calling other endpoints"* documenta esplicitamente il contratto d'uso.

Selezione worker: `selectWorker()`


```

1 export function selectWorker(workerId: string): Promise<
  ↳ SelectWorkerResponse> {
2   return apiFetch<SelectWorkerResponse>('/workers/select', {
3     method: 'POST',
4     body: JSON.stringify({ worker_id: workerId }),
5   });
6 }

```

Il commento JSDoc di questa funzione è il più critico del file: *“Must be awaited BEFORE fetching match results so the SPARQL FILTER(?selected = true) sees the correct worker.”* Documenta esplicitamente il vincolo di ordinamento che WorkersPage rispetta nella sequenza handleSelectWorker.

Dettaglio skill: fetchSkillDetail()

```

1 export function fetchSkillDetail(workerId: string, jobId: string) {
2   return apiFetch<SkillDetailResponse>('/match/detail', {
3     method: 'POST',
4     body: JSON.stringify({ worker_id: workerId, job_id: jobId }),
5   });
6 }

```

Unica funzione che usa POST non per una mutazione ma per una query. Il commento JSDoc della classe Pydantic (MatchDetailRequest) spiega la ragione: la coppia (worker, job) è un “oggetto di ricerca” non adatto ai query parameters GET.

Import file binario: importWorkers()

```

1 export async function importWorkers(
2   fileBytes: number[], fileName: string
3 ): Promise<ImportWorkersResult> {
4   const body = new Uint8Array(fileBytes);
5
6   const res = await fetch(`${BASE_URL}/import/workers`, {
7     method: 'POST',
8     headers: {
9       'Content-Type': 'application/octet-stream',
10      'X-Filename': fileName,
11    },
12    body,
13  });
14  ...
15 }

```

Questa funzione bypassa apiFetch perché l’upload binario ha tre requisiti incompatibili con il helper generico:

1. Content-Type: application/octet-stream anziché application/json.

2. Header aggiuntivo **X-Filename** per trasmettere il nome del file (il backend lo usa per il logging e per il backup).
3. Body come **Uint8Array** anziché stringa JSON.

La conversione `new Uint8Array(fileBytes)` trasforma il `number[]` ricevuto da Electron IPC (`readFileBuffer`) in un buffer binario che `fetch` accetta come body. Questo chiude il ciclo del flusso import: Electron legge il file come `Buffer` Node.js → serializzato come `number[]` attraverso IPC → ricostruito come `Uint8Array` → inviato come body HTTP binario.

Mutazione job: `updateWorkerJobs()`

```

1 export function updateWorkerJobs(workerId: string, jobIds: string
  ↳ []) {
2   return apiFetch<UpdateWorkerJobsResponse>(
3     '/workers/${encodeURIComponent(workerId)}/jobs/update',
4     { method: 'POST', body: JSON.stringify({ job_ids: jobIds }) }
5   );
6 }

```

Il commento JSDoc specifica: *“Triggers a full Pellet re-run on the backend so match results are refreshed.”*. Questo allinea la documentazione TS con il comportamento di `_refresh_reasoning_artifacts()` in `reasoner.py`, rendendo esplicito il costo dell'operazione.

16.1.6 Uso di `encodeURIComponent`

Tutte le funzioni che inseriscono ID utente negli URL usano `encodeURIComponent`:

```

1 '/health-conditions/${encodeURIComponent(workerId)}'
2 '/workers/${encodeURIComponent(workerId)}/jobs/update'
3 '/jobs/${encodeURIComponent(jobId)}/profile'

```

I local name degli individui OWL (es. `Job_AssemblyWorker`, `des_MarioBianchi_b110`) possono contenere caratteri che hanno significato speciale negli URL (`_`, `/`, `#`). Sebbene i nomi correnti non richiedano encoding, l'uso sistematico di `encodeURIComponent` è una misura preventiva: se un futuro dataset importasse worker con apostrofi o caratteri accentati (come gestisce già `_sanitize_id()` in `importer.py`), il frontend non produrrebbe URL malformati.

16.2 package.json — Dipendenze e configurazione build

16.2.1 Metadata e entry point

```

1 {
2   "name": "rientra-app",

```

```

3  "version": "1.0.0",
4  "main": "electron/main.js",
5  "description": "RIENTR@ returns -- Decision Support System
   ↪ desktop application"
6  }

```

Il campo "main": "electron/main.js" è il campo che Electron usa per trovare il processo principale. Questo campo è specifico di Electron: npm lo ignora per applicazioni web normali, ma `electron .` lo legge per avviare il processo.

16.2.2 Script npm — flusso di sviluppo e build

Table 43: Script npm con motivazione di ogni scelta.

Script	Comando e motivazione
dev:vite	<code>vite</code> — avvia il dev server Vite su porta 5173 con Hot Module Replacement.
dev:electron	<code>cross-env NODE_ENV=development electron .</code> — avvia Electron con la variabile d'ambiente che attiva <code>win.loadURL('http://localhost:5173')</code> anziché <code>loadFile</code> .
dev	<code>concurrently -k "dev:vite" "wait-on ... && dev:electron"</code> — avvia entrambi i processi in parallelo. <code>wait-on</code> attende che Vite sia pronto (risponda su porta 5173) prima di avviare Electron, evitando che Electron tenti di caricare una pagina non ancora servita. <code>-k</code> termina entrambi i processi quando uno esce.
build	<code>tsc -b && vite build</code> — type-checking completo (<code>tsc -b</code> per build references) seguito dal bundle di produzione. Il flag <code>-b</code> (build mode) gestisce i riferimenti tra <code>tsconfig.app.json</code> e <code>tsconfig.node.json</code> .
build:app	<code>npm run build && electron-builder</code> — crea il bundle React poi lo impacchetta con electron-builder in un installer nativo (DMG, NSIS, AppImage).
lint	<code>eslint .</code> — controllo qualità del codice su tutti i file TypeScript.
preview	<code>vite preview</code> — serve il bundle di produzione localmente per verificarlo prima del packaging.

Perché cross-env e wait-on

cross-env: il comando `NODE_ENV=development electron .` fallisce su Windows perché CMD non supporta la sintassi `VAR=value`. **cross-env** normalizza il comportamento su tutte le piattaforme.

wait-on: senza l'attesa, Electron si avvia prima che Vite abbia completato la compilazione iniziale. Il renderer tenterebbe di caricare `http://localhost:5173` ricevendo un errore di connessione rifiutata, con conseguente schermata bianca.

wait-on esegue un polling HTTP su `http://localhost:5173` ogni 250ms fino alla risposta, poi sblocca il comando successivo.

16.2.3 Dipendenze di produzione

Table 44: Dipendenze di produzione e motivazioni.

Pacchetto	Versione	Motivazione
<code>react</code>	<code>^19.2.4</code>	React 19 con Concurrent Mode; versione più recente al momento dello sviluppo.
<code>react-dom</code>	<code>^19.2.4</code>	Renderer DOM di React; deve sempre corrispondere alla versione di <code>react</code> .
<code>d3-force</code>	<code>^3.0.0</code>	Simulazione fisica per grafi di forza (layout automatico dei nodi). Presente ma non visibile nei file analizzati: potrebbe essere usata da <code>react-force-graph-2d</code> o per future viste a grafo dell'ontologia.
<code>react-force-graph-2d</code>	<code>^1.29.1</code>	Visualizzazione interattiva di grafi 2D. Non appare nei componenti documentati: verosimilmente una vista ontologia in sviluppo o un'esplorazione prototipale.

Nota — dipendenze non usate nei file analizzati

`d3-force` e `react-force-graph-2d` sono nelle dipendenze di produzione ma non importate nei componenti documentati (`HomePage`, `WorkersPage`, `JobAnalysisView`, `HealthConditionWizard`). Potrebbero essere usate in componenti non ancora inviati, oppure sono residui di una funzionalità in sviluppo.

In entrambi i casi, se non usate in produzione, andrebbero spostate in `devDependencies` o rimosse per ridurre le dimensioni del bundle.

16.2.4 Dipendenze di sviluppo chiave

Table 45: DevDependencies chiave con motivazioni.

Pacchetto	Versione	Motivazione
<code>electron</code>	<code>^41.1.0</code>	Electron 41 bundla Chromium 130 e Node.js 22. La versione determina le API CSS e JavaScript disponibili nel renderer.
<code>electron-builder</code>	<code>^26.8.1</code>	Packaging per distribuzione: crea DMG (macOS), NSIS installer (Windows), AppImage (Linux).
<code>vite</code>	<code>^8.0.1</code>	Bundler con HMR. Versione 8 (ultima al momento).
<code>typescript</code>	<code>~5.9.3</code>	Il tilde (~) anziché il caret (^) fissa la patch version: aggiorna solo bug fix, non feature. TypeScript è un compilatore: una feature di minor version potrebbe cambiare il comportamento del type checker, causando errori inattesi.
<code>@vitejs/plugin-react</code>	<code>^6.0.1</code>	Integra Babel/SWC per la trasformazione JSX e Fast Refresh in Vite.
<code>concurrently</code>	<code>^9.2.1</code>	Esecuzione parallela di processi npm.
<code>wait-on</code>	<code>^9.0.4</code>	Polling HTTP/TCP per sincronizzazione processi.
<code>cross-env</code>	<code>^10.1.0</code>	Variabili d'ambiente cross-platform.

16.2.5 Configurazione electron-builder

```

1 "build": {
2   "appId": "it.cnr.stiima.rientra",
3   "productName": "RIENTR@ returns",
4   "files": ["dist/**", "electron/**", "package.json"],
5   "mac": { "target": "dmg", "category": "public.app-category.
  ↪ productivity" },

```

```

6   "win":    { "target": "nsis" },
7   "linux": { "target": "AppImage" }
8 }

```

L'appId segue la convenzione DNS inverso (`it.cnr.stiima.rientra`), necessaria per identificare univocamente l'applicazione nei sistemi operativi (macOS App Store, Windows Registry, Linux desktop file).

I files specificano cosa viene incluso nel pacchetto: `dist/**` (bundle React), `electron/**` (main + preload), `package.json` (necessario per "main": "electron/main.js"). **Non** include `src/**`, `node_modules/**` (le dipendenze di produzione vengono ri-installate), `python-service/` (da distribuire separatamente o tramite installer dedicato).

Nota — python-service non nel bundle

La directory `python-service/` non è nei files di electron-builder. Questo significa che la distribuzione dell'app **non include** il microservizio Python: l'utente finale deve installarlo separatamente. Per una distribuzione *self-contained* occorrerebbe aggiungere `python-service/**` e gestire il venv/Python bundlato tramite la sezione `extraResources` di electron-builder.

16.3 vite.config.ts — Configurazione del bundler

```

1 import { defineConfig } from 'vite'
2 import react from '@vitejs/plugin-react'
3
4 export default defineConfig({
5   plugins: [react()],
6   base: './', // path relativo per file://
7   ↪ protocol
8   build: { outDir: 'dist', emptyOutDir: true },
9   server: { port: 5173 },
10 })

```

base: './' — la scelta più critica

Il campo `base` controlla come Vite genera i percorsi degli asset nel bundle di produzione. Con il default (`base: '/'`), Vite genera percorsi assoluti come `<script src="/assets/index.js">`. Questi percorsi funzionano su un server HTTP ma **falliscono** con il protocollo `file://` di Electron, perché il browser interpreterebbe `/assets/index.js` come radice del filesystem.

`base: './'` genera percorsi relativi (`./assets/index.js`) che funzionano sia con `file://` (produzione Electron) sia con `http://` (dev server). Il commento nel

codice lo documenta esplicitamente: *“Use './' so Electron can load assets with file:// protocol in production”*.

`emptyOutDir: true`

Svuota la directory `dist/` ad ogni build prima di scrivere i nuovi file. Previene la presenza di file obsoleti da build precedenti che potrebbero essere inclusi nel pacchetto `electron-builder`.

`port: 5173`

Porta fissa per il dev server, coerente con le origini CORS di `main.py` (`http://localhost:5173`) e con il comando `wait-on` in `package.json`.

`plugins: [react()]`

Il plugin `@vitejs/plugin-react` abilita: trasformazione JSX tramite Babel, Fast Refresh (HMR per React senza perdita di stato), decoratori se configurati. Non viene usato `@vitejs/plugin-react-swc` (versione SWC più veloce) perché la compatibilità con le versioni attuali di Electron e TypeScript è più consolidata con Babel.

16.4 tsconfig.json — Configurazione TypeScript

```
1 {  
2   "files": [],  
3   "references": [  
4     { "path": "./tsconfig.app.json" },  
5     { "path": "./tsconfig.node.json" }  
6   ]  
7 }
```

Il file radice `tsconfig.json` è intenzionalmente vuoto (`"files": []`) e funge solo da aggregatore di riferimenti. Questo è il pattern TypeScript **Project References** (introdotto in TypeScript 3.0): separa la configurazione del codice browser (`tsconfig.app.json`: JSX, React, DOM types, moduli ESNext) da quella del codice Node.js (`tsconfig.node.json`: CommonJS o ESM, tipi Node.js, nessun DOM).

Perché Project References per Electron

In un progetto Electron coesistono due ambienti JavaScript radicalmente diversi:

- **Renderer** (`src/`): browser API, JSX, ESM, `@types/react`, DOM types.
- **Main process** (`electron/`): Node.js API, CommonJS, `@types/node`, nessun DOM.

Usare un unico `tsconfig.json` con entrambi i tipi (DOM + Node) causa falsi

positivi: TypeScript vedrebbe `process` e `require` come globali nel codice React, e `window` e `document` nel codice Electron. I Project References risolvono questo isolando la type-checking per contesto. Il comando `tsc -b` nello script `build` compila entrambi i riferimenti in sequenza, rispettando le dipendenze tra progetti.

16.5 `eslint.config.js` — Qualità del codice

```
1 export default defineConfig([
2   globalIgnores(['dist']),
3   {
4     files: ['**/*.ts', '**/*.tsx'],
5     extends: [
6       js.configs.recommended,
7       tseslint.configs.recommended,
8       reactHooks.configs.flat.recommended,
9       reactRefresh.configs.vite,
10    ],
11    languageOptions: { ecmaVersion: 2020, globals: globals.browser
12      ↪ },
13  },
14 ])
```

Il file usa il **nuovo formato flat config** di ESLint 9 (introdotto in ESLint 8.21 come sperimentale, default da ESLint 9). Rispetto al formato legacy (`.eslintrc.json`), il flat config è un modulo JavaScript puro: più componibile, senza magia di caricamento e compatibile con ESM.

Table 46: Plugin ESLint configurati e motivazioni.

Plugin / Config	Motivazione
<code>js.configs.recommended</code>	Regole JavaScript base (no <code>==</code> , no <code>var</code> , ecc.).
<code>tseslint.configs.recommended</code>	Regole TypeScript: no <code>any</code> implicito, no non-null assertion non necessarie, tipi corretti su funzioni.
<code>reactHooks.configs.flat.recommended</code>	Regole critiche per React Hooks: <code>rules-of-hooks</code> (hooks solo al top level), <code>exhaustive-deps</code> (tutte le dipendenze dichiarate negli array di effect). Quest'ultima regola previene i bug di stale closure documentati nella sezione <code>JobAnalysisView</code> .
<code>reactRefresh.configs.vite</code>	Verifica che i componenti siano compatibili con Fast Refresh: solo export default per componente, nessun export misto di componenti e non-componenti nello stesso file.

`globalIgnores(['dist'])` esclude il bundle di produzione dalla linting: i file generati da Vite non devono essere analizzati (contengono codice minificato non leggibile) e includono import transforms che ESLint interpreterebbe come errori.

16.6 `start.sh` — Script di avvio rapido

```

1  #!/bin/bash
2  set -e
3
4  DIR="$(cd "$(dirname "$0")" && pwd)"
5  cd "$DIR"
6
7  # Installa le dipendenze se mancanti
8  if [ ! -d "node_modules/electron" ]; then
9      npm install
10 fi
11
12 export NPM_CONFIG_CACHE=/tmp/npm-cache-rientra
13 export NODE_ENV=development
14 npm run dev

```

Listing 91: `start.sh` — contenuto completo.

`start.sh` è uno script di convenienza per avviare l'ambiente di sviluppo con un singolo comando da terminale o da iTerm2. Analisi delle scelte:

- `set -e`: termina lo script immediatamente se qualsiasi comando fallisce (es. `npm install` con errore di rete). Senza questo flag, lo script continuerebbe ad eseguire `npm run dev` anche con dipendenze incomplete.
- `DIR="$(cd "$(dirname "$0")" && pwd)":` calcola il percorso assoluto della directory dello script, indipendentemente da dove viene invocato. Questo permette di eseguire `./start.sh` da qualsiasi directory senza che i percorsi relativi a `node_modules` risultino errati.
- `if [ ! -d "node_modules/electron" ]:` verifica la presenza di Electron (pacchetto più grande e indicativo di un'installazione completa) invece di controllare l'intera `node_modules`. Un check su `node_modules` esisterebbe sempre dopo il primo install parziale.
- `NPM_CONFIG_CACHE=/tmp/npm-cache-rientra`: usa una cache npm dedicata in `/tmp` per evitare conflitti con la cache globale di npm e per garantire che la cache venga ripulita al riavvio del sistema.

16.7 Script di utilità — `replace.js`, `fix_box.py`, `restructure.py`

Questi tre file sono **script di patch una-tantum** usati durante lo sviluppo per applicare modifiche strutturali ai componenti React. Non fanno parte del processo di build né di runtime.

16.7.1 `replace.js` — patch dell'espansore skill

```

1 // PRIMA (struttura piatta):
2 <td className="ja-td-skill" style={{ color: col }}>
3   <span className="ja-skill-name">{s.id.replace(/_/g, ' ')}</span>
4   {hasDesc && (<span className='{ja-desc-toggle...}'>...</span>)}
5 </td>
6
7 // DOPO (struttura con div e freccia SVG assoluta):
8 <td className="ja-td-skill" style={{ color: col }}>
9   <div className="ja-skill-name-cell">
10     {hasDesc && (
11       <span className="ja-row-expander-arrow ...">
12         <svg>...</svg>    {/* chevron animato */}
13       </span>

```

```
14     })
15     <span className="ja-skill-name">{s.id.replace(/_/g, ' ')}</span
    ↪ >
16 </div>
17 </td>
```

Listing 92: `replace.js` — obiettivo della patch.

Lo script sostituisce il vecchio toggle testuale (▲/▼) con una freccia SVG animata posizionata in `absolute` a sinistra della cella, come documentato nella sezione CSS di `JobAnalysisView.css`. Gestisce entrambi i terminatori di riga (CRLF e LF) perché il file sorgente potrebbe avere terminatori misti a seconda del sistema su cui è stato modificato.

La motivazione di usare uno script Node.js invece di modificare il file manualmente è la riproducibilità: lo script è deterministico e può essere rieseguito dopo un merge che ripristina la versione precedente.

16.7.2 `fix_box.py` — ristrutturazione layout wizard

Lo script rimuove un wrapper `<div className="hc-wizard-box">` che conteneva sia la toolbar che il contenitore scrollabile, e lo sposta a racchiudere *solo* il contenitore scrollabile. La motivazione è un bug di layout: con il box che avvolgeva anche la toolbar, la toolbar veniva inclusa nel calcolo dello scroll interno, causando un comportamento visivo scorretto (la toolbar spariva durante lo scroll del catalogo ICF).

La patch usa regex Python sul testo del file TSX piuttosto che un AST parser come Babel, per semplicità: la struttura da modificare è abbastanza unica nel file da non rischiare false corrispondenze.

16.7.3 `restructure.py` — spostamento header wizard

Lo script estrae il blocco `hc-wizard-header` dalla posizione originale (fuori dalla toolbar) e lo reinserisce all'interno della toolbar di ogni step (select e review), aggiungendo anche il pulsante di chiusura. L'obiettivo è avere header specifici per step ("*Choose New Codes*" / "*Review Changes*") invece di un header generico condiviso.

Entrambi gli script Python usano `re.sub` con pattern che includono `re.DOTALL` per attraversare i newline, necessario per i blocchi JSX multi-riga. Il pattern 1 nei replacement string mantiene i gruppi catturati `(...)` consentendo di inserire nuovo markup intorno all'elemento esistente senza riscriverlo.

Appendice A — Struttura sintetica di main.py

```

main.py
|-- Docstring modulo (architettura, run command)
|-- Import (stdlib + fastapi + moduli interni)
|
|-- lifespan()                                # @asynccontextmanager
|   |-- ensure_mapping()                      # genera rientra_mapping.ttl
|   |-- load_snapshot_cache()                 # pre-carica cache
|   \-- Thread(load_and_reason)               # avvia Pellet in background
|
|-- app = FastAPI(...)                        # istanza principale
|-- app.add_middleware(CORS)                  # origini consentite
|
|-- _require_ready()                          # guard 503/500
|
|-- GET    /status
|-- GET    /workers
|-- GET    /workers/{id}
|-- GET    /jobs
|-- GET    /health-conditions/{id}
|-- GET    /jobs/{id}/importance
|-- GET    /match
|-- GET    /match/{id}
|-- POST   /match/detail
|-- GET    /jobs/{id}/profile
|-- POST   /workers/select
|-- GET    /icf-codes
|-- GET    /core-sets
|-- POST   /health-conditions/{id}/update
|-- POST   /workers/{id}/jobs/update
|-- POST   /import/workers
|
\-- if __name__ == "__main__": uvicorn.run(...)

```

Listing 93: Pseudo-outline di main.py.

Appendice B — Struttura sintetica di models.py

```
models.py
|-- Import: __future__, typing, pydantic
|
|-- [Status]
|   |-- OntologyStats
|   \-- StatusResponse      <-- contiene OntologyStats
|
|-- [Workers]
|   |-- WorkerSummary
|   |-- WorkerDetail
|   |-- SelectWorkerRequest
|   |-- SelectWorkerResponse
|   |-- UpdateWorkerJobsRequest
|   \-- UpdateWorkerJobsResponse
|
|-- [Jobs]
|   |-- JobSummary
|   |-- JobSkillProfileEntry
|   |-- SkillScore          <-- ausiliario di ImportanceEntry
|   \-- ImportanceEntry     <-- contiene list[SkillScore]
|
|-- [Matching]
|   |-- MatchResult
|   |-- SkillDetail
|   |-- SkillDetailResponse  <-- contiene list[SkillDetail]
|   \-- MatchDetailRequest
|
|-- [ICF Catalogue]
|   \-- IcfCodeEntry
|
|-- [Health Conditions Update]
|   |-- HealthCondition
|   |-- HealthConditionsResponse <-- contiene list[HealthCondition
↪ ]
|   |-- HcChangeItem
|   |-- UpdateHealthConditionsRequest <-- contiene list[
↪ HcChangeItem]
|   \-- UpdateHealthConditionsResponse
|
\-- [Worker Jobs Update]
    |-- UpdateWorkerJobsRequest
    \-- UpdateWorkerJobsResponse
```

Listing 94: Pseudo-outline di models.py.

Appendice C — Struttura sintetica di reasoner.py

```

reasoner.py
|-- Import stdlib + owlready2 (con ImportError guard)
|-- Silenziamento log (warnings, logging, owlready2)
|
|-- [Costanti IRI ontologia Rientra@]
|   |-- IRI_IS_VERY_IMP / IRI_IS_IMP / IRI_IS_SOMEWHAT /
↪ IRI_IS_LESS
|   |-- IRI_HAS_CRIT, IRI_REQUIRES, IRI_CONCERNS, IRI_HAS_SCORE
|   |-- IRI_IS_EVAL_JOB, IRI_IS_IN_HC, IRI_IS_SELECTED,
↪ IRI_IS_TRANSL
|   |-- IRI_BFQUAL, IRI_APIQUAL, IRI_FIRST_NAME, IRI_SURNAME
|   |-- IRI_IS_DESCRIBED_BY, IRI_INVOLVES_ICF
|   |-- IRI_ICF_DESCRIPTION, IRI_ONET_DEFINITION
|   |-- JS_RED_INTERCEPT, JS_YELLOW_INTERCEPT, JS_SLOPE (soglie
↪ idoneit.)
|   |-- IMPORTANCE_IRIS: dict label -> IRI
|   |-- \-- SUITABILITY_COLORS: dict label -> hex color
|
|-- [Stato globale]
|   |-- class ReasonerState                # singleton thread-safe
|   |   |-- set_ready()
|   |   |-- set_ready_from_cache()
|   |   |-- set_loading()
|   |   |-- set_error()
|   |   |-- \-- is_ready()
|   |-- state = ReasonerState()            # singleton esposto a main.py
|   |-- _selection_lock                    # threading.Lock
|   |-- _icf_core_set_map                   # dict[str, list[str]]
|   |-- _snapshot_cache                     # dict / None
|   |-- _live_reasoner_ready                # bool
|   |-- _pending_selected_worker_id         # Optional[str]
|   |-- \-- _loaded_ontology                # oggetto owlready2 / None
|
|-- [Mappa ICF -> Core Set]
|   |-- IRI_ICF_CORE_SET
|   |-- _build_icf_core_set_map()           # SPARQL subclassOf+
|   |-- \-- get_all_core_sets()             # API pubblica
|
|-- [Utility]
|   |-- local_name(entity)                  # frammento IRI
|   |-- criticality_label(cs)               # Fig. 2 paper
|   |-- job_suitability(gcs, aisa)          # Fig. 4 paper -> (label,
↪ color)
|   |-- _score_to_anchor(score)             # Tab. 2 paper: 0*NET ->
↪ [0-3]

```

```

|   |-- _sparql(query)                                # wrapper default_world.
↪ sparql
|   |-- _find_rdf_file()                               # auto-detection
|   |-- _resolve_rdf_path(rdf_path)                   # arg / env / autodetect
|   |-- _ontology_fingerprint(path)                   # SHA-256 size+mtime
|   |-- _snapshot_cache_path(path)                   # percorso file cache
|   |-- _use_snapshot_cache()                         # bool: usa cache?
|   \-- is_live_reasoner_ready()                     # bool pubblico
|
|-- [Cache snapshot]
|   |-- load_snapshot_cache()                         # carica cache JSON da disco
|   |-- _save_snapshot_cache()                       # precalcola tutto, salva
↪ JSON
|   |-- _save_live_ontology()                         # owlready2 -> file RDF
|   \-- inject_imported_workers()                   # rdflib inject + cache
↪ refresh
|
|-- _refresh_reasoning_artifacts()                   # salva RDF, riesegue Pellet,
|                                                    # ricalcola cache
|-- [Fase 1: Load]
|   \-- _load_ontology(path)                         # get_ontology().load()
|
|-- [Fase 2: Pellet]
|   \-- _run_pellet(onto)                            # sync_reasoner_pellet con
|                                                    # monkey-patch anti-ciclo
|
|-- load_and_reason(rdf_path)                       # entry point pubblico (
↪ thread)
|
|-- [Query Q0]
|   |-- get_workers()                                # Person con
↪ isEvaluatedForJob
|   \-- get_jobs()                                   # Job con requires
|
|-- [Query HC]
|   |-- get_health_conditions(id)                    # ICF + qualificatori per
↪ worker
|   \-- get_all_icf_codes()                          # catalogo ICF completo (
↪ wizard)
|
|-- [Query Q4]
|   |-- get_importance_summary(job)                  # is*ImportantFor (SWRL 9-12)
|   \-- get_job_skill_profile(job)                  # profilo O*NET puro
|
|-- [Query Q1+Q2]
|   |-- get_match_results(worker)                   # GCS% + AISA%
|   \-- _match_results_fallback()                   # via hasSpecificCriticality

```

```

|    \-- _compute_metrics(agg)          # aggregazione -> risultati
|
|-- [Query Q3]
|    \-- get_skill_detail(worker, job) # CS dettaglio per coppia
|
|-- [Mutazione: selezione]
|    \-- set_selected_worker(id)        # flip isSelected (thread-
↳ safe)
|
|-- [Mutazione: HC]
|    |-- _hc_lock                      # threading.Lock
|    |-- _icf_ind_cache                # dict[str, object]
|    |-- _build_icf_ind_cache()        # popola cache ICF
↳ individuals
|    |-- _resolve_icf_individual(code) # cache -> wildcard -> scan
|    |-- _is_api_code(code)           # d-prefix -> APIqual
|    \-- update_health_conditions()    # batch ADD/REMOVE/MODIFY
|
\-- [Mutazione: job]
    |-- _job_lock                      # threading.Lock
    \-- update_worker_jobs()           # sostituisce
↳ isEvaluatedForJob

```

Listing 95: Pseudo-outline di `reasoner.py`.

Appendice D — Tabella delle query SPARQL

Query	Funzione	Proprietà chiave	Output
Q0-W	<code>get_workers()</code>	<code>isEvaluatedForJob</code>	Lista Person
Q0-J	<code>get_jobs()</code>	<code>requires</code>	Lista Job
Q-HC	<code>get_health-conditions()</code>	<code>isInHealthCondition</code> , <code>isDescribedBy</code> , <code>involvesICFCode</code> , <code>BFqual</code> , <code>AP1qual</code>	Condizioni ICF per worker
Q-ICF	<code>get_all_icf_codes()</code>	<code>involvesICFCode</code> , <code>description</code>	Catalogo ICF
Q-CS	<code>_build_icf_core_set_map()</code>	<code>rdfs:subClassOf+</code>	Mappa ICF → core set
Q4	<code>get_importance-summary()</code>	<code>is*ImportantFor</code> (SWRL 9-12), <code>concerns</code> , <code>hasScore</code>	Skill per livello imp.
Q-SP	<code>get_job_skill-profile()</code>	<code>requires</code> , <code>concerns</code> , <code>hasScore</code>	Profilo O*NET job
Q1+Q2	<code>get_match_results()</code>	<code>isSelected</code> , <code>isEvaluatedForJob</code> , <code>isTranslatedWithICFCode</code> , <code>BFqual</code> , <code>AP1qual</code>	GCS%, AISA%, idoneità
Q1+Q2b	<code>_match_results-fallback()</code>	<code>hasSpecificCriticality</code>	GCS%, AISA% (fallback)
Q3	<code>get_skill_detail()</code>	<code>hasSpecificCriticality</code> , <code>concerns</code> , <code>hasScore</code> , <code>ONet_def</code>	CS dettaglio per (w,j)
Q-ICF-I	<code>_build_icf_ind_cache()</code>	<code>involvesICFCode</code>	Cache ind. ICF

Appendice E — Namespace IRI dell'ontologia

Prefisso	IRI base	Contenuto
(default)	.../RientraReturns#	Box principale
SkAb	.../SkAb#	Skills & Abilities O*NET
ICF-	.../ICF-exc-coreset#	Estratto ICF + core set
JobL	.../JobList#	Profili lavorativi
JobD	.../JobDescription#	Descrizioni job
Rien	.../RientraOnt3#	Ontologia Rientr@ v3
Rien2	.../RientraOnt3Merged#	Core set aggiuntivi
Rien3	.../RientraHC#	Health Condition
FOAF	.../FOAF-excerpt#	Estratto FOAF (Person)
swrl	.../swrl#	SWRL standard W3C
owl	.../owl#	OWL 2 standard W3C
rdf	.../rdf-syntax-ns#	RDF standard W3C
rdfs	.../rdf-schema#	RDFS standard W3C
xsd	.../XMLSchema#	Tipi XSD

```

importer.py (package: import_workers/)
|
|-- [Import condizionale con _DEPS_OK guard]
|   |-- morph_kgc, rdflib, rdflib.namespace
|   \-- stub _SAFE_ANY se ImportError
|
|-- [Namespace rdflib]
|   |-- NS_FOAF      (.../FOAF-excerpt#)
|   |-- NS_HC       (.../RientraHC#)

```

```

| |-- NS_ICF      (.../ICF-exc-coreset#)
| |-- NS_PERSON  (.../Person-CommonBox#)
| |-- NS_JOB     (.../JobList#)
| \-- NS_RIEONT3 (.../RientraOnt3#)
|
|-- [Costanti di modulo]
| |-- CLOSING_TAG = "</rdf:RDF>"
| \-- QUALIFIER_MAP = {b: BFqual, d: AP1qual, s: BS1qual}
|
|-- @dataclass ImportResult
| |-- persons_added, persons_skipped
| |-- icf_valid, icf_skipped
| |-- jobs_valid, jobs_skipped
| |-- backup_path, new_person_ids, skipped_ids
| \-- error: Optional[str]
|
|-- [Utility]
| |-- _sanitize_id(name)           # regex [^A-Za-z0-9_-] -> _
| |-- _person_exists(rdf_text, id) # ricerca IRI nel testo
↪ grezzo
| |-- _extract_known_icf(rdf_text) # regex sul testo -> set[
↪ str]
| \-- _extract_known_jobs(onto_path) # rdflib, 3 pattern (
↪ punning)
|
|-- [Database SQLite in-memory]
| |-- _load_sql(sql_path)           # connect(":memory:") +
↪ executescript
| |-- _export_db_to_file(conn)       # iterdump() -> file .db
↪ temporaneo
| \-- _create_views(conn, icf, jobs) # VIEW ausiliarie + tabelle
↪ controllo
| |-- CREATE TABLE _known_icf      # codici ICF validi (da
↪ ontologia)
| |-- CREATE TABLE _known_jobs     # job validi (da ontologia)
| |-- CREATE VIEW v_desc_b          # codici b* -> BFqual
| |-- CREATE VIEW v_desc_d          # codici d* -> AP1qual
| |-- CREATE VIEW v_desc_s          # codici s* -> BS1qual
| |-- CREATE VIEW v_hc_links        # UNION b+d+s ->
↪ isDescribedBy
| \-- CREATE VIEW v_job_valid       # job nel SQL che esistono
↪ nell'onto
|
|-- [Mapping R2RML]
| |-- _generate_mapping(onto, map)  # scoperta auto prop. OWL
↪ -> .ttl
| |-- Parsing rdflib dell'ontologia

```

```

|   |   |-- Classificazione prop: plain / typed / boolean
|   |   |-- Correzione alias ZIPcode -> zip_code
|   |   |-- Scrittura PersonMap, HealthConditionMap
|   |   |-- Scrittura DescriptorMapB/D/S, DescriptorLinkMap
|   |   \-- Scrittura JobEvaluationMap
|   \-- _run_r2rml(mapping_path, db_file) # morph_kgc.materialize
↪ ()
|
|-- [Conversione triple -> RDF/XML]
|   \-- _triples_to_rdfxml_blocks(graph, ids) -> list[str]
|       |-- Blocchi HC_Descriptor (uno per ICF valido)
|       |-- Blocco Health_Condition (con isDescribedBy)
|       \-- Blocco Person (anagrafica + isInHealthCondition
|               + isEvaluatedForJob + isSelected=false)
|           \-- xml_escape() su tutti i valori stringa
|
|-- [Entry point pubblico per FastAPI]
|   \-- import_sql_dataset(sql, onto, mapping) -> ImportResult
|       |-- _load_sql()
|       |-- read_text(onto) + _extract_known_icf() +
↪ _extract_known_jobs()
|       |-- Calcolo new_ids (pazienti non ancora in ontologia)
|       |-- Early exit se new_ids e' vuoto
|       |-- _create_views()
|       |-- _export_db_to_file() -> _run_r2rml() [finally: unlink
↪ .db]
|       |-- _triples_to_rdfxml_blocks()
|       |-- shutil.copy2() backup .bak
|       |-- str.replace(CLOSING_TAG) + write_text()
|       \-- return ImportResult (con contatori, ids, backup_path)
|
\-- [Entry point pubblico per lifespan FastAPI]
    \-- ensure_mapping(onto_path, map_path) -> None
        |-- Se mapping esiste: no-op (idempotente)
        \-- Altrimenti: _generate_mapping()

```

Listing 96: Pseudo-outline di importer.py.

Appendice G — Schema SQL del dataset di import

```

1  -- =====
2  -- Rientr@ Import Dataset Schema
3  -- Versione: 0.1-test / Aprile 2026 / STIIMA-CNR
4  -- =====
5
6  -- TABELLA 1: Dati anagrafici del paziente (OBBLIGATORIA)
7  CREATE TABLE IF NOT EXISTS person (
8      person_id TEXT PRIMARY KEY, -- frammento IRI (es.
9      ↪ MarioBianchi)
10     first_name TEXT NOT NULL,
11     surname TEXT NOT NULL,
12     TIN TEXT, -- codice fiscale (opzionale)
13     birthday TEXT, -- formato YYYY-MM-DDTHH:MM:
14     ↪ SS
15     city TEXT,
16     country TEXT,
17     zip_code INTEGER -- xsd:int (NULL ammesso)
18 );
19
20 -- TABELLA 2: Profilo ICF del paziente (OBBLIGATORIA, anche vuota)
21 -- I codici e* vengono automaticamente ignorati.
22 -- I codici non presenti nell'ontologia vengono ignorati.
23 CREATE TABLE IF NOT EXISTS hc_descriptor (
24     person_id TEXT NOT NULL REFERENCES person(person_id),
25     icf_code TEXT NOT NULL, -- es. b110, d175, s710 (non
26     ↪ e*)
27     qualifier INTEGER NOT NULL CHECK (qualifier BETWEEN 0 AND 4),
28     PRIMARY KEY (person_id, icf_code)
29 );
30
31 -- TABELLA 3: Job da valutare per il matching (OPZIONALE)
32 -- Se assente, nessun link isEvaluatedForJob viene creato.
33 -- I job_id non presenti nell'ontologia vengono ignorati.
34 CREATE TABLE IF NOT EXISTS job_evaluation (
35     person_id TEXT NOT NULL REFERENCES person(person_id),
36     job_id TEXT NOT NULL, -- es. Carpenter, FileClerk
37     PRIMARY KEY (person_id, job_id)
38 );
39
40 -- ESEMPIO DI POPOLAMENTO
41 INSERT INTO person VALUES (
42     'MarioBianchi', 'Mario', 'Bianchi',
43     'BNCMRA80A01H501Z', '1980-01-01T00:00:00',
44     'Milano', 'Italia', 20100
45 );

```

```
43 INSERT INTO hc_descriptor VALUES
44     ('MarioBianchi', 'b130', 2),
45     ('MarioBianchi', 'd175', 1),
46     ('MarioBianchi', 'b280', 3);
47 INSERT INTO job_evaluation VALUES
48     ('MarioBianchi', 'Carpenter'),
49     ('MarioBianchi', 'FileClerk');
```

Listing 97: Schema completo del dataset SQL di import (v0.1-test).

Appendice H — Struttura sintetica dei file Electron

```

electron/main.js                                     (Node.js -- Main Process)
|
|-- [Dipendenze]
|   |-- electron: app, BrowserWindow, ipcMain, dialog, Menu
|   |-- child_process: spawn
|   \-- fs, path (stdlib Node.js)
|
|-- [Costanti]
|   |-- isDev = (NODE_ENV === 'development')
|   |-- PYTHON_PORT = 8000
|   |-- SERVICE_DIR = __dirname/./python-service
|   |-- VENV_PYTHON (cross-platform: Scripts/ o bin/)
|   \-- FALLBACK_PYTHON ('python' o 'python3')
|
|-- [Stato]
|   \-- pythonProcess = null
|
|-- startPythonService()
|   |-- Risolve pythonExe (venv > system fallback)
|   |-- spawn('-m', 'uvicorn', 'main:app', \n|                                     '--host', '
↪ 127.0.0.1', '--port', '8000')
|   |-- Pipe stdout -> console.log('[Python]')
|   \-- Pipe stderr -> console.error('[Python ERR]')
|
|-- stopPythonService()
|   \-- pythonProcess.kill('SIGTERM') + null
|
|-- createWindow()
|   |-- BrowserWindow({1280x800, min 900x600, '#1a2a4a',
|   |   contextIsolation:true, nodeIntegration:false, preload})
|   |-- isDev ? loadURL('localhost:5173') : loadFile('dist/index.
↪ html')
|
|-- [Ciclo di vita app]
|   |-- whenReady: setApplicationMenu(null) + start + createWindow
|   |-- activate (macOS): ricrea finestra se assente
|   |-- window-all-closed: stop + quit (non darwin)
|   \-- will-quit: stop (garantito tutte le piattaforme)
|
\-- [IPC handlers]
    |-- 'get-python-port' -> PYTHON_PORT (8000)
    |-- 'show-open-dialog' -> dialog.showOpenDialog -> {canceled,
↪ filePaths}
    \-- 'read-file-buffer' -> fs.readFileSync -> {ok, data:
↪ number[]}
```

Listing 98: Pseudo-outline di `electron/main.js`.

```
electron/preload.js                                (Bridge -- Contesto isolato  
↔ )  
|  
|-- require: contextBridge, ipcRenderer  
|  
\-- contextBridge.exposeInMainWorld('electronAPI', {  
  |-- platform: process.platform  
  |-- showOpenDialog(options)  
  |   -> ipcRenderer.invoke('show-open-dialog', options)  
  \-- readFileBuffer(filePath)  
      -> ipcRenderer.invoke('read-file-buffer', filePath)  
})
```

Listing 99: Pseudo-outline di `electron/preload.js`.

Glossario dei termini tecnici e di dominio

Il presente glossario raccoglie i termini tecnici, clinici e informatici usati nel documento, organizzati per area tematica.

Classificazione ICF e dominio clinico

ICF	<i>International Classification of Functioning, Disability and Health</i> . Classificazione internazionale dell'OMS (2001) per descrivere e misurare la salute e la disabilità. Organizza le sue componenti in codici gerarchici con prefisso b (Body Functions), s (Body Structures), d (Activities & Participation), e (Environmental Factors).
Qualifier ICF	Valore intero [0–4] che quantifica la gravità di una limitazione per un codice ICF: 0 = nessun problema, 1 = lieve (5–24%), 2 = moderato (25–49%), 3 = grave (50–95%), 4 = completo (96–100%).
ICF Core Set	Sottoinsieme selezionato di codici ICF rilevanti per una condizione clinica specifica (es. Stroke, Low Back Pain, Hearing Loss). L'ontologia Rientr@ include 8 core set.
BFqual	Body Function qualifier: proprietà datatype OWL del descriptor ICF per codici di tipo b* (funzioni corporee).
AP1qual	Activities & Participation qualifier: proprietà datatype OWL per codici di tipo d* (attività e partecipazione).
Health_Condition (HC)	Individuo OWL che aggrega tutti i descrittori ICF di un paziente. Funge da nodo intermedio tra la classe <code>FOAF:Person</code> e i singoli <code>HC_Descriptor</code> .
HC_Descriptor	Individuo OWL che rappresenta la valutazione di un singolo codice ICF per un paziente specifico, contenendo il codice e il qualifier.
O*NET	Occupational Information Network: database del Dipartimento del Lavoro USA che classifica le competenze (Skills), abilità (Abilities) e requisiti delle mansioni lavorative con punteggi 0–100.

SOC	Standard Occupational Classification: sistema di classificazione delle professioni usato per organizzare i profili lavorativi nell'ontologia.
SkAb	Skills e Abilities: entità OWL che rappresentano le competenze e abilità cognitive e motorie richieste da un job (da O*NET).
Job_Descriptor	Individuo OWL che collega un Job a una SkAb con il relativo punteggio O*NET (hasScore). Preesistente nell'ontologia, non creato dall'import.
isSelected	Proprietà booleana OWL di FOAF:Person che indica il worker attivo per il calcolo del matching. Solo un worker può essere selezionato alla volta.
isEvaluatedForJob	Object property OWL che collega una FOAF:Person a uno o più individui JobList:Job , indicando i lavori da valutare per il matching.
Matching	Processo centrale di Rientr@: confronto tra il profilo ICF del paziente e i requisiti O*NET del job, producendo GCS% e AISA% per ogni coppia (worker, job).

Metriche di matching

CS	<i>Criticality Score</i> : misura dell'impatto di una condizione di salute su una specifica Skill/Ability per un job. Formula: $CS = \text{qualifier} \times \text{anchor}$, range $[0, 12]$.
Anchor	Peso dell'importanza di una SkAb per un job, derivato dal punteggio O*NET tramite discretizzazione in 4 livelli: $\leq 24 \rightarrow 0$, $25-49 \rightarrow 1$, $50-74 \rightarrow 2$, $\geq 75 \rightarrow 3$.
GCS%	<i>General Criticality Score %</i> : media normalizzata dei CS di tutte le SkAb valutate per una coppia (worker, job). Formula: $GCS\% = \frac{\sum_i CS_i / 12}{N} \times 100$.
AISA%	<i>Amount of Impaired Skill/Abilities %</i> : percentuale di SkAb con $CS > 0$ sul totale valutato. Formula: $AISA\% = \frac{ \{i: CS_i > 0\} }{N} \times 100$.

Suitability Classificazione dell'idoneità lavorativa basata su GCS% e AISA%: **SUITABLE** ($GCS < \text{soglia gialla}$), **SUITABLE WITH PRECAUTIONS** (tra le due soglie), **NOT SUITABLE** ($GCS > \text{soglia rossa}$). Soglie: $GCS = -0.5 \cdot AISA + 21$ (rossa), $GCS = -0.5 \cdot AISA + 15.5$ (gialla).

Terminologia ontologica e semantica

OWL *Web Ontology Language*: linguaggio W3C per la definizione di ontologie formali. Il profilo OWL 2 DL usato da Rientr@ garantisce decidibilità del ragionamento.

OWL 2 DL Profilo di OWL 2 basato su Description Logic (DL): sottoinsieme decidibile di OWL 2 che garantisce completezza e terminazione del reasoner. Rientr@ usa questo profilo per compatibilità con Pellet.

SWRL *Semantic Web Rule Language*: linguaggio per regole Horn su ontologie OWL. Pellet esegue le regole SWRL e materializza le conclusioni come nuove triple RDF nell'ontologia.

RDF *Resource Description Framework*: modello di dati W3C basato su triple (soggetto, predicato, oggetto). Il file `Rientra.rdf` serializza l'ontologia in formato RDF/XML.

SPARQL *SPARQL Protocol and RDF Query Language*: linguaggio di query per grafi RDF. Il modulo `reasoner.py` usa SPARQL tramite `owlready2.default_world.sparql()` per estrarre i risultati dell'inferenza Pellet.

IRI *Internationalized Resource Identifier*: identificatore univoco di un'entità nel grafo RDF (es. `http://www.stiima.cnr.it/Person-CommonBox#Patient1`).

Local name Frammento identificativo di un IRI dopo il carattere # o l'ultimo /. Usato dal microservizio come chiave nei JSON di risposta (es. `Patient1`, `Job_Carpenter`).

Pellet Reasoner OWL 2 DL con supporto SWRL, sviluppato da Clark & Parsia. Implementato in Java, invocato da owlready2 tramite JVM. Tempi tipici di reasoning su `Rientra.rdf`: 30–120 secondi.

owlready2	Libreria Python per il caricamento e la manipolazione di ontologie OWL, con interfaccia nativa a Pellet. Mantiene il grafo RDF in memoria tramite SQLite interno.
Punning (OWL)	Pattern in cui la stessa entità OWL è dichiarata sia come <code>owl:Class</code> sia come individuo (<code>owl:NamedIndividual</code>). Usato dai job di Rientr@ per essere sia classi (per i <code>Job_Descriptor</code>) sia individui (per <code>isEvaluatedForJob</code>).
Materializzazione	Processo di scrittura esplicita delle triple inferite da Pellet/SWRL nel grafo RDF in memoria. Dopo la materializzazione, le proprietà inferite (es. <code>isVeryImportantFor</code> , <code>hasSpecificCriticality</code>) sono interrogabili via SPARQL come triple normali.

Terminologia RDB2RDF e pipeline di import

RDB2RDF	Standard W3C che definisce come trasformare dati relazionali (tabelle SQL) in triple RDF. Il progetto usa l'approccio R2RML.
R2RML	<i>RDB to RDF Mapping Language</i> : linguaggio dichiarativo W3C (formato Turtle) che specifica la corrispondenza tra strutture relazionali e strutture ontologiche tramite <code>TriplesMap</code> .
TriplesMap	Unità di mapping R2RML che descrive come generare un insieme di triple RDF da una sorgente SQL (tabella o VIEW). Il file <code>rientra_mapping.ttl</code> contiene 5 <code>TriplesMap</code> .
Morph-KGC	Engine R2RML scritto in Python (Universidad Politécnica de Madrid). Esegue il mapping R2RML su un database SQLite e restituisce un oggetto <code>rdflib.Graph</code> in memoria.
Iniezione testuale	Tecnica di integrazione dell'ontologia adottata da <code>importer.py</code> : i blocchi RDF/XML dei nuovi individui vengono inseriti nel file <code>Rientra.rdf</code> tramite sostituzione stringa, anziché ri-serializzare l'intera ontologia con <code>rdflib</code> (incompatibile con Pellet).

Snapshot cache	File JSON pre-calcolato (<code>inference_{fingerprint}.json</code>) che contiene i risultati delle query SPARQL principali. Permette al microservizio di rispondere in millisecondi durante il warm-up di Pellet.
Fingerprint	Hash SHA-256 troncato (16 caratteri) calcolato su percorso, dimensione e timestamp del file ontologico. Usato per invalidare automaticamente la snapshot cache quando il file RDF cambia.

Terminologia Electron e frontend

Main Process	Processo principale di Electron: esegue Node.js completo, gestisce la finestra, avvia il microservizio Python e registra gli handler IPC.
Renderer Process	Processo che esegue il frontend React in un contesto Chromium isolato. Non ha accesso diretto a Node.js per ragioni di sicurezza.
Preload Script	Script eseguito in un contesto intermedio con accesso sia a Node.js sia al DOM del renderer. Usa <code>contextBridge</code> per esporre API native controllate al renderer.
IPC	<i>Inter-Process Communication</i> : meccanismo di comunicazione tra Main Process e Renderer Process in Electron. Implementato tramite <code>ipcMain.handle (main)</code> e <code>ipcRenderer.invoke (renderer)</code> .
contextBridge	API Electron che permette al preload script di esporre funzioni al renderer in modo sicuro, attraversando il confine di isolamento del contesto JavaScript.
contextIsolation	Modalità Electron (default <code>true</code> da Electron 12) che isola il contesto JavaScript del renderer da quello del preload, impedendo al codice React di accedere direttamente alle API Node.js.
HMR	<i>Hot Module Replacement</i> : funzionalità di Vite che aggiorna i moduli JavaScript nel browser in tempo reale senza ricaricare la pagina, preservando lo stato dei componenti React.

- Session cache** Struttura dati in-memory (non `localStorage`) usata da `JobAnalysisView` per ricordare la selezione dei job e le impostazioni di ordinamento per ogni worker visitato durante la sessione.
- Stale closure** Bug React in cui una funzione cattura variabili di stato con i valori del render al momento del mount, non del render corrente. Risolto in `JobAnalysisView` tramite il pattern `liveState.current`.
- Project References (TS)** Funzionalità TypeScript 3.0+ che permette di separare la type-checking per sottoprogetti distinti (es. `renderer browser` vs. `main process Node.js`) con configurazioni TypeScript dedicate.
- Flat config (ESLint)** Formato di configurazione ESLint 9 basato su un modulo JavaScript puro (`eslint.config.js`) anziché file JSON/YAML legacy (`.eslintrc`). Introdotto come default in ESLint 9.

Terminologia API e protocolli

- ASGI** *Asynchronous Server Gateway Interface*: interfaccia Python per server web asincroni. FastAPI è un framework ASGI; Uvicorn è il server ASGI che lo esegue.
- FastAPI** Framework Python ad alte prestazioni per API REST, basato su ASGI. Genera automaticamente documentazione OpenAPI/Swagger dalla definizione degli endpoint e dei modelli Pydantic.
- Pydantic v2** Libreria Python per la validazione e la serializzazione dei dati tramite annotazioni di tipo. FastAPI usa Pydantic per validare automaticamente i request body e serializzare i response model in JSON.
- Lifespan handler** Pattern FastAPI (da v0.95) basato su `@asynccontextmanager` per gestire l'inizializzazione e il cleanup dell'applicazione in modo asincrono. Sostituisce i deprecati `on_event("startup")`.

CORS	<i>Cross-Origin Resource Sharing</i> : meccanismo HTTP che permette a pagine web di fare richieste a un dominio diverso da quello di origine. Il microservizio configura CORS per accettare richieste da React (porta 5173) e Spring Boot (porta 8080).
503 Service Unavailable	Codice HTTP restituito dal microservizio mentre Pellet è in corso. Segnala che il servizio esiste ma non è temporaneamente disponibile, incoraggiando il retry da parte del client.
OpenAPI	Standard per la documentazione delle API REST. FastAPI genera automaticamente la specifica OpenAPI disponibile a <code>/openapi.json</code> , <code>/docs</code> (Swagger UI) e <code>/redoc</code> (ReDoc).
Uvicorn	Server ASGI di produzione per Python, basato su <code>uvloop</code> e <code>httptools</code> . Avviato dal Main Process di Electron tramite <code>spawn('-m', 'uvicorn', 'main:app', ...)</code> .

17 Scenari d'uso e sequenza operativa

17.1 Scenario tipico: valutazione idoneità di un nuovo paziente

Il seguente scenario descrive la sequenza operativa completa dall'avvio dell'applicazione fino alla lettura dei risultati di matching per un paziente appena importato.

1. **Avvio applicazione** — L'utente lancia `start.sh` (o doppio clic sull'app pacchettizzata). Electron avvia il Main Process, che lancia il microservizio Python tramite `spawn` e apre la finestra React.
2. **Warm-up Pellet** — La `WorkersPage` inizia il polling su `GET /status`. La `LoadingScreen` mostra le statistiche dell'ontologia in tempo reale (classi, individui, proprietà) e il timer Pellet. Durata tipica: 30–120 secondi.
3. **Importazione paziente** — L'operatore clicca “Add Worker” e seleziona il file `dataset.sql` tramite il file dialog nativo. Il flusso Electron IPC legge il file e lo invia a `POST /import/workers`. La pipeline RDB2RDF (`importer.py`) inserisce i nuovi individui nell'ontologia. Il microservizio risponde con il riepilogo: persone aggiunte, codici ICF validi, job assegnati.
4. **Selezione del paziente** — L'operatore seleziona il paziente nella sidebar. `POST /workers/select` aggiorna `isSelected` nell'ontologia. La sidebar mostra l'indicatore “switching...” durante l'attesa.
5. **Visualizzazione condizioni ICF** — La tabella ICF mostra le condizioni di salute del paziente con i qualifier [0–4]. L'operatore può ordinare per codice (b-first/d-first), per nome o per gravità, ed espandere le righe per leggere le descrizioni ICF.
6. **Analisi del matching** — L'operatore naviga a “Job Analysis”. `GET /match/{worker_id}` restituisce GCS% e AISA% per tutti i job assegnati. Lo scatter plot mostra la distribuzione dei job nelle tre zone di idoneità.
7. **Dettaglio skill** — L'operatore seleziona un job nello scatter plot. `POST /match/detail` restituisce il breakdown per skill con CS, anchor, qualifier e label di criticità. La tabella è ordinabile per criticità decrescente per identificare immediatamente le skill più problematiche.
8. **Modifica condizioni ICF** — Se l'operatore vuole simulare uno scenario di riabilitazione (miglioramento di alcune condizioni), apre la wizard `Health`

Condition Wizard, seleziona i nuovi codici ICF, assegna i qualifier aggiornati e salva. Il backend applica il batch di modifiche e aggiorna la snapshot cache.

9. **Rivalutazione** — L'operatore torna a "Job Analysis" e rilancia il matching. I risultati riflettono le condizioni ICF aggiornate, permettendo un confronto prima/dopo la riabilitazione.

17.2 Sequenza operativa: aggiornamento dei job di valutazione

1. L'operatore apre la modale "Edit Jobs" in `JobAnalysisView`.
2. Il catalogo completo dei job (`GET /jobs`) viene caricato la prima volta (lazy loading).
3. L'operatore seleziona/deseleziona i job con le checkbox.
4. Salva: `POST /workers/{id}/jobs/update` sostituisce atomicamente tutti i link `isEvaluatedForJob`.
5. Il backend riesegue Pellet (`_refresh_reasoning_artifacts`). Il microservizio torna temporaneamente a stato `loading` (503) per la durata del re-reasoning (30–120 s).
6. Al completamento, il frontend ricarica automaticamente i match results e reimposta il job A al primo della nuova lista.

17.3 Diagramma di sequenza UML semplificato

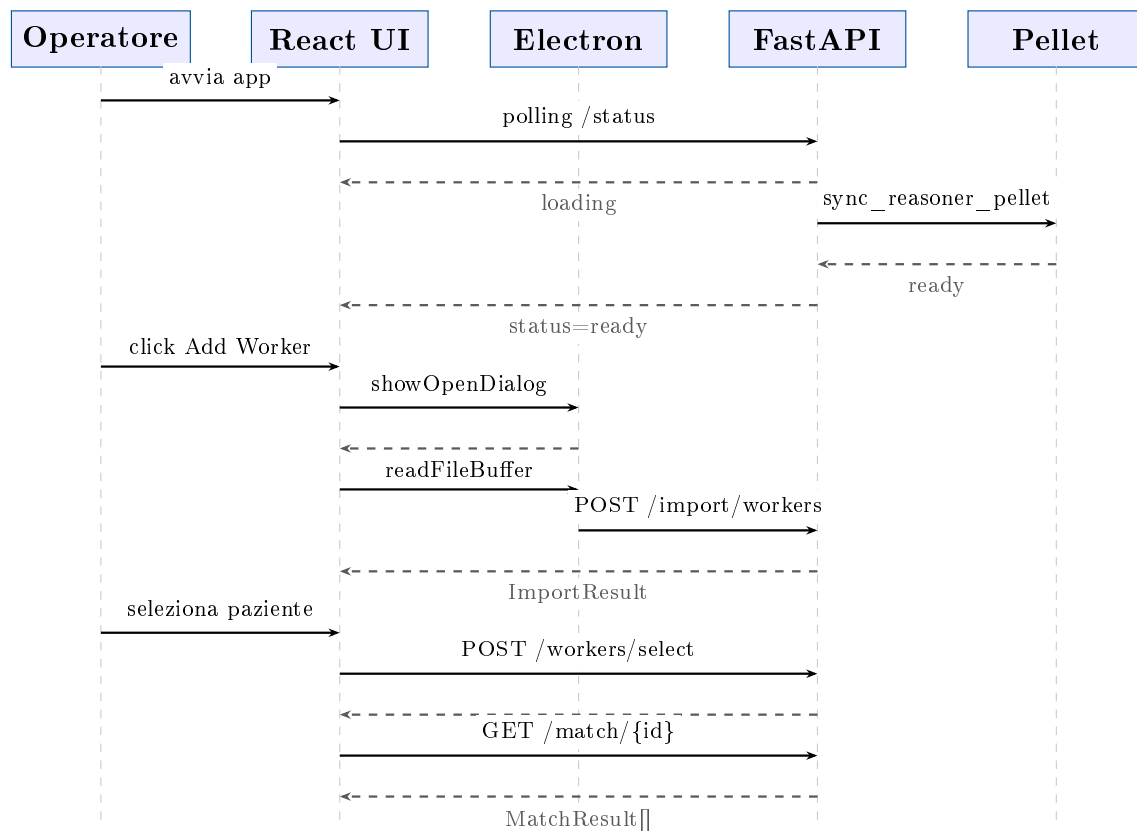


Figure 15: Diagramma di sequenza semplificato per lo scenario di valutazione (parte 1/2).

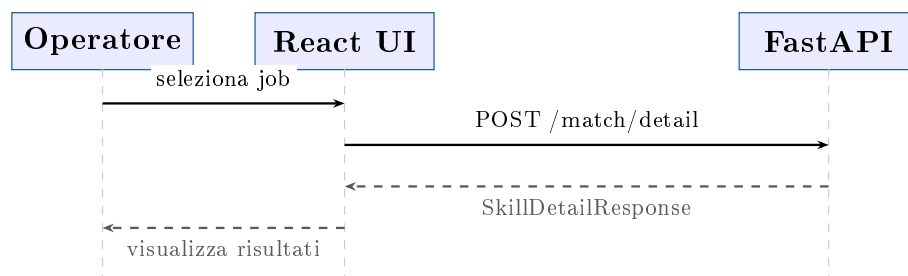


Figure 16: Diagramma di sequenza semplificato per lo scenario di valutazione (parte 2/2).

18 Known issues e debiti tecnici

18.1 Known issues

Table 49: Problemi noti e workaround.

Problema	Descrizione	Workaround
Parallelizzazione Morph-KGC su macOS	Morph-KGC stampa un warning e disabilita la parallelizzazione quando invocato come libreria su darwin.	Atteso e ignorabile; non impatta la correttezza.
Re-reasoning dopo import lento	<code>POST /import/workers</code> non riesegue Pellet automaticamente; il matching dei nuovi pazienti richiede che l'operatore assegni i job e poi aggiorni.	Assegnare i job via "Edit Jobs" triggera il re-reasoning.
Datatype hash in mapping.ttl	Le TriplesMap B/D/S mostrano un datatype con hash SHA invece di <code>xsd:integer</code> per i qualifier.	Funzionale; rigenerare il mapping dopo fix in <code>_generate_mapping()</code> .
Duplicazione <code>parseConditionDescription</code>	La funzione è definita identicamente in <code>WorkersPage</code> e <code>HealthConditionWizard</code> .	Estrarre in <code>src/utils/icfParser.ts</code> .
Click-outside duplicato	Il pattern <code>useClickOutside</code> è ripetuto in tre componenti.	Estrarre in <code>src/hooks/useClickOutside.ts</code> .
CSS custom properties parziali	<code>HealthConditionWizard.css</code> usa <code>var()</code> mentre gli altri CSS usano valori hardcoded: sistema di theming incompleto.	Uniformare dichiarando variabili in <code>:root</code> di <code>index.css</code> .
<code>python-service</code> non bundled	La distribuzione Electron non include il microservizio Python; richiede installazione manuale.	Usare <code>extraResources</code> di electron-builder con Python bundled.
Porta 8000 hardcoded	<code>BASE_URL</code> in <code>semanticService.ts</code> e <code>PYTHON_PORT</code> in <code>main.js</code> non sono sincronizzati automaticamente.	Usare IPC <code>get-python-port</code> per leggere la porta dinamicamente.

18.2 Debiti tecnici

Table 50: Debiti tecnici identificati con priorità e impatto.

Debito tecnico	Priorità	Impatto se risolto
Estrarre <code>useClickOutside</code>	Media	Riduce codice duplicato in 3 componenti; testabilità migliorata.
Estrarre <code>useResizeObserver</code>	Media	Rimuove logica <code>ResizeObserver</code> da <code>JobAnalysisView</code> .
Estrarre <code>icfParser.ts</code>	Alta	Elimina duplicazione critica tra <code>WorkersPage</code> e <code>HealthConditionWizard</code> .
Uniformare CSS custom properties	Bassa	Abilita theming coerente e future variazioni di colore.
Aggiungere <code>tsconfig.app.json</code> / <code>tsconfig.node.json</code>	Media	Documentati ma non analizzati; completare la copertura TS.
Sincronizzare porta via IPC	Media	Elimina il rischio di disallineamento <code>PYTHON_PORT</code> .
Fix datatype hash in <code>mapping.ttl</code>	Bassa	Migliora leggibilità del mapping; nessun impatto funzionale.
Bundling Python in Electron	Alta	Abilita distribuzione self-contained senza installazione manuale.

19 Validazione e testing

19.1 Validazione dell'ontologia

L'ontologia `Rientra.rdf` è stata validata con i seguenti strumenti, come indicato nella documentazione dell'importer (v0.1-test):

Table 51: Strumenti di validazione dell'ontologia e risultati.

Strumento	Versione	Risultato
Protégé	5.6.9	Caricamento e visualizzazione corretti; individui importati visibili nel class browser.
Pellet	2.2.0	Reasoning completato senza <code>InconsistentOntologyException</code> ; proprietà SWRL materializzate.
HermiT	1.4.3.456	Classificazione corretta; nessuna classe insoddisfacibile.
owlready2	≥ 0.46	Caricamento tramite <code>get_ontology().load()</code> ; 1.458 individui.

19.2 Test rapido del microservizio

Dopo l'avvio e il completamento del warm-up (`status = "ready"`), il microservizio può essere verificato con i seguenti comandi `curl`:

```

1 # 1. Verifica readiness
2 curl http://127.0.0.1:8000/status
3
4 # 2. Lista workers
5 curl http://127.0.0.1:8000/workers
6
7 # 3. Lista job
8 curl http://127.0.0.1:8000/jobs
9
10 # 4. Condizioni ICF di un worker
11 curl http://127.0.0.1:8000/health-conditions/Patient1
12
13 # 5. Seleziona worker

```

```
14 curl -X POST http://127.0.0.1:8000/workers/select \  
15     -H "Content-Type: application/json" \  
16     -d '{"worker_id": "Patient1"}'  
17  
18 # 6. Match results  
19 curl http://127.0.0.1:8000/match/Patient1  
20  
21 # 7. Dettaglio skill per coppia (worker, job)  
22 curl -X POST http://127.0.0.1:8000/match/detail \  
23     -H "Content-Type: application/json" \  
24     -d '{"worker_id": "Patient1", "job_id": "Job_AssemblyWorker"}'  
25  
26 # 8. Documentazione interattiva  
27 open http://127.0.0.1:8000/docs
```

Listing 100: Test rapido degli endpoint principali.

La documentazione interattiva Swagger UI (/docs) permette di eseguire ogni endpoint direttamente dal browser, con auto-compilazione dei campi e visualizzazione dello schema JSON. È lo strumento raccomandato per la verifica manuale durante lo sviluppo.

20 Note su provenienza dati, licenze e privacy

20.1 Provenienza dati O*NET

I punteggi di importanza delle Skill e Abilities usati nell'ontologia Rientr@ sono derivati dal database **O*NET** (*Occupational Information Network*), mantenuto dal U.S. Department of Labor/Employment and Training Administration (USDOL/ETA).

Table 52: Dati O*NET nell'ontologia Rientr@.

Aspetto	Dettaglio
Fonte	O*NET OnLine (onetonline.org), USDOL/ETA
Licenza	Pubblico dominio (Creative Commons CC-BY 4.0 per i dati aggregati)
Utilizzo nell'ontologia	Punteggi <code>hasScore</code> [0–100] per ogni <code>Job_Descriptor</code>
Versione	Incorporata staticamente in <code>Rientra.rdf</code> ; aggiornamenti richiedono rigenerazione dell'ontologia
Classificazione SOC	Standard Occupational Classification del Bureau of Labor Statistics

20.2 Dati ICF e OMS

La classificazione ICF è pubblicata dall'**Organizzazione Mondiale della Sanità** (OMS/WHO, 2001). I codici e le descrizioni testuali incluse nell'ontologia sono estratti dalla versione ufficiale ICF e dai core set clinici sviluppati in collaborazione con il *ICF Research Branch*. L'uso è consentito per scopi di ricerca e applicazioni non commerciali in ambito sanitario.

20.3 Dati clinici e privacy

Dati sensibili — Note di conformità

I dati importati tramite `POST /import/workers` includono informazioni cliniche personali (codici ICF con qualifier, dati anagrafici). Il sistema nella versione attuale (v0.1-test) **non implementa**:

- Autenticazione o autorizzazione degli utenti.
- Cifratura dei dati in transito (HTTP, non HTTPS).
- Controllo di accesso ai file RDF su disco.
- Anonimizzazione o pseudonimizzazione automatica.

Per un deploy in ambiente clinico è obbligatorio valutare la conformità al GDPR (Regolamento UE 2016/679) e alle normative nazionali sulla protezione dei dati sanitari, introducendo almeno: autenticazione locale, cifratura TLS, e gestione dei log di accesso.

21 Configurazione IDE e ambiente di sviluppo consigliato

21.1 Strumenti consigliati

Table 53: Strumenti consigliati per lo sviluppo del progetto Rientr@.

Strumento	Versione	Utilizzo
Visual Studio Code	≥ 1.85	IDE principale per TypeScript/React e Python.
Protégé	5.6.x	Editor ontologie OWL; validazione e browsing del grafo RDF.
Insomnia / Postman	qualsiasi	Test manuale degli endpoint REST; importazione da <code>/openapi.json</code> .
Java 11+ (Temurin)	≥ 11	Runtime richiesto da Pellet; Eclipse Temurin è la distribuzione raccomandata (gratuita, LTS).
Node.js	≥ 20 LTS	Runtime per Electron e Vite.
Python	3.11–3.12	Runtime del microservizio; evitare 3.13+ finché owlready2 non aggiorna il supporto.

21.2 Estensioni VSCode raccomandate

Table 54: Estensioni VSCode per il progetto Rientr@.

Estensione	Motivazione
<code>ms-python.python</code>	IntelliSense, debugging e linting Python.
<code>ms-python.pylance</code>	Type checking Python avanzato; rileva errori di tipo in <code>owlready2</code> .
<code>dbaeumer.vscode-eslint</code>	Integrazione ESLint con flat config; mostra warning inline.
<code>bradlc.vscode-tailwindcss</code>	Autocompletamento classi Tailwind nelle stringhe TypeScript.
<code>esbenp.prettier-vscode</code>	Formattazione automatica TypeScript/JSON/CSS.
<code>ms-vscode.vscode-typescript-next</code>	Supporto TypeScript sperimentale; utile con TS 5.9+.
<code>GraphQL.vscode-graphql-syntax</code>	Syntax highlighting per file Turtle (<code>.ttl</code>) del mapping R2RML.
<code>redhat.vscode-yaml</code>	Validazione YAML per eventuali file di configurazione.

21.3 Configurazione `.vscode/settings.json` consigliata

```

1 {
2   "python.defaultInterpreterPath": "./python-service/.venv/bin/
   ↪ python3",
3   "python.analysis.typeCheckingMode": "off", // owlready2 no stubs
4   "editor.formatOnSave": true,
5   "editor.defaultFormatter": "esbenp.prettier-vscode",
6   "[python]": {
7     "editor.defaultFormatter": "ms-python.black-formatter"
8   },
9   "eslint.useFlatConfig": true,
10  "files.associations": { "*.ttl": "turtle" },
11  "editor.rulers": [88] // PEP 8 line length
12 }
```

Listing 101: Impostazioni VSCode raccomandate per Rientr@.